University Project Idea

So I want to build a project

Mm-hmm.

for my university

Okay. Great. Tell me about it. What's on your mind?

I have to make a project for my university, I have to build a website

Okay. Mm, nice. What's the idea? Is there something you're passionate about, or is there a problem you've noticed on campus?

No, just It's a bus ticket. Booking website

Mm-hmm.

For my database course

Got it. So bus ticketing. What's the one thing you'd want that site to do really well?

Okay so I want you to hear me out all the features that I'm going to tell you

Sure. Sure.

Hear me out and note it down. so that you can convert it to a professional prompt and so-and I can use that prompt to give my AI agent and tell him to tell it to. bring- just build me the website

Okay, all right, I'm ready. Go ahead. Just start when you're ready.

Yes, so there is a that will be a website or a-app you can say

Mm-hmm.

and It will control by the admin

Right.

and it is a universal app which means multiple university can sign into it and you know take the take its partnership

Mm-hmm.

to integrate in their transport system

Mm-hmm.

So the admin will control uh the implementation of adding and removing universities- the universities have to approach the admin so that admin can add their system or add them into the website so that they can use it

Right, right.

So then

Take your time.

This is the admin- admin jobs and The admin has its own admin ID

Mm-hmm.

which is a unique ID uh-own email and name And as I said, the admin can modify uh- universities

Mm-hmm.

uh add or remove them and University will have

Mm-hmm.

- Buses. which belongs to the university One certain university has to have multiple buses but that bus is only goes with the university

Right, right.

belongs to the certain university. The university

Mm-hmm.

is a different entity. The university will have its university ID, university name address, and contact

Right. Mm-hmm.

And the buses is a weak entity which depends and belongs to the university It will have. a bus ID It's a composite number. It's a composite attribute which contains registration number and tax

number

Right. Mm-hmm. Right.

which also have standing center and seat number

Mm-hmm.

also the bus type uh here bus type means the bus is belongs to two type of passengers, one is for students and one is for faculty Okay, then university will also have passengers The passenger- uh- one passenger is- must belong to

Mm-hmm.

one university uh. The passenger has is also a weak entity

Right. Mm-hmm.

uh. The passenger has its partial key which is passenger ID and attribute like name And then

Mm-hmm.

It's a superclass, and the- passenger uh- is can two types- there are two types of passengers like student and faculty

Mm-hmm. Mm-hmm.

So, a passenger can either be a student or a faculty- it can't be both cannot be both

Right. Mm-hmm. Got it.

Okay So there is also a feature like The passenger can submit a- complaints- many num- one passenger can submit many number of complaints about the services

Mm-hmm. Mm-hmm.

uh. to the university and one university can take multiple number of complaints The complaints have its own complaint ID

Mm-hmm.

And there is also a feature about the passenger- one passenger can sell his ticket or share his ticket to other passengers

Got it.

the passenger can book tickets um- The ticket has its own booking ID which is a unique entity

Right. Mm-hmm.

unique attribute uh- there- the tickets in the tickets there will be a booking date, seat number, and status

Mm-hmm. Right.

uh- when- while booking The bill will be counted

Mm-hmm.

and will be added to the certain ID in every semester

Right. Mm-hmm.

In a semestry- and uh Then the passenger can pick a- favorite route uh- oh. Before that, I want to mention that one university can manage uh a schedule, multiple schedules and multiple routes

Mm-hmm. Mm-hmm.

Then uh- the route has its start location

Mm-hmm.

And route ID which is a unique key attribute and end location

Right.

Then uh. the schedule has its unique schedule ID, departure time, arrival time, and date

Mm-hmm. Right.

uh One route can follow multiple schedule

Mm-hmm.

And one schedule contains multiple tickets. And passenger, one passenger can book only one ticket at a time

Okay.

Okay

Mm.

So that's the thing and uh

Hmm. Okay, that's

one route can uh and be a assigned to one buses

Got it. Got it. Okay.

one bus only one

Right. That sounds smooth, and honestly, a really strong base. Just keep going if there's more.

uh- multiple passengers can uh. pick multiple favorites route

Mm-hmm.

uh- sohere's the gist idea, just note it down and convert it to a professional prompt

On it. Perfect. I've got everything. So here's a clean prompt you can paste into your AI agent. Build a full-stack multi-tenant university bus ticket management web app. There's a central admin who onboards and manages universities; each university manages its own buses, routes, schedules, and passengers. Buses belong to one university and one route, have a composite ID, registration plus tax number, seat and standing capacity, and type (student or faculty). Passengers belong to one university and are either student or faculty, not both. They have a passenger ID and name, can submit multiple complaints, book only one active ticket at a time, and mark multiple favorite routes. Each favorite route can be shared across passengers. Routes have start and end locations; routes have multiple schedules. Schedules include schedule ID, departure time, arrival time, date, and contain many tickets. Tickets have booking ID, booking date, seat number, status; fare is auto-calculated and accumulated in the passenger's semester bill. Allow ticket transfer or sale between eligible passengers. Build with role-based dashboards, authentication, CRUD for all entities, validation of constraints, and a clean, responsive UI.

Okay, so convert it into text

Here you go. Prompt. Build a complete full-stack, database-driven web application for a multi-tenant university bus ticket booking and transport management system. The system has a central admin who approves and manages multiple universities. Each university independently manages its buses, routes, schedules, passengers, and complaints. Buses are weak entities belonging to one university, with a composite ID, registration number plus tax number, seat capacity, standing capacity, and type. Student or faculty. Each bus is assigned to one route. Passengers are weak entities belonging to one university, with passenger ID and name. Passenger is a superclass with two disjoint subtypes, student and faculty.

A passenger can only be one type. Passengers can submit multiple complaints, and each complaint is linked to a university. Passengers can book tickets; only one active ticket per passenger at a time. Tickets include booking ID, booking date, seat number, and status; fare is calculated automatically and added to the passenger's semester bill. Allow ticket transfer or sale between eligible passengers. Passengers can also mark multiple favorite routes; routes can be favorited by multiple passengers. Routes include route ID, start location and end location; one route has multiple schedules. Schedules include schedule ID, departure time, arrival time, date; one schedule has many tickets. Implement authentication, role-based access, CRUD for all entities, proper relationships and constraints, and a clean responsive UI. Include ER diagram, relational schema, and sample data. Deliver a well-structured, scalable, fully functional project suitable for a university database course.

Here you go. Prompt. Build a complete full-stack, database-driven web application for a multi-tenant university bus ticket booking and transport management system. The system has a central admin who approves and manages multiple universities. Each university independently manages its buses, routes, schedules, passengers, and complaints. Buses are weak entities belonging to one university, with a composite ID, registration number plus tax number, seat capacity, standing capacity, and type. Student or faculty. Each bus is assigned to one route. Passengers are weak entities belonging to one university, with passenger ID and name. Passenger is a superclass with two disjoint subtypes, student and faculty. A passenger can only be one type. Passengers can submit multiple complaints, and each complaint is linked to a university. Passengers can book tickets; only one active ticket per passenger at a time. Tickets include booking ID, booking date, seat number, and status; fare is calculated automatically and added to the passenger's semester bill. Allow ticket transfer or sale between eligible passengers. Passengers can also mark multiple favorite routes; routes can be favorited by multiple passengers. Routes include route ID, start location and end location; one route has multiple schedules. Schedules include schedule ID, departure time, arrival time, date; one schedule has many tickets. Implement authentication, role-based access, CRUD for all entities, proper relationships and constraints, and a clean responsive UI. Include ER diagram, relational schema, and sample data. Deliver a well-structured, scalable, fully functional project suitable for a university database course.
Show moreShow less

Yes — that captures the core system you described.

One thing I'd change before giving it to a coding agent: make the **database constraints, user roles, booking flow, ticket-transfer rules, and expected tech deliverables** much more explicit. Otherwise, an AI coding agent may make its own assumptions and build something that looks good but doesn't match your database-course requirements.

If you want, send me your preferred tech stack next — for example **React + Node.js + MySQL**, **PHP + MySQL**, or **Django + PostgreSQL** — and I can turn this into a much more detailed, production-style master prompt that tells the agent exactly what database, backend, frontend, pages, tables, relationships, and features to build.

Also use HTML, CSS, and JavaScript for the frontend and backend, and use PHP for database management. implement this in the final prompt too

Build a **complete full-stack, database-driven web application** for a university database course project. The application should be a **Multi-University Bus Ticket Booking and Transport Management System** that can be adopted by multiple universities.

The application must be functional, responsive, well-structured, and designed around proper database concepts such as **entities, weak entities, primary keys, partial keys, composite attributes, foreign keys, specialization/generalization, cardinality, participation constraints, and many-to-many relationships**.

1. Required Technology Stack

Use the following technologies:

- **HTML5** for frontend structure.
- **CSS3** for styling and responsive design.
- **Vanilla JavaScript** for frontend interactions, validation, dynamic UI behavior, AJAX/fetch requests, and application logic where appropriate.
- **PHP** for server-side/backend processing, authentication, CRUD operations, session handling, API/database communication, and database management.
- **MySQL** as the relational database.
- Use **PHP PDO or MySQLi with prepared statements** for secure database operations.
- Do not use React, Vue, Angular, Laravel, Django, Node.js, or other major frameworks unless absolutely necessary.
- Organize HTML, CSS, JavaScript, PHP, SQL, images, and other assets into a clean project folder structure.

The application should be able to run locally using an environment such as **XAMPP/WAMP/LAMP**.

---

2. System Concept

The system is a **universal university transportation platform**.

Multiple universities can participate in the system, but they cannot register themselves directly as fully active institutions.

A university must first approach the **System Admin**, and the admin will add/approve the university before it can use the transportation management system.

Each university operates independently inside the platform and manages its own:

- Buses
- Passengers
- Students
- Faculty members
- Routes

- Schedules
- Tickets
- Bookings
- Complaints
- Semester transportation bills

Data belonging to one university must not accidentally appear in another university's system.

---

3. User Roles

Implement proper **Role-Based Access Control (RBAC)**.

The primary roles are:

A. System Admin

The central administrator manages participating universities.

Admin attributes:

- Admin ID — Primary Key / Unique
- Name
- Email
- Password

Admin functionalities:

- Secure admin login/logout
- Admin dashboard
- Add universities
- View universities
- Edit university information
- Remove/deactivate universities
- Search universities
- View basic statistics across the entire system
- Manage the onboarding of universities

B. University

University is an independent entity.

University attributes:

- University ID — Primary Key

- University Name
- Address
- Contact Information
- Email/login information if necessary
- Status

A university can have:

- Multiple buses
- Multiple passengers
- Multiple routes
- Multiple schedules
- Multiple complaints

Each of those records must belong to a specific university.

Create a university-side dashboard from which authorized university personnel can manage transportation data.

---

4. Bus Entity

A **Bus is dependent on a University** and should be represented according to the database-course requirement as a **weak entity where appropriate**.

A university can have many buses.

A bus belongs to **exactly one university**.

The Bus ID should be represented using the following identifying information:

- Registration Number
- Tax Number

Treat these according to the project requirement as a **composite identifying attribute/key** associated with the bus.

Bus attributes should include:

- Registration Number
- Tax Number
- Seat Capacity / Number of Seats
- Standing Capacity / Standing Passenger Limit
- Bus Type
- University ID — Foreign Key

Bus Type

A bus can be designated for:

- Student
- Faculty

Enforce this using an appropriate ENUM/check constraint or normalized design.

Provide functionality to:

- Add bus
- View buses
- Edit bus
- Delete/deactivate bus
- Search/filter buses
- View bus capacity
- View assigned route/schedule

---

5. Passenger Superclass

Passenger belongs to a particular university.

A passenger cannot exist independently of a university within this system.

Passenger attributes:

- Passenger ID — Partial/identifying key according to the ER design
- University ID
- Name
- Email
- Password/login credentials if required
- Passenger Type

Passenger should be modeled as a **superclass**.

There are exactly two passenger subclasses:

1. Student
2. Faculty

The specialization must be:

- **Disjoint:** A passenger cannot be both Student and Faculty.
- Ideally **Total:** Every passenger must be either Student or Faculty.

Implement this properly in both the database and application logic.

---

6. Student Subclass

Student inherits information from Passenger.

Include fields appropriate to the university project, such as:

- Passenger ID
- University ID
- Student ID if needed
- Department
- Semester

Do not duplicate unnecessary superclass data.

---

7. Faculty Subclass

Faculty also inherits information from Passenger.

Possible attributes:

- Passenger ID
- University ID
- Faculty ID if necessary
- Department
- Designation

Again, avoid unnecessary duplicated data.

---

8. Route Entity

Each university can maintain multiple routes.

Route attributes:

- Route ID — Primary Key
- University ID — Foreign Key
- Start Location
- End Location
- Route Name if useful
- Fare if the fare is route-based
- Status

Provide:

- Add route
- View route
- Edit route
- Delete/deactivate route
- Search route
- Filter by start/end location

A passenger may select multiple favorite routes.

A route can also be favorited by multiple passengers.

Therefore:

Passenger ↔ Route is a Many-to-Many relationship for Favorite Routes.

Create an associative table such as:

`favorite_routes`

containing appropriate foreign keys.

---

9. Bus and Route Assignment

A bus can be assigned to a route.

Follow the project requirement that a bus should have **one assigned route at a given assignment context**.

Prevent conflicting assignments.

If necessary for database correctness and history, create a dedicated relation such as:

`bus_route_assignment`

rather than unnecessarily duplicating route information inside the Bus table.

---

10. Schedule Entity

A university can manage multiple schedules.

Each route may have multiple schedules.

Schedule attributes:

- Schedule ID — Primary Key
- Route ID — Foreign Key
- Bus reference
- University ID if necessary for tenant isolation
- Departure Time
- Arrival Time
- Schedule Date

- Availability/Status

Relationship:

Route 1 : N Schedule

One route can have many schedules.

Each schedule must belong to one route.

A bus should be associated with the appropriate schedule so passengers know which physical bus will operate that trip.

Provide:

- Create schedule
- View schedules
- Update schedule
- Delete/cancel schedule
- Filter schedules by date
- Search by route
- View available seats

---

11. Ticket / Booking Entity

Passengers can book tickets for available schedules.

Ticket attributes:

- Booking ID — Primary Key and unique
- Passenger ID — Foreign Key
- Schedule ID — Foreign Key
- Booking Date
- Seat Number
- Status
- Fare/Amount
- Original Passenger ID if needed for transfer history

Possible ticket statuses:

- Booked
- Confirmed
- Cancelled
- Transferred
- Sold

- Completed

A schedule can contain/book multiple tickets.

Relationship:

Schedule 1 : N Ticket

A passenger can make bookings, but implement the stated project rule:

> A passenger cannot hold more than one conflicting active ticket at the same time.

At minimum, prevent duplicate/invalid simultaneous booking for the same relevant journey/schedule.

Do not allow two passengers to successfully reserve the same seat on the same schedule.

Use database constraints and server-side transactions to prevent double booking.

---

12. Ticket Booking Flow

The booking flow should work like this:

1. Passenger logs in.
2. Passenger selects/searches a route.
3. Passenger selects a date.
4. Application displays available schedules.
5. Passenger chooses a schedule.
6. Application displays available seats.
7. Passenger selects an available seat.
8. System calculates the fare.
9. Passenger confirms booking.
10. Booking record is inserted into the database.
11. Seat becomes unavailable for that schedule.
12. Fare is added to the passenger's semester transportation bill.
13. Booking confirmation is displayed.

Use proper PHP validation and SQL transactions.

---

13. Ticket Transfer / Ticket Selling Feature

A passenger should be able to **transfer or sell a booked ticket to another eligible passenger**.

Rules:

- Both passengers must be valid users.

- Preferably both should belong to the same university unless the business logic explicitly permits otherwise.
- Student-designated tickets/buses should only be transferable to eligible students.
- Faculty-designated tickets/buses should only be transferable to eligible faculty members.
- Completed, expired, or cancelled tickets cannot be transferred.
- A transfer must not violate the recipient's active-booking restrictions.

Store transfer history.

Create a table such as:

`ticket_transfers`

with fields such as:

- Transfer ID
- Booking ID
- From Passenger ID
- To Passenger ID
- Transfer Type — Transfer/Sale
- Amount if sold
- Transfer Date
- Status

Keep historical records instead of simply overwriting all ownership information.

---

14. Complaint Entity

Passengers can submit complaints regarding transportation services.

One passenger can submit multiple complaints.

One university can receive multiple complaints.

Complaint attributes:

- Complaint ID — Primary Key
- Passenger ID — Foreign Key
- University ID — Foreign Key
- Subject
- Complaint Description
- Submission Date
- Status
- University Response if appropriate

Possible statuses:

- Submitted
- Under Review
- Resolved
- Rejected/Closed

Relationships:

Passenger 1 : N Complaint

and

University 1 : N Complaint

Passengers should be able to view their complaint history and status.

University personnel should be able to review and update complaints.

---

15. Semester Billing

When a passenger books a ticket, the cost should be added to their university transportation bill for the corresponding academic semester.

Implement a semester billing system.

Possible entities:

Semester

- Semester ID
- University ID
- Semester Name
- Start Date
- End Date

Semester Bill

- Bill ID
- Passenger ID
- Semester ID
- Total Amount
- Status
- Last Updated

Possible statuses:

- Pending

- Partially Paid
- Paid

Every valid ticket charge should contribute to the correct passenger's semester transportation total.

Avoid manually storing inconsistent totals where possible. Maintain transaction records or calculate totals safely.

Consider creating:

`billing_transactions`

with:

- Transaction ID
- Passenger ID
- Booking ID
- Semester ID
- Amount
- Transaction Type
- Date

---

16. Authentication

Implement secure authentication.

Provide separate experiences according to roles.

At minimum support:

- System Admin login
- University-authorized account login
- Passenger login

Use:

- PHP sessions
- **`password_hash()`**
- **`password_verify()`**
- Prepared SQL statements
- Server-side authorization checks

Never store plain-text passwords.

Users must not gain access to dashboards simply by typing the dashboard URL manually.

---

17. Multi-University Data Isolation

This is extremely important.

The website supports multiple universities.

Every university should only be able to access records associated with its own University ID.

For example:

University A must not be able to:

- Modify University B's buses
- See University B's passenger records
- Edit University B's schedules
- View University B's private complaints
- Modify University B's routes

Implement tenant/data isolation in PHP queries and authorization logic.

The System Admin can have global access where appropriate.

---

18. Passenger Dashboard

Create an attractive passenger dashboard.

Include:

- Passenger profile
- Student/Faculty status
- Search bus routes
- View available schedules
- Book ticket
- Select seat
- View current booking
- View booking history
- Cancel booking when permitted
- Transfer ticket
- Sell ticket
- View transfer history
- Favorite/unfavorite routes
- View favorite routes
- Submit complaint
- View complaint status
- View semester transportation bill
- Logout

19. University Dashboard

Include:

- Dashboard statistics
- Manage buses
- Manage routes
- Manage schedules
- Manage passengers
- View students
- View faculty
- View bookings
- View complaints
- Respond to complaints
- View semester billing information
- View bus capacity
- View schedule occupancy
- Search/filter information
- Logout

20. System Admin Dashboard

Include:

- Total universities
- Active universities
- Total registered passengers
- Total buses
- University management
- Add university
- Edit university
- Deactivate/remove university
- Search universities
- View university details
- View basic system-wide statistics
- Logout

21. Website Pages

Create all necessary pages, including a polished public landing page.

Suggested pages include:

- Home
- About
- How It Works
- Login
- Admin Login
- Passenger Login
- University Login
- Passenger Registration if appropriate
- Passenger Dashboard
- Admin Dashboard
- University Dashboard
- Routes
- Schedules
- Bus Management
- Route Management
- Schedule Management
- Passenger Management
- Ticket Booking
- Seat Selection
- My Tickets
- Ticket Transfer
- Favorite Routes
- Complaints
- Semester Billing
- Profile
- Logout

You may create additional pages when needed.

---

22. Frontend Design

Make the UI look like a real modern university transportation platform rather than a basic database assignment.

Use:

- Professional navigation bar
- Sidebar for dashboards
- Cards
- Data tables
- Forms
- Search bars
- Filters
- Confirmation dialogs
- Status badges
- Toast/alert messages
- Responsive layouts
- Clean typography
- Consistent spacing
- Appropriate transportation/university visual theme
- Mobile-friendly responsive CSS

Use JavaScript for:

- Form validation
- Interactive seat selection
- AJAX/fetch requests
- Search/filter interactions
- Modal dialogs
- Confirmation prompts
- Dynamic route/schedule loading
- Dynamic booking totals
- Notifications
- UI interactions

Do not rely only on JavaScript validation. All important validation must also happen in PHP.

---

23. Seat Selection

Create a visual bus seat selection interface using HTML, CSS, and JavaScript.

Clearly distinguish:

- Available seats

- Selected seat
- Already booked seats
- Unavailable seats

Booked-seat data must come from the MySQL database.

The frontend cannot be the ultimate authority over seat availability.

PHP must verify the seat again immediately before booking.

---

24. Database Requirements

Design a properly normalized relational database, preferably up to **Third Normal Form (3NF)** where appropriate.

Possible tables include:

- admins
- universities
- university_users
- buses
- passengers
- students
- faculty
- routes
- bus_route_assignments
- schedules
- tickets/bookings
- favorite_routes
- complaints
- ticket_transfers
- semesters
- semester_bills
- billing_transactions

Modify table names/structure when necessary for better normalization.

Use:

- Primary Keys
- Composite Keys where appropriate
- Foreign Keys
- Unique Constraints

- NOT NULL
- CHECK/ENUM constraints where supported
- Referential integrity
- Appropriate `ON DELETE` / `ON UPDATE` behavior
- Indexes on frequently searched foreign keys and fields

---

25. ER Diagram Requirements

Provide an ER diagram or ER-diagram specification showing:

- Admin
- University
- Bus
- Passenger
- Student
- Faculty
- Route
- Schedule
- Ticket
- Complaint
- Semester/Billing
- Favorite Route
- Ticket Transfer

Clearly identify:

- Strong entities
- Weak entities
- Primary keys
- Partial keys
- Composite attributes
- Foreign keys
- Cardinalities
- Participation
- Superclass/subclass relationship
- Disjoint constraint
- One-to-many relationships
- Many-to-many relationships

The ER design should be academically explainable during a university project presentation/viva.

---

26. Relational Schema

After designing the ER model, provide the relational schema.

Show tables in a format similar to:

`UNIVERSITY(University_ID PK, University_Name, Address, Contact)`

`ROUTE(Route_ID PK, University_ID FK, Start_Location, End_Location, Fare)`

Continue this for every table.

Clearly mark:

- PK
- FK
- Composite PK
- Unique keys

---

27. SQL Database File

Create a complete MySQL **`.sql`** file containing:

- **`CREATE DATABASE`**
- **`USE`**
- **`CREATE TABLE`**
- Constraints
- Foreign keys
- Indexes
- Sample **`INSERT`** statements
- Demo users
- Demo universities
- Demo buses
- Demo passengers
- Demo students
- Demo faculty
- Demo routes
- Demo schedules
- Demo bookings
- Demo complaints

The sample data should make it possible to demonstrate the application immediately after installation.

---

28. Database Course Features

Since this is specifically a **Database Course Project**, prioritize database quality.

Where academically useful, demonstrate:

- Complex SQL joins
- Aggregate functions
- GROUP BY
- Subqueries
- Views
- Transactions
- Constraints
- Indexing

If appropriate, also include a small number of useful:

- Views
- Stored procedures
- Triggers

Do not add them merely for decoration. Explain what they accomplish.

Example possibilities:

- Trigger or transaction logic related to semester billing
- View for available schedules
- View for passenger booking history
- Aggregation for university transportation statistics

---

29. Security

Implement:

- Password hashing
- Prepared statements
- SQL injection prevention
- XSS protection using proper output escaping
- Session security
- Role authorization

- Input sanitization and validation
- CSRF protection for important state-changing forms where practical

Never expose database passwords or sensitive credentials to browser-side JavaScript.

---

30. Validation and Business Rules

Enforce rules on the server side.

Examples:

- University ID must exist before adding dependent data.
- Bus must belong to one university.
- Passenger must belong to one university.
- Passenger must be either Student or Faculty, not both.
- Routes and schedules must belong to the correct university.
- Arrival time must logically follow departure time where applicable.
- Cannot book an unavailable seat.
- Cannot double-book the same seat.
- Cannot make prohibited simultaneous bookings.
- Cannot transfer invalid/expired/cancelled tickets.
- Cannot transfer a student-only ticket to faculty or vice versa.
- Complaint must belong to the passenger's university.
- Favorite route must be from an eligible university context.
- Semester billing must reflect valid ticket transactions.

---

31. CRUD Operations

Implement complete CRUD where appropriate:

- Create
- Read
- Update
- Delete / deactivate

CRUD should exist for major manageable entities including:

- Universities
- Buses
- Routes
- Schedules

- Passengers
- Complaints where appropriate

Ticket/history records should generally preserve historical integrity rather than being permanently deleted without reason.

---

32. Search, Sort, and Filters

Add useful search/filter options.

For example:

Routes

Filter by:

- Start location
- End location

Schedules

Filter by:

- Route
- Date
- Bus type

Buses

Filter by:

- University
- Bus type
- Availability

Passengers

Filter by:

- Student
- Faculty
- Department

Tickets

Filter by:

- Date

- Status
- Passenger
- Schedule

---

33. Dashboard Statistics

Create useful database-driven statistics.

Examples:

System Admin

- Number of universities
- Number of buses
- Number of passengers
- Number of bookings

University

- Number of buses
- Number of students
- Number of faculty
- Today's schedules
- Today's bookings
- Pending complaints
- Total semester transport charges

Passenger

- Active ticket
- Upcoming journey
- Favorite routes
- Semester transportation bill
- Complaint status

All statistics must come from actual database queries.

---

34. Project Structure

Use a professional folder structure similar to:

```
university-bus-system/
├── index.php
```

```
├── login.php
├── logout.php
│
├── admin/
├── university/
├── passenger/
│
├── api/
│
├── config/
│   └── database.php
│
├── includes/
│   ├── auth.php
│   ├── header.php
│   ├── footer.php
│   └── functions.php
│
├── assets/
│   ├── css/
│   │   └── style.css
│   ├── js/
│   │   └── app.js
│   └── images/
│
├── database/
│   └── university_bus_system.sql
│
└── README.md
```

You may improve the structure where necessary.

---

35. README

Create a complete **`README.md`** containing:

- Project name
- Project overview
- Features
- Technologies
- Requirements
- XAMPP/WAMP setup instructions
- Database import instructions
- Configuration instructions
- Demo credentials
- Folder structure
- Main database relationships
- How to run the project

---

36. Important Implementation Instruction

Do not generate only mockups or static HTML pages.

Build the actual working application.

Every important feature should connect to the MySQL database through PHP.

For example:

- Logging in must check database credentials.
- Adding a bus must insert it into MySQL.
- Updating routes must modify database records.
- Schedule listings must come from MySQL.
- Seat availability must come from ticket records.
- Booking must create a database record.
- Favorite routes must persist in the database.
- Complaints must persist in the database.
- Ticket transfers must update/create database records safely.
- Semester bills must derive from actual booking transactions.

Do not hard-code fake dashboard numbers or transportation records into HTML/JavaScript.

---

37. Code Quality

Write:

- Clean code
- Meaningful variable names
- Reusable PHP functions
- Proper comments where necessary
- Consistent naming conventions
- Modular CSS
- Organized JavaScript
- Secure SQL
- Reusable page components

Avoid unnecessary duplication.

---

38. Final Deliverables

Generate the complete project, including:

1. Full frontend using HTML, CSS, and JavaScript.
2. PHP backend.
3. MySQL database.
4. SQL setup file.

5. Authentication system.
6. System Admin dashboard.
7. University dashboard.
8. Passenger dashboard.
9. Student/Faculty specialization.
10. Bus management.
11. Route management.
12. Schedule management.
13. Ticket booking system.
14. Visual seat-selection system.
15. Ticket cancellation where appropriate.
16. Ticket transfer/selling functionality.
17. Favorite routes.
18. Complaint management.
19. Semester transportation billing.
20. Multi-university data isolation.
21. Complete CRUD functionality.
22. Search and filtering.
23. Responsive design.
24. ER diagram/specification.
25. Relational schema.
26. Sample database data.
27. README and installation instructions.

---

39. Development Order

Build the project systematically in this order:

1. Analyze requirements.
2. Design the ER model.
3. Convert the ER model into a normalized relational schema.
4. Create the MySQL database.
5. Add foreign keys and constraints.
6. Insert sample data.
7. Create PHP database connection.
8. Implement authentication and authorization.

9. Implement System Admin functionality.
10. Implement university management.
11. Implement passenger/student/faculty management.
12. Implement buses.
13. Implement routes.
14. Implement schedules.
15. Implement booking and seat availability.
16. Implement semester billing.
17. Implement favorite routes.
18. Implement complaints.
19. Implement ticket transfer/selling.
20. Build dashboards and statistics.
21. Improve frontend styling and responsiveness.
22. Test database constraints and security.
23. Fix bugs.
24. Create README/documentation.

Do not skip unfinished modules just to make the interface look complete.

---

40. Final Goal

The finished application should demonstrate both **database theory and practical web development**.

It should be polished enough to present as a university project while making the following concepts easy to explain during evaluation:

- Strong entity
- Weak entity
- Composite key/attribute
- Partial key
- Superclass
- Subclass
- Disjoint specialization
- One-to-many relationship
- Many-to-many relationship
- Primary key
- Foreign key

- Referential integrity
- Normalization
- Transactions
- SQL queries
- Authentication
- CRUD operations
- Database-backed business rules

Prioritize **correct database architecture, functionality, data integrity, security, and clear code** over unnecessary visual complexity.

Do not merely explain how to build the application. **Generate the actual complete working project and all required source files.**

Section06_Group06_cse370 project ideas.pdf
PDF
read these features and also implement them into your final prompt and rewrite the whole prompt again

I reviewed the PDF and merged its additional requirements into the specification, including the route dashboard with seat-availability bars and booking alerts, the **10-row × 4-seat interactive seat map**, QR-code confirmation/download, up to **10 standing slots**, booking cancellation/history, admin occupancy statistics, profile controls, notification preferences, and password verification. I also incorporated the notification bell, complaints, favorite-route prioritization, and ticket sharing/selling requirements.

Section06_Group06_cse370 projec…

I've also made the requirements more precise so an AI coding agent is less likely to invent conflicting database relationships.

Section06_Group06_cse370 projec…

MASTER PROJECT DEVELOPMENT PROMPT

Build a **complete, fully functional, database-driven Multi-University Bus Ticket Booking and Transport Management System** for a university Database Management Systems course project.

This must be an actual working web application, **not a static prototype or UI mockup**.

The application should demonstrate strong knowledge of:

- Database design
- ER modeling
- Weak entities
- Strong entities
- Composite keys and attributes
- Partial keys

- Superclass/subclass specialization
- Disjoint constraints
- Cardinality
- Participation constraints
- Primary and foreign keys
- Many-to-many relationships
- Database normalization
- Referential integrity
- Transactions
- CRUD operations
- Authentication
- Authorization
- SQL queries
- Views
- Triggers where useful
- Database-backed business rules

The system should initially contain **2–3 sample universities for demonstration**, while the database architecture must be scalable so additional universities can be added later.

---

1. REQUIRED TECHNOLOGY STACK

Use:

Frontend

- HTML5
- CSS3
- Vanilla JavaScript

Backend

- PHP

Database

- MySQL

Database Communication

Use PHP with:

- PDO preferred, or MySQLi
- Prepared statements
- Transactions
- Parameterized queries

Local Development Environment

The project must be runnable using:

- XAMPP
- WAMP
- LAMP

Do **not** use:

- React
- Vue
- Angular
- Node.js as the main backend
- Laravel
- Django
- Firebase
- Supabase
- MongoDB

The project should primarily demonstrate traditional:

HTML + CSS + JavaScript + PHP + MySQL

web development.

JavaScript should handle client-side interactivity and AJAX/fetch requests.

PHP must handle:

- Authentication
- Authorization
- Sessions
- Server-side validation
- CRUD operations
- Booking transactions
- Database access
- File uploads
- Business rules

- Secure data processing

2. PROJECT NAME

Use a professional project name such as:

UniRide — Multi-University Bus Ticket Management System

The name may be changed if a better professional name is appropriate.

3. MAIN SYSTEM IDEA

The system is a **universal university transportation platform**.

Multiple universities can partner with the platform and use the same application while maintaining completely separate transportation data.

For demonstration purposes, populate the database with approximately **2–3 universities**.

The architecture must support adding more universities later.

A university should **not automatically become active by simply registering itself**.

Instead:

1. The university approaches the platform administrator.
2. The System Admin reviews the university.
3. The System Admin adds or approves the university.
4. The university receives access to the transportation management system.
5. The university can then manage its own transportation data.

Each participating university manages its own:

- Buses
- Routes
- Schedules
- Passengers
- Students
- Faculty
- Tickets
- Complaints
- Semester bills
- Transportation operations

4. USER ROLES

Implement proper **Role-Based Access Control (RBAC)**.

There are three major system access levels:

1. System Admin
2. University Administrator / University Management
3. Passenger

Passenger has two subtypes:

- Student
- Faculty

---

5. SYSTEM ADMIN

The System Admin controls the overall platform.

Admin Attributes

Include:

- Admin ID — Primary Key
- Name
- Email — Unique
- Password
- Created At
- Status

Passwords must never be stored in plain text.

Use PHP:

`password_hash()`

and:

`password_verify()`

---

6. SYSTEM ADMIN FUNCTIONS

The System Admin must be able to:

- Login securely
- Logout
- View admin dashboard
- Add universities

- Approve universities
- Edit university information
- Activate/deactivate universities
- Delete universities when appropriate
- Search universities
- View university information
- View university statistics
- View total buses
- View total passengers
- View total bookings
- View total active routes
- View overall system occupancy information
- Manage participating universities

The System Admin has global platform access.

---

7. UNIVERSITY ENTITY

University is a strong entity.

University Attributes

Include:

- University ID — Primary Key
- University Name
- Address
- Contact Number
- Email
- Status
- Created At

Possible status:

- Active
- Inactive
- Pending

---

8. UNIVERSITY MANAGEMENT ACCOUNT

Each university should have an authorized management account.

Create an entity/table such as:

`university_users`

Attributes can include:

- University User ID
- University ID
- Name
- Email
- Password
- Role
- Status
- Created At

University administrators can only manage information belonging to their own university.

---

9. MULTI-UNIVERSITY DATA ISOLATION

This is one of the most important requirements.

University A must never be able to access or modify private data belonging to University B.

For example, University A cannot:

- Edit University B buses
- View University B passengers
- Change University B routes
- Change University B schedules
- Read University B complaints
- Modify University B tickets
- Access University B billing records

Use **`University_ID`** relationships and PHP authorization checks to guarantee tenant isolation.

Never rely only on hidden form values for authorization.

Every important PHP database query must verify the logged-in user's university.

---

10. BUS ENTITY

A university can have multiple buses.

Every bus belongs to **exactly one university**.

For the academic ER model, treat Bus as a **weak/dependent entity** according to the course design.

Use university ownership as part of its identification context.

---

11. BUS IDENTIFICATION

The bus should contain identifying attributes:

- Registration Number
- Tax Number

Represent these as the required **composite identifying attribute/key structure** in the ER design.

The relational implementation may use:

- University ID
- Registration Number
- Tax Number

as a composite uniqueness constraint.

An internal surrogate ID may additionally be used for implementation convenience, but the academic ER design must still clearly represent the required identifying relationship.

---

12. BUS ATTRIBUTES

Include:

- Bus Internal ID if needed
- University ID — Foreign Key
- Registration Number
- Tax Number
- Seat Capacity
- Standing Capacity
- Bus Type
- Status

Bus Type:

- Student
- Faculty

Status may include:

- Active
- Maintenance
- Inactive

13. STANDARD BUS SEATING CONFIGURATION

For the required project demonstration, each standard bus should use:

10 rows × 4 seats = 40 seated passengers

Example layout:

Row 1: A1 A2 | A3 A4
Row 2: B1 B2 | B3 B4
...
Row 10: J1 J2 | J3 J4

Use an aisle visually between the second and third seat.

The seat-selection interface must visually display all **40 seats**.

Keep seat capacity in the database so the architecture remains extensible.

---

14. STANDING / EXTRA SLOT SYSTEM

When all 40 seats on a bus have been booked, passengers should be allowed to reserve standing positions.

Provide up to:

10 standing slots

Example:

- Standing Slot 1
- Standing Slot 2
- ...
- Standing Slot 10

Standing slots should only become available when regular seating reaches full capacity, unless university policy allows otherwise.

The database must track them so they cannot be double booked.

---

15. PASSENGER ENTITY

Passenger belongs to a university.

Passenger should be modeled as a **weak/dependent entity according to the academic design**.

Passenger attributes include:

- Passenger ID — Partial/identifying key
- University ID
- Name

- Email
- Password
- Avatar
- Passenger Type
- Notification Preferences
- Status
- Created At

Passenger ID must be unique within the required identifying context.

---

16. PASSENGER SPECIALIZATION

Passenger is a **superclass**.

It has exactly two subclasses:

Student

Faculty

The specialization must be:

Disjoint

A passenger cannot simultaneously be Student and Faculty.

Also implement total participation:

Every Passenger must be either:

- Student

or

- Faculty.

---

17. STUDENT SUBCLASS

Student inherits its common attributes from Passenger.

Student-specific attributes may include:

- Passenger ID
- Student ID
- Department
- Program
- Semester

Avoid unnecessarily duplicating:

- Name
- Email
- Password

from the Passenger table.

---

18. FACULTY SUBCLASS

Faculty also inherits common Passenger information.

Faculty-specific attributes may include:

- Passenger ID
- Faculty ID
- Department
- Designation

Do not duplicate superclass information unnecessarily.

---

19. BUS TYPE ELIGIBILITY

Student-designated buses should serve eligible Students.

Faculty-designated buses should serve eligible Faculty passengers.

The system must prevent incompatible bookings unless university policy explicitly changes.

At minimum:

- Student → Student Bus
- Faculty → Faculty Bus

must be enforced.

---

20. ROUTE ENTITY

Each university can create multiple transportation routes.

Route attributes:

- Route ID — Primary Key
- University ID — Foreign Key
- Route Name
- Start Location
- End Location

- Fare
- Status

Example:

`R01 — University Campus → Uttara`

Provide route CRUD:

- Add route
- View route
- Update route
- Deactivate/delete route
- Search route

---

21. ROUTE FILTERING

Passengers should be able to search/filter routes by:

- Start location
- End location
- University
- Bus type where appropriate

Favorite routes should appear **at the top of the passenger's route list**.

---

22. BUS-ROUTE ASSIGNMENT

A bus is assigned to one route for a particular operating assignment.

Do not permanently hard-code route information into the Bus table if doing so would create unnecessary redundancy.

Prefer a relation such as:

`bus_route_assignments`

Possible attributes:

- Assignment ID
- Bus ID
- Route ID
- Assignment Date
- Status

Prevent conflicting assignments.

---

23. SCHEDULE ENTITY

A university manages multiple schedules.

One route can have multiple schedules.

Relationship:

Route 1 : N Schedule

Schedule attributes:

- Schedule ID — Primary Key
- University ID
- Route ID
- Bus ID
- Departure Time
- Arrival Time
- Schedule Date
- Status

Possible statuses:

- Scheduled
- Boarding
- Departed
- Completed
- Cancelled

---

24. SCHEDULE BUSINESS RULES

Each Schedule:

- Belongs to one route
- Uses one bus
- Belongs to one university

One route may have many schedules on different:

- Dates
- Departure times

The same bus must not have conflicting schedules at overlapping times.

---

25. PASSENGER DASHBOARD

Build a visually polished passenger dashboard.

It must display:

- Available bus routes
- Favorite routes first
- Route information
- Schedule information
- Seat availability
- Seat availability progress bars
- Active booking alerts
- Upcoming journey
- Current booking
- Semester transport bill
- Notifications
- Complaint information

---

26. ROUTE AVAILABILITY PROGRESS BARS

Every route/schedule card should display seat occupancy using a progress bar.

Examples:

```
12 / 40 seats booked
```

or:

```
30% occupied
```

As capacity fills, visually update the progress.

Once 40 seats are occupied, indicate:

Seats Full — Standing Slots Available

and display remaining standing capacity.

All numbers must come from MySQL queries.

Do not hard-code them.

---

27. ACTIVE BOOKING ALERT

When a passenger has an active booking, display a visible alert/card on the dashboard.

Example information:

- Route
- Date

- Departure time
- Bus
- Seat/Standing Slot
- Booking status
- QR access

---

28. FAVORITE ROUTES

Passengers can favorite multiple routes.

A route can be favorited by multiple passengers.

This creates:

Passenger M : N Route

Use a bridge table such as:

`favorite_routes`

Possible fields:

- Passenger ID
- Route ID
- Created At

Use a composite unique constraint to prevent duplicate favorites.

Favorite routes must automatically appear above non-favorite routes on the passenger route list.

Provide:

- Add favorite
- Remove favorite
- View favorite routes

---

29. SEAT BOOKING INTERFACE

Create a fully interactive seat-selection interface.

Use:

- HTML
- CSS
- JavaScript

Display:

10 rows × 4 seats = 40 seats

The UI should clearly distinguish:

- Available
- Selected
- Booked
- Unavailable

Use different visual states.

Provide a legend explaining each state.

---

30. BOOKING PAGE LAYOUT

Create a modern booking interface containing:

Main Section

Interactive seat map.

Route Details

Show:

- University
- Route
- Start location
- Destination
- Date
- Departure
- Arrival
- Bus
- Bus type

Booking Summary Sidebar

Display:

- Passenger
- Selected seat/standing slot
- Route
- Schedule
- Fare
- Booking date
- Total

The booking summary should dynamically update when the passenger selects a seat.

---

31. SEAT AVAILABILITY

Seat availability must be loaded from the database.

JavaScript can display the information, but JavaScript must **not** be the final authority.

Before confirming a booking, PHP must recheck whether the selected seat is still available.

---

32. DOUBLE-BOOKING PREVENTION

Never allow two passengers to reserve the same seat on the same Schedule.

Enforce this using:

- Unique database constraints
- Server-side PHP validation
- SQL transactions

Use database transactions when making bookings.

---

33. TICKET / BOOKING ENTITY

Create a Ticket/Booking entity.

Attributes:

- Booking ID — Primary Key
- Passenger ID — Foreign Key
- Schedule ID — Foreign Key
- Booking Date
- Seat Number or Standing Slot
- Booking Type
- Fare
- Status
- QR Token
- Created At

Possible Booking Type:

- Seat
- Standing

Possible status:

- Booked
- Confirmed
- Cancelled
- Transferred
- Sold
- Completed

---

34. PASSENGER BOOKING RULE

A passenger may not hold conflicting active tickets at the same time.

At minimum:

- Passenger cannot book the same Schedule twice.
- Passenger cannot hold multiple active tickets for overlapping schedules.
- Passenger cannot book an already occupied seat.
- Passenger eligibility must match the bus type.

Implement server-side validation.

---

35. BOOKING WORKFLOW

Implement the following workflow:

1. Passenger logs in.
2. Passenger opens dashboard.
3. Passenger chooses route.
4. Passenger chooses date.
5. Available schedules appear.
6. Passenger chooses schedule.
7. Seat map loads.
8. Passenger chooses available seat.
9. If all seats are occupied, standing slots appear.
10. Booking summary updates.
11. Fare is calculated.
12. Passenger confirms booking.
13. PHP rechecks availability.
14. Transaction begins.
15. Booking is saved.

16. Seat or standing slot becomes unavailable.
17. Semester billing transaction is created.
18. QR token is generated.
19. Transaction commits.
20. Confirmation page is displayed.

If any critical operation fails, rollback the transaction.

---

36. QR CODE TICKET

After successful booking, generate a unique **QR code** for the ticket.

The QR token should securely identify the booking without exposing sensitive database information directly.

The booking confirmation page must display:

- Booking ID
- Passenger Name
- University
- Route
- Bus
- Date
- Departure Time
- Seat/Standing Slot
- Status
- QR Code

Provide a button to:

Download Ticket / Download QR

The user should be able to save their booking confirmation.

Use a suitable local PHP or JavaScript QR library if necessary.

Do not require an online external API for QR generation.

---

37. BOOKING MANAGEMENT

Passenger must have a **My Bookings** section.

Display:

Active Bookings

Show:

- Booking ID
- Route
- Schedule
- Seat
- Status
- QR code/token
- Cancel button where permitted
- Transfer/share button where permitted

Booking History

Display previous:

- Completed
- Cancelled
- Sold
- Transferred

bookings in a table.

---

38. BOOKING CANCELLATION

Passengers may cancel eligible future bookings.

Cancellation must:

1. Change booking status to Cancelled.
2. Release the seat or standing slot.
3. Make the capacity available to another passenger.
4. Adjust billing correctly according to the chosen cancellation policy.
5. Create appropriate notifications.

Completed journeys cannot be cancelled.

---

39. CLEAR BOOKING HISTORY

Provide a **Clear History** function in the user interface.

However, because this is a database project, do **not physically delete important booking records** required for data integrity or auditing.

Instead, implement it as:

- Hide from passenger history

or

- Archive history

using a field such as:

`hidden_from_passenger = 1`

This preserves database history while satisfying the interface requirement.

---

40. TICKET SELL / SHARE / TRANSFER

Passengers can transfer/share eligible tickets.

At minimum, implement the required student feature:

Students can sell/share tickets with other eligible students.

For consistency with passenger types, same-type transfers may be supported:

- Student → Student
- Faculty → Faculty

Never allow:

- Student → Faculty
- Faculty → Student

when the ticket belongs to a type-restricted bus.

Users should belong to the same university unless explicitly allowed otherwise.

For this project, enforce:

Ticket transfers occur only within the same university.

---

41. TICKET TRANSFER RULES

Allow transfer only when:

- Booking exists
- Booking is active
- Journey has not started
- Sender owns the booking
- Recipient exists
- Recipient belongs to the same university
- Recipient is the correct passenger type
- Recipient does not have a conflicting booking
- Ticket has not already been transferred/cancelled/completed

42. TICKET TRANSFER TABLE

Create:

`ticket_transfers`

Attributes:

- Transfer ID — Primary Key
- Booking ID
- From Passenger ID
- To Passenger ID
- Transfer Type
- Sale Amount
- Transfer Date
- Status

Transfer Type:

- Share
- Sale

Possible Status:

- Pending
- Accepted
- Rejected
- Completed
- Cancelled

Keep historical ownership records.

---

43. WAITLIST / EXTRA SLOT SYSTEM

When regular seats reach:

40 / 40

display:

Bus Full — Standing Slots Available

Provide up to:

10 standing slots

Track them independently.

Example:

`Standing 01`

through:

`Standing 10`

Display:

`6 / 10 standing slots remaining`

Once both seated and standing capacity are full, show:

Fully Booked

If desired, create a waitlist after all standing slots are filled, but the mandatory requirement is the **10 extra standing slots**.

---

44. COMPLAINT SYSTEM

Passengers can submit complaints to their university.

Relationship:

Passenger 1 : N Complaint

and:

University 1 : N Complaint

Complaint attributes:

- Complaint ID — Primary Key
- Passenger ID
- University ID
- Subject
- Description
- Submission Date
- Status
- University Response
- Updated At

Statuses:

- Submitted
- Under Review
- Resolved
- Closed
- Rejected

Passengers can:

- Submit complaint
- View complaint
- View complaint history
- View university response
- View complaint status

University administrators can:

- View complaints
- Search complaints
- Respond
- Change status

---

45. PROFILE MANAGEMENT

Create a complete passenger Profile page.

Passengers should be able to:

- View profile
- Edit name
- Edit permitted personal information
- Upload avatar/profile picture
- Change password
- Configure notification preferences

---

46. AVATAR UPLOAD

Allow users to upload profile images.

Implement secure server-side file validation.

Validate:

- MIME type
- Extension
- File size

Use randomized file names.

Never allow uploaded PHP/executable files.

Store only the image location in the database.

---

47. PASSWORD CHANGE

Password change requires verification.

Recommended flow:

1. User enters current password.
2. PHP verifies current password.
3. User enters new password.
4. User confirms new password.
5. Validate password.
6. Store using `password_hash()`.

Do not allow password change based only on frontend validation.

---

48. NOTIFICATION PREFERENCES

Passenger profile should contain notification preference toggles:

- Email Notifications
- Push/In-App Notifications

Store the preferences in MySQL.

Actual external email/push service integration is not mandatory unless easy to implement.

The in-app notification system is mandatory.

---

49. NOTIFICATION SYSTEM

Create a database-backed notification feature.

Display a **bell icon** in the top navigation.

If unread notifications exist, show a count badge.

Clicking the bell should open a:

- Popover/dropdown
- Scrollable notification list

Each notification includes:

- Message
- Date/time
- Read/unread status

Functions:

- Open notification
- Mark individual notification as read
- Mark all as read

---

50. NOTIFICATION ENTITY

Create a table such as:

`notifications`

Attributes:

- Notification ID
- Passenger ID
- Title
- Message
- Type
- Is Read
- Created At

Generate notifications for events such as:

- Successful booking
- Booking cancellation
- Ticket transfer request
- Ticket transfer completed
- Schedule cancellation
- Complaint response

---

51. SEMESTER BILLING

Ticket fare should be added to the passenger's transportation bill for the appropriate semester.

Create a Semester entity.

Attributes:

- Semester ID
- University ID
- Semester Name
- Start Date
- End Date
- Status

Examples:

- Spring 2026
- Summer 2026
- Fall 2026

---

52. BILLING TRANSACTIONS

Prefer maintaining individual billing transactions rather than changing an unexplained total.

Create:

`billing_transactions`

Attributes:

- Transaction ID
- Passenger ID
- Booking ID
- Semester ID
- Amount
- Transaction Type
- Transaction Date

Possible types:

- Ticket Charge
- Cancellation Adjustment
- Transfer Adjustment

---

53. SEMESTER BILL

Create:

`semester_bills`

Possible attributes:

- Bill ID
- Passenger ID
- Semester ID
- Total Amount
- Status
- Updated At

Possible status:

- Pending
- Paid
- Partially Paid

Ensure the total can be correctly reconciled against billing transactions.

---

54. UNIVERSITY DASHBOARD

Create a professional university administration dashboard.

Display:

- Total buses
- Active buses
- Total routes
- Active routes
- Total schedules
- Today's schedules
- Total passengers
- Number of students
- Number of faculty
- Total bookings
- Today's bookings
- Overall occupancy rate
- Pending complaints
- Semester transport revenue/charges

All values must come from database queries.

---

55. ADMIN DASHBOARD ANALYTICS

The System Admin dashboard must specifically display:

- Total bookings
- Occupancy rate
- Active routes
- Total buses
- Total universities
- Active universities

- Total passengers

Provide useful visual cards/progress indicators.

The Admin must also be able to:

- Add buses where appropriate
- Delete/deactivate buses
- Add routes
- Delete/deactivate routes

while respecting university ownership and permissions.

---

56. OCCUPANCY RATE

Calculate occupancy using database information.

For a schedule, a simple seated occupancy percentage can be:

```
Booked Seats / Total Seats × 100
```

Also display standing usage separately when applicable.

Example:

```
Seats: 40/40
```

```
Standing: 6/10
```

Do not hard-code percentages.

---

57. BUS MANAGEMENT

Authorized university management should be able to:

- Add bus
- View buses
- Edit bus
- Deactivate bus
- Delete bus if safe
- Search buses
- Filter buses
- View bus route assignment
- View occupancy information

---

58. ROUTE MANAGEMENT

Provide:

- Add route
- View route
- Edit route
- Deactivate route
- Delete route when safe
- Search route
- Filter by location
- View schedule count

---

59. SCHEDULE MANAGEMENT

Provide:

- Add schedule
- Edit schedule
- View schedule
- Cancel schedule
- Search schedule
- Filter by date
- Filter by route
- View occupancy
- View passengers/bookings

Cancelling a schedule should appropriately notify affected passengers.

---

60. PASSENGER MANAGEMENT

University management can:

- View passengers
- Search passengers
- Filter Students
- Filter Faculty
- View passenger information
- Activate/deactivate passenger where appropriate

Do not expose password hashes.

---

61. LANDING PAGE

Create a polished public landing page.

Include:

- Navigation bar
- Project logo/name
- Hero section
- Explanation of the bus system
- Participating universities
- Features
- How it works
- Login buttons
- Footer

The website should look like a real university transportation platform.

---

62. LOGIN SYSTEM

Provide login access for:

- System Admin
- University Administrator
- Passenger

Authentication must use:

- PHP sessions
- Secure password verification
- Server-side role checks

After login, redirect users to their appropriate dashboard.

---

63. AUTHORIZATION

Do not assume a user is authorized merely because they know a URL.

For example:

A Passenger manually visiting:

`/admin/dashboard.php`

must be denied.

Likewise, one university administrator cannot modify another university by changing URL parameters.

Implement centralized PHP authentication/authorization functions.

---

64. SESSION SECURITY

Use proper PHP sessions.

Include:

- Session regeneration after login
- Logout/session destruction
- Authorization middleware/helper functions
- Appropriate timeout if desired

---

65. REQUIRED WEBSITE PAGES

Create all pages required for a complete system.

At minimum:

Public

- Home
- About
- How It Works
- Login

Admin

- Admin Login
- Admin Dashboard
- Universities
- University Details
- System Statistics

University

- University Dashboard
- Bus Management
- Route Management
- Schedule Management
- Passenger Management

- Complaint Management
- Booking Management
- Semester Billing

Passenger

- Passenger Dashboard
- Routes
- Schedules
- Seat Booking
- Booking Confirmation
- My Bookings
- Booking History
- Ticket Transfer
- Favorite Routes
- Complaints
- Notifications
- Semester Bill
- Profile
- Change Password

---

66. RESPONSIVE DESIGN

The application must work on:

- Desktop
- Laptop
- Tablet
- Mobile

Use responsive CSS.

Provide:

- Mobile navigation
- Responsive dashboards
- Responsive data tables
- Responsive forms
- Responsive seat-selection interface

---

67. UI DESIGN STYLE

Make the website visually polished but appropriate for a university project.

Use:

- Clean professional typography
- Modern cards
- Dashboard sidebar
- Navigation bar
- Tables
- Status badges
- Progress bars
- Modals
- Toast notifications
- Confirmation dialogs
- Proper form spacing
- Icons where useful
- Consistent color system
- Hover/focus states

Avoid overly complicated animations.

Functionality and database correctness are more important than decoration.

---

68. JAVASCRIPT FEATURES

Use Vanilla JavaScript for:

- Interactive seat selection
- Fetch/AJAX
- Search/filter interactions
- Dynamic route loading
- Dynamic schedule loading
- Booking summary updates
- Modal windows
- Confirmation dialogs
- Notification popover
- Mark-as-read interactions
- Progress display

- Favorite button
- Client-side validation
- UI feedback

Do not rely on JavaScript alone for security or database rules.

---

69. SERVER-SIDE VALIDATION

Every important operation must be validated again in PHP.

Never trust:

- Browser HTML validation
- JavaScript
- URL parameters
- Hidden inputs
- Client-supplied University IDs
- Client-supplied fares

Calculate/validate critical values using server-side database records.

---

70. DATABASE NORMALIZATION

Design the relational database to approximately **Third Normal Form (3NF)** where appropriate.

Avoid:

- Repeating groups
- Duplicated passenger information
- Duplicated route details in ticket rows
- Unnecessary calculated data
- Multi-valued columns
- Comma-separated relationships

Use proper bridge tables for many-to-many relationships.

---

71. SUGGESTED DATABASE TABLES

Design an appropriate normalized schema containing tables similar to:

- **`admins`**
- **`universities`**
- **`university_users`**

- `buses`
- `passengers`
- `students`
- `faculty`
- `routes`
- `bus_route_assignments`
- `schedules`
- `bookings`
- `favorite_routes`
- `complaints`
- `ticket_transfers`
- `notifications`
- `semesters`
- `semester_bills`
- `billing_transactions`

Additional tables may be created when needed.

Do not add redundant tables merely to increase table count.

---

72. DATABASE CONSTRAINTS

Use:

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- NOT NULL
- CHECK where supported
- ENUM where appropriate
- Composite keys where appropriate
- Referential integrity
- Indexes

Carefully choose:

- `ON DELETE`
- `ON UPDATE`

actions.

Avoid destructive cascade behavior that could accidentally remove important booking history.

---

73. IMPORTANT UNIQUE CONSTRAINTS

Examples include:

- Admin email unique
- University email unique where applicable
- Passenger email unique within appropriate context
- Favorite `(Passenger_ID, Route_ID)` unique
- Schedule-seat combination unique
- University bus registration/tax combination unique
- Student/faculty identifiers unique within university

---

74. ER DIAGRAM

Produce a complete ER Diagram/specification.

It must identify:

- Admin
- University
- Bus
- Passenger
- Student
- Faculty
- Route
- Schedule
- Booking/Ticket
- Complaint
- Favorite Route
- Ticket Transfer
- Semester
- Billing
- Notification

Clearly show:

- Strong entity
- Weak entity

- Partial key
- Composite attribute/key
- Primary key
- Foreign key
- Cardinality
- Participation
- Identifying relationship
- Superclass/subclass
- Disjoint specialization
- Total specialization
- One-to-many
- Many-to-many

Make the ER diagram easy to explain during a viva/presentation.

---

75. RELATIONAL SCHEMA

After the ER model, provide the complete relational schema.

Use notation like:

`UNIVERSITY(University_ID PK, University_Name, Address, Contact, Email, Status)`

`ROUTE(Route_ID PK, University_ID FK, Start_Location, End_Location, Fare, Status)`

`SCHEDULE(Schedule_ID PK, Route_ID FK, Bus_ID FK, Schedule_Date, Departure_Time, Arrival_Time, Status)`

Continue for every relation.

Clearly mark:

- PK
- FK
- Composite PK
- Unique keys

---

76. SQL DATABASE FILE

Create a complete file:

`database/university_bus_system.sql`

It must contain:

- CREATE DATABASE

- USE database
- CREATE TABLE statements
- Foreign keys
- Constraints
- Unique constraints
- Indexes
- Sample data

---

77. SAMPLE DATA

Insert enough sample data to demonstrate every major feature.

At minimum include:

- 1 System Admin
- 2–3 Universities
- University administrator accounts
- Student passengers
- Faculty passengers
- Student buses
- Faculty buses
- Routes
- Bus-route assignments
- Schedules
- Active bookings
- Historical bookings
- Favorite routes
- Complaints
- Notifications
- Semesters
- Billing transactions

Include safe demo credentials in the README.

---

78. DATABASE COURSE SQL FEATURES

Demonstrate academically meaningful use of:

- INNER JOIN

- LEFT JOIN
- Aggregate functions
- COUNT
- SUM
- AVG
- GROUP BY
- HAVING
- Subqueries
- Transactions
- Indexes
- Views

Use stored procedures/triggers only when they make sense.

---

79. DATABASE VIEWS

Create useful views where appropriate.

Examples:

`view_schedule_availability`

`view_passenger_booking_history`

`view_university_booking_statistics`

`view_semester_transport_charges`

Explain each view in the documentation.

---

80. TRANSACTIONS

Use transactions for operations involving multiple dependent changes.

Especially:

Booking

- Create booking
- Reserve seat
- Create billing transaction
- Generate related data

Cancellation

- Update booking

- Release capacity
- Adjust bill

Ticket Transfer

- Update ownership/status
- Create transfer history
- Validate recipient

Rollback if any critical operation fails.

---

81. TRIGGERS

A small number of useful MySQL triggers may be included.

Examples:

- Audit important ticket status changes
- Maintain a history record

However, avoid hiding too much application logic inside triggers.

Important business rules should still be understandable from the PHP code.

---

82. SECURITY REQUIREMENTS

Implement:

- Prepared statements
- SQL injection protection
- Output escaping against XSS
- Password hashing
- PHP session security
- Role authorization
- Server-side input validation
- Safe file uploads
- CSRF tokens for important POST requests where practical

Never expose database credentials to browser JavaScript.

---

83. PROJECT FILE STRUCTURE

Use a professional structure similar to:

```
uniride/
│
├── index.php
├── login.php
├── logout.php
├── about.php
│
├── admin/
│   ├── dashboard.php
│   ├── universities.php
│   └── ...
│
├── university/
│   ├── dashboard.php
│   ├── buses.php
│   ├── routes.php
│   ├── schedules.php
│   ├── passengers.php
│   ├── complaints.php
│   └── ...
│
├── passenger/
│   ├── dashboard.php
│   ├── routes.php
│   ├── booking.php
│   ├── my-bookings.php
│   ├── favorites.php
│   ├── complaints.php
│   ├── notifications.php
│   ├── profile.php
│   └── ...
│
├── api/
│   ├── available-seats.php
│   ├── schedules.php
│   ├── favorite-route.php
│   ├── notifications.php
│   └── ...
│
├── config/
│   └── database.php
│
├── includes/
│   ├── auth.php
│   ├── functions.php
│   ├── header.php
│   ├── footer.php
│   └── csrf.php
│
├── assets/
│   ├── css/
│   │   ├── style.css
│   │   └── dashboard.css
│   ├── js/
│   │   ├── app.js
│   │   ├── booking.js
│   │   └── notifications.js
│   ├── images/
│   └── uploads/
│
├── database/
│   └── university_bus_system.sql
│
└── README.md
```

Improve this structure where beneficial.

---

84. README FILE

Create a professional **`README.md`**.

Include:

- Project title
- Project description
- Main features
- Technology stack
- Database concepts demonstrated
- Requirements
- XAMPP installation
- Database setup
- SQL import instructions
- Configuration
- Application URL
- Demo accounts
- Folder structure
- ER model description
- Relational schema summary
- Security notes
- Screens/features explanation

---

85. INSTALLATION INSTRUCTIONS

The README should explain steps similar to:

1. Install XAMPP.
2. Start Apache.
3. Start MySQL.
4. Copy project folder into **`htdocs`**.
5. Open phpMyAdmin.
6. Import **`university_bus_system.sql`**.
7. Configure **`database.php`**.
8. Visit the project through localhost.
9. Login using demo credentials.

---

86. DATA-DRIVEN APPLICATION REQUIREMENT

Do not create fake hard-coded application behavior.

Examples:

When adding a university:

INSERT into MySQL.

When editing a bus:

UPDATE MySQL.

When deleting/deactivating a route:

Update database state.

When displaying schedules:

SELECT from MySQL.

When booking:

INSERT booking using a secure transaction.

When favoriting:

Persist favorite in MySQL.

When submitting complaint:

Persist complaint.

When marking a notification as read:

Update notification record.

When showing occupancy:

Calculate from database bookings.

When showing semester bills:

Calculate/retrieve database billing records.

---

87. ERROR HANDLING

Provide clear and user-friendly errors.

Examples:

- Seat already booked
- Schedule unavailable
- Wrong passenger type
- Bus full
- Standing slots full
- Duplicate favorite
- Invalid transfer
- Unauthorized access
- Invalid login

- Current password incorrect

Do not expose raw SQL errors to regular users.

---

88. SUCCESS FEEDBACK

Use attractive success feedback for:

- Booking confirmed
- Booking cancelled
- Complaint submitted
- Route favorited
- Ticket transferred
- Profile updated
- Password changed
- Notification marked as read

Use:

- Toasts
- Alerts
- Modals

where appropriate.

---

89. ACCESSIBILITY

Use:

- Form labels
- Keyboard-accessible controls
- Proper button elements
- Focus states
- Meaningful alt text
- Readable contrast

Do not represent seat states using color alone.

Add text/icon/state differences where useful.

---

90. CORE BUSINESS RULE SUMMARY

The completed project must enforce these rules:

1. System Admin controls participating universities.
2. Multiple universities use the same platform.
3. Demonstration data contains approximately 2–3 universities.
4. Each university has isolated transportation data.
5. One university has many buses.
6. Every bus belongs to one university.
7. Bus has registration and tax identification.
8. Buses are Student or Faculty type.
9. Passenger belongs to one university.
10. Passenger is either Student or Faculty, never both.
11. One university has many routes.
12. One route has many schedules.
13. One schedule uses one bus.
14. Schedule contains many bookings.
15. Standard booking UI contains 40 seats: 10 rows × 4.
16. A seat cannot be double booked.
17. Once all 40 seats are occupied, up to 10 standing slots become available.
18. Passenger cannot hold conflicting active bookings.
19. Favorite Route is a many-to-many Passenger–Route relationship.
20. Favorite routes appear first.
21. Passenger may submit many complaints.
22. University may receive many complaints.
23. Successful bookings generate QR tickets.
24. Passenger can download/view their QR ticket.
25. Cancelling a booking releases the seat/slot.
26. Booking history is preserved.
27. Clearing history should archive/hide it rather than destroy required records.
28. Eligible passengers can share/transfer tickets within their university.
29. Student-to-student ticket sale/share must work.
30. Cross-type ticket transfer is prohibited for type-restricted buses.
31. Booking fare contributes to semester billing.
32. Dashboard occupancy is calculated from database data.
33. Notifications are persisted in MySQL.
34. Profile supports avatar, notification preferences, and password change.
35. Authorization must be enforced in PHP.

91. REQUIRED PASSENGER FEATURES

Passenger must be able to:

- Login
- Logout
- View dashboard
- View routes
- See favorite routes first
- View route seat availability progress
- View active booking alert
- Search routes
- View schedules
- Select seat
- Reserve standing slot when applicable
- Book ticket
- View booking confirmation
- View QR ticket
- Download ticket/QR
- View active bookings
- Cancel eligible booking
- View booking history
- Clear/archive visible history
- Favorite route
- Remove favorite
- Submit complaint
- View complaint status
- Sell/share/transfer eligible ticket
- View transfer history
- View notifications
- Mark notification read
- Manage notification preferences
- View semester bill
- Edit profile
- Upload avatar

- Change password

92. REQUIRED UNIVERSITY FEATURES

University management must be able to:

- Login
- Logout
- View dashboard
- View occupancy
- View booking statistics
- Manage buses
- Manage routes
- Manage schedules
- Manage passengers
- View Students
- View Faculty
- View bookings
- View complaints
- Respond to complaints
- View semester billing
- Search/filter information
- View route statistics
- View schedule occupancy

93. REQUIRED SYSTEM ADMIN FEATURES

System Admin must be able to:

- Login
- Logout
- View global dashboard
- Add universities
- Edit universities
- Activate/deactivate universities
- Remove universities when appropriate
- Search universities

- View total bookings
- View total passengers
- View total buses
- View occupancy rate
- View active routes
- View university statistics

---

94. TESTING REQUIREMENTS

Test important scenarios.

Authentication

- Correct login
- Wrong password
- Unauthorized page access

Booking

- Normal seat booking
- Duplicate seat booking
- Passenger attempts second conflicting booking
- Student attempts Faculty bus
- Faculty attempts Student bus

Standing

- Seat capacity reaches 40
- Standing slots appear
- Standing slots reach 10
- Bus becomes fully booked

Cancellation

- Cancel active booking
- Released seat becomes available

Transfer

- Valid student-to-student transfer
- Different university transfer
- Student-to-faculty transfer

- Transfer of completed ticket

Favorites

- Add favorite
- Prevent duplicate
- Favorite appears first

Complaints

- Submit complaint
- University response
- Status update

Profile

- Avatar validation
- Password verification

Security

- SQL injection attempt
- URL ID manipulation
- Cross-university access attempt

---

95. DATABASE COURSE DOCUMENTATION

Along with the working application, provide documentation explaining:

Entities

Which entities are strong and weak.

Keys

- Primary keys
- Foreign keys
- Composite keys
- Partial keys

Relationships

Explain all cardinalities.

Specialization

Explain:

Passenger → Student / Faculty

including:

- Superclass
- Subclass
- Disjoint
- Total participation

Normalization

Explain why the schema is in approximately 3NF.

Transactions

Explain why booking uses transactions.

Many-to-Many

Explain:

Passenger ↔ Favorite Route

through the associative table.

Make this documentation useful for viva preparation.

---

96. CODE QUALITY

Code must be:

- Organized
- Readable
- Modular
- Commented where necessary
- Secure
- Consistent

Use reusable PHP functions.

Avoid copying the same database logic into many pages unnecessarily.

Use reusable:

- Header
- Footer
- Navigation

- Authentication helpers
- Database functions
- Validation utilities

---

97. IMPORTANT IMPLEMENTATION PRINCIPLE

Do **not** spend all development effort creating a beautiful dashboard while leaving backend functionality unfinished.

Prioritize in this order:

1. Correct database structure
2. Correct relationships
3. Authentication
4. Authorization
5. Functional CRUD
6. Booking transaction
7. Seat/standing availability
8. Billing
9. Ticket transfer
10. Complaints
11. Favorites
12. Notifications
13. Profile
14. Dashboard statistics
15. UI polish

---

98. DEVELOPMENT ORDER

Build the project in this sequence:

Phase 1 — Analysis

Review all requirements.

Phase 2 — Database Design

Create ER model.

Phase 3 — Relational Design

Convert ER model into normalized relations.

Phase 4 — MySQL

Create tables, keys, constraints, indexes, and sample data.

Phase 5 — Project Foundation

Create folder structure and PHP database connection.

Phase 6 — Authentication

Implement Admin, University, and Passenger authentication.

Phase 7 — Authorization

Implement role and university isolation.

Phase 8 — CRUD

Implement Universities, Buses, Routes, Schedules, and Passengers.

Phase 9 — Booking

Implement seat availability and booking transactions.

Phase 10 — Standing Slots

Implement extra standing-capacity system.

Phase 11 — QR Tickets

Generate confirmation QR codes.

Phase 12 — Booking Management

Implement cancellation and history.

Phase 13 — Billing

Implement semester transportation billing.

Phase 14 — Favorites

Implement favorite routes and sorting.

Phase 15 — Ticket Transfer

Implement share/sale feature.

Phase 16 — Complaints

Implement complaint workflow.

Phase 17 — Notifications

Implement bell/dropdown/read status.

Phase 18 — Profile

Implement avatar, preferences, password change.

Phase 19 — Dashboards

Implement real database statistics.

Phase 20 — Frontend Polish

Make application responsive and professional.

Phase 21 — Testing

Test database constraints and security.

Phase 22 — Documentation

Generate README, ER documentation, relational schema, and demo credentials.

---

99. FINAL DELIVERABLES

Generate the entire project.

Required deliverables:

1. Complete HTML frontend
2. Complete CSS
3. Complete Vanilla JavaScript
4. Complete PHP backend
5. MySQL database
6. SQL installation file
7. System Admin authentication
8. University authentication
9. Passenger authentication
10. Multi-university architecture
11. University onboarding
12. Bus CRUD
13. Passenger superclass
14. Student subclass

15. Faculty subclass
16. Route CRUD
17. Schedule CRUD
18. Bus-route assignment
19. 40-seat interactive booking layout
20. Seat availability progress bars
21. 10 standing-slot system
22. Booking transaction
23. Booking summary sidebar
24. Active booking alerts
25. QR-code ticket generation
26. QR/ticket download
27. Booking cancellation
28. Booking history
29. Clear/archive history functionality
30. Ticket share/sell/transfer
31. Favorite routes
32. Favorite-first route sorting
33. Complaints
34. Complaint responses
35. In-app notifications
36. Notification bell/popover
37. Mark notifications read
38. Notification preferences
39. Profile editing
40. Avatar upload
41. Password change with verification
42. Semester billing
43. Admin dashboard
44. University dashboard
45. Passenger dashboard
46. Occupancy statistics
47. Search/filter functionality
48. Role-based authorization
49. Multi-university data isolation

50. ER diagram/specification
51. Relational schema
52. Database normalization explanation
53. Sample data for 2–3 universities
54. README
55. XAMPP installation guide
56. Demo credentials
57. Testing instructions

---

100. FINAL INSTRUCTION TO THE CODING AGENT

Do not merely describe or explain the application.

Build it.

Generate all required files and code necessary to run the project locally.

Do not provide only:

- Wireframes
- Pseudocode
- Static HTML
- Database diagrams
- Partial snippets

The final result must be a **working end-to-end database application** where the frontend, PHP backend, and MySQL database are properly connected.

Every major action must persist to or retrieve from the database.

Before considering the project complete, verify:

- Login works.
- Roles work.
- University data isolation works.
- CRUD works.
- Routes load from MySQL.
- Schedules load from MySQL.
- Seat availability is real.
- 40-seat interface works.
- Standing slots work.
- Double booking is blocked.
- Booking creates a ticket.

- QR ticket is generated.
- Ticket can be downloaded/viewed.
- Cancellation releases capacity.
- Favorites persist.
- Favorite routes appear first.
- Complaints persist.
- Ticket transfers persist.
- Notifications persist.
- Profile changes persist.
- Semester billing reflects bookings.
- Dashboard statistics come from SQL queries.
- The project runs correctly in XAMPP.

When an implementation decision is not explicitly specified, choose the solution that provides the **best database normalization, data integrity, maintainability, security, and academic clarity**.

The final system should be polished enough for a university project demonstration while also making its database architecture easy to explain during a **DBMS presentation and viva**.

give me a-z sql code for phpmyadmin

Done. I created the **complete A–Z phpMyAdmin SQL database** for your project.

It includes **19 tables, 21 triggers, 6 stored procedures, 4 database views, a seat-label function, foreign keys, indexes, demo data, hashed demo passwords, semester billing, QR-token storage, complaints, favorites, notifications, ticket transfers, 40-seat booking logic, and 10 standing slots**. The seat/QR/standing/booking-management requirements come directly from your project specification. The complaints, favorites, notifications, and student ticket-sharing features are also included.

Section06_Group06_cse370 projec…

Section06_Group06_cse370 projec…

Download the complete SQL

How to import it

In XAMPP, start **Apache** and **MySQL**, then open phpMyAdmin. Go to **Import** → **Choose File** → **select** `uniride_complete_phpmyadmin.sql` → **Import**.

The script automatically creates:

```
Database: uniride_db
```

and all required tables.

Main tables included

```
universities
```

```
admins
university_users

passengers
students
faculty

buses
routes
bus_route_assignments
schedules

bookings
booking_status_history

semesters
semester_bills
billing_transactions

favorite_routes
complaints
ticket_transfers
notifications
```

Important booking rules already enforced in SQL

The database itself prevents things such as:

```
Student booking a Faculty bus
Faculty booking a Student bus

Passenger booking another university's bus

Two passengers booking the same seat

Passenger having overlapping active bookings

Bus having overlapping schedules

Bus being assigned to another university's route

Passenger favoriting another university's route

Ticket transfer between different universities

Student → Faculty ticket transfer

Faculty → Student ticket transfer

Standing ticket before all seats are occupied
```

For your required bus layout, the database treats seats as:

```
1  = A1
2  = A2
3  = A3
4  = A4

5  = B1
6  = B2
7  = B3
8  = B4

...

37 = J1
38 = J2
39 = J3
40 = J4
```

Then:

```
Standing Slot 1
Standing Slot 2
...
Standing Slot 10
```

become available after all regular seats are occupied.

PHP can book a ticket using

```
SQL
CALL sp_create_booking(
    7,          -- passenger_id
    1,          -- schedule_id
    'SEAT',
    2           -- seat number
);
```

Cancel it with:

```
SQL
CALL sp_cancel_booking(
    1,          -- booking_id
    1           -- passenger_id
);
```

Request a student-to-student ticket share:

```
SQL
CALL sp_request_ticket_transfer(
    1,          -- booking_id
    1,          -- sender passenger_id
    7,          -- receiver passenger_id
    'SHARE',
    0.00
);
```

Accept the transfer:

```
SQL
CALL sp_respond_ticket_transfer(
    1,          -- transfer_id
    7,          -- recipient passenger_id
    'ACCEPT'
);
```

The SQL also automatically adds the ticket fare to the passenger's **semester bill**, generates a unique **QR token**, generates booking notifications, refunds/adjusts the semester bill when a booking is cancelled, and releases the cancelled seat so another student can book it.

Demo accounts

System Admin

```
Email: admin@uniride.demo
Password: Admin@123
```

University Admin

```
Email: admin@mcu.demo
Password: Uni@123
```

There are also:

```
admin@gfu.demo
admin@etu.demo
```

with the same demo password.

Passenger

```
Email: ayan@mcu.demo
Password: Passenger@123
```

Other demo passengers are already inserted too.

The next piece you should build against this is the **PHP database connection + login system**, because once those are working, every dashboard, route page, seat map, booking button, favorites system, and complaint page can use this schema directly.

> give me in one cell so that i can copy paste easily

Copy everything inside this **single SQL block** and paste it directly into **phpMyAdmin → SQL → Run**.

```
SQL
-- ============================================================
-- UniRide - Multi-University Bus Ticket Management System
-- Complete database for phpMyAdmin / MySQL 8+ / MariaDB 10.4+
-- Frontend: HTML + CSS + JavaScript | Backend: PHP | DB: MySQL
-- ============================================================

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

DROP DATABASE IF EXISTS uniride_db;
CREATE DATABASE uniride_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
USE uniride_db;

SET FOREIGN_KEY_CHECKS = 1;

-- ============================================================
-- 1. CORE / AUTHENTICATION TABLES
-- ============================================================

CREATE TABLE universities (
    university_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    university_code VARCHAR(20) NOT NULL,
    university_name VARCHAR(150) NOT NULL,
    address VARCHAR(255) NOT NULL,
    contact_number VARCHAR(30) NOT NULL,
    email VARCHAR(150) NOT NULL,
    status ENUM('PENDING','ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    UNIQUE KEY uq_university_code (university_code),
    UNIQUE KEY uq_university_email (email)
) ENGINE=InnoDB;
```

```
CREATE TABLE admins (
```

also add a prompt to integrate this database into the code. add this to the final prompts

Yes — add the following section to the **final master prompt** after the database requirements and before the final completion/testing instructions. This tells the coding agent that the SQL database is **already defined and must be integrated into the PHP code exactly**, instead of inventing a different database.

DATABASE INTEGRATION REQUIREMENTS

A complete MySQL database for this project has already been designed.

The database name is:

```
uniride_db
```

The provided SQL file/schema is the **authoritative database structure for the project**.

Do **not** redesign the database from scratch, rename tables unnecessarily, replace the relationships, or create an unrelated database structure.

The PHP application must be built directly on top of this existing database.

---

1. DATABASE TECHNOLOGY

Use:

- MySQL
- phpMyAdmin
- PHP PDO preferred
- Prepared statements
- Transactions
- UTF-8 / **`utf8mb4`**

The project should run locally using:

- XAMPP
- Apache
- PHP
- MySQL
- phpMyAdmin

---

2. DATABASE NAME

Connect the application to:

`uniride_db`

Create the PHP database configuration file:

`config/database.php`

Use a structure similar to:

```
<?php

$host = 'localhost';
$db   = 'uniride_db';
$user = 'root';
$pass = '';
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";

$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,
```

```
];

try {
    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (PDOException $e) {
    exit('Database connection failed.');
}
```

Do not display sensitive database errors to normal users.

---

3. EXISTING DATABASE TABLES

The application must integrate with the existing tables:

```
universities
admins
university_users

passengers
students
faculty

buses
routes
bus_route_assignments
schedules

bookings
booking_status_history

semesters
semester_bills
billing_transactions

favorite_routes
complaints
ticket_transfers
notifications
```

Do not create duplicate tables such as:

```
users2
tickets_new
routes_backup
new_passengers
bus_data
```

unless there is a genuinely necessary database migration.

Use the existing schema.

---

4. AUTHENTICATION INTEGRATION

Authentication must query the existing database tables.

System Admin

Authenticate against:

```
admins
```

Using:

```
email
password_hash
status
```

After login, store important session information such as:

```
$_SESSION['user_id']
$_SESSION['role']
$_SESSION['name']
```

Role:

```
SYSTEM_ADMIN
```

---

5. UNIVERSITY LOGIN INTEGRATION

University administrators must authenticate against:

```
university_users
```

Use:

```
university_user_id
university_id
email
password_hash
role
status
```

After successful login, store:

```
$_SESSION['user_id']
$_SESSION['university_id']
$_SESSION['role']
$_SESSION['name']
```

Every university-management query must use the logged-in:

```
university_id
```

to enforce university isolation.

---

6. PASSENGER LOGIN INTEGRATION

Passengers authenticate against:

```
passengers
```

Use:

```
passenger_id
university_id
email
password_hash
passenger_type
status
```

After successful login, store:

```
$_SESSION['passenger_id']
$_SESSION['university_id']
$_SESSION['passenger_type']
$_SESSION['role']
$_SESSION['name']
```

Do not trust Passenger ID or University ID supplied through URL parameters when session information already identifies the user.

---

7. PASSWORD VERIFICATION

All PHP authentication must use:

```
password_verify($password, $row['password_hash'])
```

Never compare plain-text passwords.

When creating or changing passwords, use:

```
password_hash($password, PASSWORD_DEFAULT)
```

---

8. UNIVERSITY DATA ISOLATION

The existing database supports multiple universities.

PHP must maintain this separation.

For example, when an MCU university administrator requests buses, use logic equivalent to:

```
SELECT *
FROM buses
WHERE university_id = ?
ORDER BY registration_number;
```

Bind:

```
$_SESSION['university_id']
```

Do not allow the browser to decide the university.

Incorrect:

```
$universityId = $_GET['university_id'];
```

for authorization purposes.

Correct:

```
$universityId = $_SESSION['university_id'];
```

The System Admin may have global access.

---

9. PASSENGER SUPERCLASS INTEGRATION

The main passenger information comes from:

```
passengers
```

Student-specific data comes from:

```
students
```

Faculty-specific data comes from:

```
faculty
```

For a Student profile, use a query similar to:

```
SELECT
    p.*,
    s.student_identifier,
    s.department,
    s.program,
    s.semester_label
FROM passengers p
LEFT JOIN students s
    ON s.passenger_id = p.passenger_id
WHERE p.passenger_id = ?;
```

For Faculty:

```
SELECT
    p.*,
    f.faculty_identifier,
    f.department,
    f.designation
FROM passengers p
LEFT JOIN faculty f
    ON f.passenger_id = p.passenger_id
WHERE p.passenger_id = ?;
```

Respect the Student/Faculty specialization already implemented by the database.

---

10. BUS MANAGEMENT INTEGRATION

Bus CRUD pages must operate on:

```
buses
```

Use:

```
bus_id
university_id
registration_number
tax_number
seat_capacity
standing_capacity
bus_type
status
```

University administrators may only manipulate buses whose:

```
university_id = logged-in university_id
```

Do not bypass database constraints.

If MySQL rejects invalid data, display a clean application-level error.

---

11. ROUTE MANAGEMENT INTEGRATION

Route CRUD must use:

```
routes
```

Use:

```
route_id
university_id
route_code
route_name
start_location
end_location
fare
status
```

Example retrieval:

```
SELECT *
FROM routes
WHERE university_id = ?
  AND status = 'ACTIVE'
ORDER BY route_name;
```

---

12. BUS-ROUTE ASSIGNMENT INTEGRATION

Use:

```
bus_route_assignments
```

for bus-route relationships.

Do not duplicate Route ID inside multiple unnecessary tables.

The existing database prevents invalid cross-university bus-route assignments.

PHP should still validate selections before attempting inserts.

---

13. SCHEDULE INTEGRATION

Schedules must use:

```
schedules
```

and connect to:

```
routes
buses
```

Typical schedule query:

```
SELECT
    s.schedule_id,
    s.schedule_date,
    s.departure_time,
    s.arrival_time,
    s.status,

    r.route_name,
    r.start_location,
    r.end_location,
    r.fare,

    b.registration_number,
```

```
    b.bus_type

FROM schedules s

JOIN routes r
    ON r.route_id = s.route_id

JOIN buses b
    ON b.bus_id = s.bus_id

WHERE r.university_id = ?
ORDER BY
    s.schedule_date,
    s.departure_time;
```

---

14. ROUTE AVAILABILITY VIEW

The database already provides:

`v_schedule_availability`

Use this view for passenger route/schedule cards and occupancy bars.

It provides information such as:

- University
- Route
- Bus
- Schedule
- Seat capacity
- Booked seats
- Available seats
- Standing capacity
- Booked standing slots
- Available standing slots
- Occupancy percentage

Example:

```
SELECT *
FROM v_schedule_availability
WHERE university_id = ?
  AND schedule_status = 'SCHEDULED'
ORDER BY schedule_date, departure_time;
```

Use:

`seated_occupancy_percent`

to render the dashboard progress bar.

---

15. FAVORITE ROUTES INTEGRATION

Favorites use:

```
favorite_routes
```

To favorite a route:

```
INSERT INTO favorite_routes (
    passenger_id,
    route_id
)
VALUES (?, ?);
```

To remove:

```
DELETE FROM favorite_routes
WHERE passenger_id = ?
  AND route_id = ?;
```

Favorite routes must appear first.

Use a query similar to:

```
SELECT
    r.*,

    CASE
        WHEN fr.passenger_id IS NULL
        THEN 0
        ELSE 1
    END AS is_favorite

FROM routes r

LEFT JOIN favorite_routes fr
    ON fr.route_id = r.route_id
    AND fr.passenger_id = ?

WHERE r.university_id = ?
  AND r.status = 'ACTIVE'

ORDER BY
    is_favorite DESC,
    r.route_name ASC;
```

---

16. 40-SEAT BOOKING INTERFACE

The standard bus contains:

```
10 rows × 4 seats = 40 seats
```

Generate the visual seat interface using JavaScript.

Seat numbers stored in MySQL are:

```
1–40
```

Display them as:

```
A1 A2 | A3 A4
B1 B2 | B3 B4
C1 C2 | C3 C4
...
J1 J2 | J3 J4
```

The database provides:

`fn_seat_label(seat_number)`

to convert numeric seat values into display labels.

---

17. LOAD BOOKED SEATS FROM MYSQL

Never hard-code booked seats in JavaScript.

Create an API endpoint such as:

`api/available-seats.php`

It should query:

```
SELECT
    seat_number
FROM bookings
WHERE schedule_id = ?
  AND slot_type = 'SEAT'
  AND status IN (
      'BOOKED',
      'CONFIRMED',
      'TRANSFER_PENDING'
  );
```

Return JSON.

JavaScript should use `fetch()` to receive the booked-seat list and visually disable those seats.

---

18. STANDING SLOT INTEGRATION

The system supports:

`10 standing slots`

for the standard demonstration bus.

Do not display standing positions as bookable while normal seats remain available.

Use:

`v_schedule_availability`

or database queries to determine:

`available_seats = 0`

before displaying standing slots.

Standing values are stored in:

`bookings.standing_slot`

---

19. BOOKING INTEGRATION

Do not manually perform several unrelated INSERT operations from PHP when the provided database procedure already handles the booking transaction.

Use:

```
CALL sp_create_booking(
    passenger_id,
    schedule_id,
    slot_type,
    slot_number
);
```

Example PHP PDO integration:

```
$stmt = $pdo->prepare(
    "CALL sp_create_booking(?, ?, ?, ?)"
);

$stmt->execute([
    $_SESSION['passenger_id'],
    $scheduleId,
    $slotType,
    $slotNumber
]);

$booking = $stmt->fetch();
```

The database handles important booking rules.

PHP should catch exceptions and display clean messages.

---

20. DO NOT CALCULATE FARE FROM JAVASCRIPT

JavaScript may display the fare, but it must not determine the authoritative booking amount.

Incorrect:

```
fare = selectedRouteFare;
```

followed by sending that value as trusted data.

The authoritative fare must come from:

```
routes.fare
```

The booking trigger/procedure handles this server-side.

---

21. BOOKING DATABASE AUTOMATION

When a booking succeeds, the provided database automatically handles important operations including:

- Booking validation
- Student/Faculty eligibility
- Same-university validation
- Conflicting-booking detection
- Seat validation
- Standing-slot validation

- Fare retrieval
- Booking reference generation
- QR token generation
- Billing transaction
- Semester bill update
- Booking history
- Notification creation

Do not duplicate or fight this logic in PHP.

PHP should provide additional validation for user experience, but the database remains the final integrity layer.

---

22. BOOKING REFERENCE

The application should display:

`booking_reference`

as the user-friendly ticket reference.

Do not expose only the numeric database ID as the primary public ticket identifier.

---

23. QR CODE INTEGRATION

The database already stores:

`bookings.qr_token`

Use this token to generate the QR code.

After booking:

1. Retrieve the booking.
2. Retrieve **`qr_token`**.
3. Generate a QR image using a local PHP or JavaScript QR library.
4. Display it on the confirmation page.

Do not store sensitive passenger information directly inside the QR code.

The QR should contain a secure identifier such as:

`qr_token`

or a local ticket-verification URL containing the token.

---

24. BOOKING CONFIRMATION

Create:

passenger/booking-confirmation.php

Retrieve booking information using SQL joins or:

v_passenger_booking_history

Display:

- Booking reference
- Passenger
- University
- Route
- Start location
- End location
- Bus
- Bus type
- Date
- Departure
- Arrival
- Seat/standing slot
- Fare
- Status
- QR code

---

25. BOOKING HISTORY INTEGRATION

Use:

v_passenger_booking_history

Example:

```
SELECT *
FROM v_passenger_booking_history
WHERE passenger_id = ?
  AND hidden_from_passenger = 0
ORDER BY booking_date DESC;
```

Do not create a separate booking-history table when the existing database already maintains history appropriately.

---

26. BOOKING CANCELLATION

Use the existing stored procedure:

```
CALL sp_cancel_booking(
    booking_id,
    passenger_id
);
```

Example PHP:

```
$stmt = $pdo->prepare(
    "CALL sp_cancel_booking(?, ?)"
);

$stmt->execute([
    $bookingId,
    $_SESSION['passenger_id']
]);
```

Cancellation automatically handles related database operations such as releasing the active slot and adjusting billing.

Do not delete the booking row.

---

27. CLEAR HISTORY

The user-facing "Clear History" function should call:

```
CALL sp_archive_booking_history(
    booking_id,
    passenger_id
);
```

This hides eligible history from the passenger instead of destroying important records.

---

28. TICKET SHARE / SALE INTEGRATION

Ticket sharing/selling must use:

```
ticket_transfers
```

Do not simply change:

```
bookings.passenger_id
```

directly from a normal PHP form.

Create the transfer through:

```
CALL sp_request_ticket_transfer(
    booking_id,
    from_passenger_id,
    to_passenger_id,
    transfer_type,
    sale_amount
);
```

Example:

```
$stmt = $pdo->prepare(
    "CALL sp_request_ticket_transfer(?, ?, ?, ?, ?)"
);

$stmt->execute([
    $bookingId,
```

```
    $_SESSION['passenger_id'],
    $recipientId,
    $transferType,
    $saleAmount
]);
```

29. ACCEPT / REJECT TRANSFER

Recipients must use:

```
CALL sp_respond_ticket_transfer(
    transfer_id,
    recipient_passenger_id,
    action
);
```

Valid actions:

```
ACCEPT
REJECT
```

The procedure must be used so ownership and transfer history remain consistent.

30. STUDENT TICKET SHARING

The required feature:

Students can sell/share tickets among themselves.

must be implemented using the database transfer workflow.

At minimum:

Student → Student

must work.

The current database also protects:

- Same-university requirement
- Same passenger type
- Bus-type eligibility
- Recipient booking conflict
- Ticket ownership
- Ticket status

Display database validation errors cleanly.

31. COMPLAINT INTEGRATION

Use:

complaints

Passenger submission:

```
INSERT INTO complaints (
    passenger_id,
    university_id,
    subject,
    description
)
VALUES (?, ?, ?, ?);
```

Use:

```
$_SESSION['passenger_id']
$_SESSION['university_id']
```

rather than trusting IDs provided by the browser.

---

32. UNIVERSITY COMPLAINT MANAGEMENT

University dashboard query:

```
SELECT
    c.*,
    p.name AS passenger_name,
    p.passenger_type

FROM complaints c

JOIN passengers p
    ON p.passenger_id = c.passenger_id

WHERE c.university_id = ?

ORDER BY c.submitted_at DESC;
```

University administrators can update:

```
status
university_response
```

The database generates passenger notifications when complaints are updated.

---

33. NOTIFICATION INTEGRATION

Notifications use:

```
notifications
```

Display unread count:

```
SELECT COUNT(*) AS unread_count
FROM notifications
WHERE passenger_id = ?
  AND is_read = 0;
```

Display notification popover:

```
SELECT *
FROM notifications
WHERE passenger_id = ?
ORDER BY created_at DESC
LIMIT 30;
```

34. MARK NOTIFICATION READ

For a single notification:

```
UPDATE notifications
SET
    is_read = 1,
    read_at = NOW()
WHERE notification_id = ?
  AND passenger_id = ?;
```

For all notifications:

```
CALL sp_mark_all_notifications_read(?);
```

Use AJAX/fetch so the bell interface can update without a complete page reload where appropriate.

---

35. SEMESTER BILL INTEGRATION

Use:

```
semester_bills
billing_transactions
semesters
```

Passenger's current semester bill can be retrieved through:

```
v_semester_transport_charges
```

Example:

```
SELECT *
FROM v_semester_transport_charges
WHERE passenger_id = ?
ORDER BY semester_id DESC;
```

Do not calculate an unrelated semester total in JavaScript.

---

36. BILLING TRANSACTION HISTORY

Display detailed charges using:

```
SELECT
    bt.transaction_id,
    bt.transaction_type,
    bt.amount,
    bt.description,
    bt.transaction_date,
    sem.semester_name

FROM billing_transactions bt

JOIN semesters sem
    ON sem.semester_id = bt.semester_id

WHERE bt.passenger_id = ?

ORDER BY bt.transaction_date DESC;
```

---

37. UNIVERSITY DASHBOARD INTEGRATION

Use the existing database view:

```
v_university_dashboard_stats
```

Example:

```
SELECT *
FROM v_university_dashboard_stats
WHERE university_id = ?;
```

Use it to populate dashboard cards such as:

- Total buses
- Active buses
- Total routes
- Active routes
- Total passengers
- Students
- Faculty
- Total bookings
- Pending complaints

Do not hard-code dashboard numbers.

---

38. OCCUPANCY PROGRESS BAR

Use:

```
v_schedule_availability.seated_occupancy_percent
```

Example HTML/PHP:

```
<div class="progress">
    <div
        class="progress-bar"
        style="width:
        <?= htmlspecialchars($row['seated_occupancy_percent']) ?>%">
    </div>
</div>
```

Also display:

```
booked_seats / seat_capacity
```

and, when full:

```
booked_standing / standing_capacity
```

---

39. PROFILE INTEGRATION

Passenger profile data comes from:

```
passengers
students
faculty
```

Allow updating permitted fields such as:

```
name
avatar_path
email_notifications
in_app_notifications
```

Do not let passengers modify:

```
university_id
passenger_type
passenger_id
```

through a normal profile form.

---

40. AVATAR STORAGE

Uploaded avatar files should be saved in a directory such as:

```
assets/uploads/avatars/
```

The database stores only:

```
avatar_path
```

Validate:

- MIME type
- Extension
- File size

Use randomized filenames.

---

41. PASSWORD CHANGE DATABASE INTEGRATION

Retrieve the logged-in passenger's:

```
password_hash
```

Verify current password using:

```
password_verify()
```

Then create a new hash:

```
$newHash = password_hash(
    $newPassword,
    PASSWORD_DEFAULT
);
```

Update:

```
UPDATE passengers
SET password_hash = ?
WHERE passenger_id = ?;
```

Perform equivalent secure logic for university administrators and System Admin accounts.

---

42. PHP ERROR HANDLING

Database triggers and procedures may intentionally throw business-rule exceptions.

PHP must catch:

```
PDOException
```

and convert expected business-rule errors into clean messages.

Example:

```
try {

    $stmt = $pdo->prepare(
        "CALL sp_create_booking(?, ?, ?, ?)"
    );

    $stmt->execute([
        $passengerId,
        $scheduleId,
        $slotType,
        $slotNumber
    ]);

} catch (PDOException $e) {

    $message = $e->getMessage();

    // Log technical details privately.
    // Display an appropriate clean message to the user.

}
```

Never display database credentials or stack traces publicly.

---

43. USE PREPARED STATEMENTS EVERYWHERE

Never write:

```
$sql = "SELECT * FROM passengers WHERE email = '$email'";
```

Instead:

```
$stmt = $pdo->prepare(
    "SELECT *
     FROM passengers
     WHERE email = ?"
);

$stmt->execute([$email]);
```

All user-supplied values must use prepared statements.

---

44. FETCH / AJAX API ENDPOINTS

Create secure PHP API endpoints where useful.

Suggested endpoints:

```
api/get-schedules.php
api/get-seat-availability.php
api/create-booking.php
api/cancel-booking.php
api/toggle-favorite.php
api/get-notifications.php
api/mark-notification-read.php
api/request-transfer.php
api/respond-transfer.php
```

Every API endpoint must:

1. Start/check PHP session.
2. Verify authentication.
3. Verify authorization.
4. Validate inputs.
5. Use prepared statements.
6. Return JSON.
7. Never trust client-supplied user ownership.

---

45. JSON API FORMAT

Use consistent responses such as:

```
{
    "success": true,
    "message": "Booking confirmed.",
    "data": {}
}
```

For errors:

```
{
    "success": false,
    "message": "This seat is already booked."
}
```

Do not expose raw SQL errors.

---

46. CSRF PROTECTION

State-changing requests should use CSRF tokens where practical.

Protect actions such as:

- Add university
- Add bus
- Add route
- Add schedule

- Booking
- Cancellation
- Transfer
- Complaint
- Profile update
- Password change

---

47. DATABASE FILE

Include the supplied SQL database as:

`database/uniride_complete_phpmyadmin.sql`

or:

`database/university_bus_system.sql`

Do not silently create a different schema.

The README must instruct the user to import this SQL file through phpMyAdmin.

---

48. INSTALLATION

README instructions must include:

1. Install XAMPP.
2. Start Apache and MySQL.
3. Copy project folder into **`htdocs`**.
4. Open phpMyAdmin.
5. Import the provided SQL file.
6. Confirm database **`uniride_db`** exists.
7. Open:

`config/database.php`

8. Configure MySQL credentials.
9. Visit the application through localhost.
10. Login using demo credentials.

---

49. DATABASE-FIRST DEVELOPMENT RULE

Before creating each PHP page, identify exactly which existing database table, view, function, trigger, or procedure the page will use.

For example:

```
Passenger Dashboard
→ v_schedule_availability
→ bookings
→ favorite_routes
→ notifications
→ semester_bills

Seat Booking
→ schedules
→ bookings
→ sp_create_booking()

My Bookings
→ v_passenger_booking_history

Cancellation
→ sp_cancel_booking()

Ticket Transfer
→ ticket_transfers
→ sp_request_ticket_transfer()
→ sp_respond_ticket_transfer()

Complaints
→ complaints

Notifications
→ notifications
→ sp_mark_all_notifications_read()

University Dashboard
→ v_university_dashboard_stats
```

The code and database must operate as one integrated system.

---

50. DO NOT DUPLICATE DATABASE BUSINESS LOGIC INCORRECTLY

Some important integrity rules already exist inside MySQL.

PHP should provide user-friendly validation, but do not create PHP behavior that contradicts the database.

Examples of database-enforced rules include:

- Same-university relationships
- Student/Faculty eligibility
- Duplicate-seat prevention
- Bus schedule conflicts
- Passenger schedule conflicts
- Standing-slot restrictions
- Semester billing
- Ticket-transfer validation
- Favorite-route ownership
- Complaint university ownership

Treat the database as the final authority for integrity.

51. FRONTEND ↔ PHP ↔ MYSQL FLOW

Every interactive feature should follow this architecture:

```
HTML/CSS Interface
        ↓
Vanilla JavaScript
        ↓
fetch()/Form Request
        ↓
PHP Controller/API
        ↓
Authentication + Validation
        ↓
PDO Prepared Statement
        ↓
MySQL uniride_db
        ↓
Result
        ↓
PHP JSON/HTML Response
        ↓
JavaScript updates UI
```

Never connect JavaScript directly to MySQL.

---

52. COMPLETE INTEGRATION TEST

Before declaring the application finished, perform an end-to-end test.

Authentication

Verify:

- System Admin can login.
- University Admin can login.
- Passenger can login.
- Password hashes work.
- Sessions work.
- Unauthorized roles are blocked.

University Isolation

Verify:

- MCU administrator cannot access GFU records.
- GFU administrator cannot access ETU records.
- Passenger cannot access another university's private data.

Routes

Verify:

- Routes are read from MySQL.
- Favorite route persists.
- Favorite route appears first.

Schedules

Verify:

- Schedules come from MySQL.
- Bus and route details join correctly.
- Occupancy is database-driven.

Booking

Verify:

- Seat map reads booked seats from MySQL.
- Available seat can be selected.
- Booking calls **`sp_create_booking()`**.
- Booking appears in **`bookings`**.
- Duplicate seat is blocked.
- Conflicting passenger booking is blocked.
- Student cannot use Faculty bus.
- Faculty cannot use Student bus.

Standing Slots

Verify:

- Standing slots remain unavailable while seats remain.
- Slots become available after seats fill.
- Standing slot duplication is blocked.
- Maximum standing capacity is respected.

QR Ticket

Verify:

- `qr_token` exists.
- QR code appears.
- Booking confirmation displays correct database information.

Cancellation

Verify:

- `sp_cancel_booking()` works.
- Booking becomes cancelled.
- Seat becomes available.
- Semester bill adjusts.
- Notification is generated.

Ticket Transfer

Verify:

- Student can share/sell to eligible Student.
- Invalid recipient is rejected.
- Transfer appears in `ticket_transfers`.
- Acceptance changes current ticket holder correctly.
- Transfer history remains preserved.

Complaints

Verify:

- Complaint inserts into MySQL.
- Correct university receives it.
- University response updates database.
- Passenger receives notification.

Notifications

Verify:

- Bell count comes from MySQL.
- Notifications load.

- Mark-as-read persists.
- Mark-all-read works.

Billing

Verify:

- Booking creates charge.
- Semester bill updates.
- Cancellation adjustment works.
- Passenger sees accurate balance.

Profile

Verify:

- Name update persists.
- Avatar path persists.
- Preferences persist.
- Password change works using secure hash verification.

---

53. CRITICAL FINAL DATABASE INSTRUCTION

Do not build the frontend first and then simulate database behavior.

The application must be implemented around the supplied **`uniride_db`** database from the beginning.

For every feature:

1. Identify the database relation.
2. Write the secure PHP query/procedure call.
3. Verify the database transaction.
4. Then connect the frontend interface.
5. Test persistence by refreshing the page and verifying the data remains.

No important application state should exist only inside JavaScript.

---

54. FINAL COMMAND TO THE CODING AGENT

Use the supplied UniRide SQL database as the **single source of truth for persistent application data**.

Integrate every HTML/CSS/JavaScript page with PHP and MySQL.

Do not return disconnected frontend mockups.

Do not replace the supplied schema.

Do not hard-code database records.

Do not fake booking availability.

Do not fake dashboard statistics.

Do not fake notifications.

Do not fake semester bills.

Do not use browser localStorage as a substitute for the database.

Build the application so that:

HTML/CSS/JavaScript → PHP → PDO → `uniride_db`

works end-to-end.

If the supplied database already provides a stored procedure, view, trigger, function, key, or constraint for a feature, **use it rather than creating a conflicting alternative**.

The project is only complete when the supplied SQL database and the PHP application function together as one complete working system.

Use this section together with the earlier **Master Development Prompt** and give your coding agent the SQL file/schema as well. This closes an important loophole: the agent is now explicitly told to build against **`uniride_db`**, use its existing tables/views/procedures, and connect every UI feature to real MySQL data rather than generating a separate mock database.
Today 2:11 AM

the for students and faculty, the email should look like student.name@g.universityinitial.acbd. for example: samir.hossain@g.bracu.ac.bd, for admin, it should also look like an official main which contains a Google address, like Gmail. now create a prompt so that i can give it to my ai agent to integrate this features
Show moreShow less

Yes. I'd standardize the student/faculty format as **`.ac.bd`** because that matches your example **`samir.hossain@g.bracu.ac.bd`**. The coding agent should update **both the database demo data and**

every place these accounts appear in the UI/README, so you don't end up with one email on the login page and another in MySQL.

Integrate Realistic University Email Addresses and Demo Accounts

Update the existing **UniRide — Multi-University Bus Ticket Management System** so that all demo users use professional and realistic email-address formats.

Do not rebuild the project from scratch.

Modify the **existing PHP application, MySQL database seed/demo data, login pages, README, demo account section, and any hard-coded sample data** to follow the email conventions below.

The existing technology stack remains:

- HTML5
- CSS3
- Vanilla JavaScript
- PHP
- MySQL
- phpMyAdmin
- XAMPP

The database remains:

```
uniride_db
```

---

1. STUDENT EMAIL FORMAT

Every Student account must use a realistic university Google Workspace-style academic email.

Required format:

```
firstname.lastname@g.universityinitial.ac.bd
```

Example:

```
samir.hossain@g.bracu.ac.bd
```

Other examples:

```
samiha.tasnim@g.bracu.ac.bd
nabil.hossain@g.bracu.ac.bd
rafi.hasan@g.nsu.ac.bd
tahmid.islam@g.aiub.ac.bd
```

Rules:

- Use lowercase only.

- Use the student's real demo first name.
- Use a period between first and last name.
- Use `@g.` before the university domain.
- End university academic domains with `.ac.bd`.
- Do not use `.demo`.
- Do not use fake formats such as:

```
student@mcu.demo
student123@test.com
user@example.com
```

---

2. FACULTY EMAIL FORMAT

Faculty members must follow the same university Google Workspace-style academic format.

Required pattern:

```
firstname.lastname@g.universityinitial.ac.bd
```

Examples:

```
kamal.uddin@g.bracu.ac.bd
nusrat.karim@g.bracu.ac.bd
samira.ahmed@g.nsu.ac.bd
farhana.noor@g.aiub.ac.bd
```

Student and Faculty emails must remain unique.

---

3. UNIVERSITY DOMAIN FORMAT

Each university must have a consistent university abbreviation/domain.

For demonstration, use realistic fictional or project-approved university domains consistently.

For example:

BRAC University-style demo

```
bracu.ac.bd
```

Student/Faculty Google Workspace:

```
@g.bracu.ac.bd
```

North South University-style demo

```
nsu.ac.bd
```

Student/Faculty Google Workspace:

```
@g.nsu.ac.bd
```

AIUB-style demo

aiub.ac.bd

Student/Faculty Google Workspace:

@g.aiub.ac.bd

If the project's existing university names are kept instead of these universities, create equivalent academic domains based on each university's abbreviation.

The important rule is:

Student / Faculty:
firstname.lastname@g.<university-domain>.ac.bd

Do not mix several domain styles for members of the same university.

---

4. SYSTEM ADMIN EMAIL

The central UniRide System Admin should use a professional Gmail address.

Use a format such as:

uniride.admin@gmail.com

or:

uniride.system@gmail.com

Preferred demo account:

uniride.admin@gmail.com

Do not use:

admin@uniride.demo

---

5. UNIVERSITY ADMIN EMAIL FORMAT

University transport administrators should also use professional-looking administration email accounts.

Use Gmail-style addresses for the demo administrators as requested.

Recommended format:

<university>.transport@gmail.com

Examples:

bracu.transport@gmail.com
nsu.transport@gmail.com
aiub.transport@gmail.com

Alternatively, a professional admin pattern may be:

bracu.busadmin@gmail.com
nsu.busadmin@gmail.com
aiub.busadmin@gmail.com

Use **one consistent pattern throughout the project**.

Preferred pattern:

```
<university>.transport@gmail.com
```

---

6. FINAL RECOMMENDED DEMO ACCOUNTS

Update the demonstration accounts to use a clean structure similar to this.

System Admin

```
Email: uniride.admin@gmail.com
Password: password
Role: SYSTEM_ADMIN
```

University Admin — BRACU

```
Email: bracu.transport@gmail.com
Password: password
Role: UNIVERSITY_ADMIN
```

University Admin — NSU

```
Email: nsu.transport@gmail.com
Password: password
Role: UNIVERSITY_ADMIN
```

University Admin — AIUB

```
Email: aiub.transport@gmail.com
Password: password
Role: UNIVERSITY_ADMIN
```

Student — BRACU

```
Name: Samiha Tasnim
Email: samiha.tasnim@g.bracu.ac.bd
Password: password
Type: STUDENT
```

Second Student — BRACU

```
Name: Samir Hossain
Email: samir.hossain@g.bracu.ac.bd
Password: password
Type: STUDENT
```

Faculty — BRACU

```
Name: Kamal Uddin
Email: kamal.uddin@g.bracu.ac.bd
Password: password
Type: FACULTY
```

Student — NSU

```
Name: Rafi Hasan
Email: rafi.hasan@g.nsu.ac.bd
Password: password
Type: STUDENT
```

Faculty — NSU

```
Name: Samira Ahmed
Email: samira.ahmed@g.nsu.ac.bd
Password: password
Type: FACULTY
```

Student — AIUB

```
Name: Tahmid Islam
Email: tahmid.islam@g.aiub.ac.bd
Password: password
Type: STUDENT
```

Faculty — AIUB

```
Name: Farhana Noor
Email: farhana.noor@g.aiub.ac.bd
Password: password
Type: FACULTY
```

---

7. IMPORTANT PASSWORD REQUIREMENT

The visible demo password may be:

```
password
```

But **never store the literal plain-text word `password` in the database password field**.

All demo passwords must be stored using PHP:

```
password_hash('password', PASSWORD_DEFAULT);
```

Login must continue to verify passwords using:

```
password_verify($password, $passwordHash);
```

Do not compare passwords using SQL or plain-text equality.

---

8. UPDATE MYSQL DEMO DATA

Modify the SQL seed/demo records in:

```
database/uniride_complete_phpmyadmin.sql
```

Update the appropriate:

```
admins.email
university_users.email
passengers.email
```

records.

For example, replace old values such as:

```
admin@uniride.demo
mcu.admin@uniride.com
samiha.tasnim@mcu.edu
kamal.uddin@mcu.edu
```

with the new professional formats.

Example SQL concept:

```
UPDATE admins
SET email = 'uniride.admin@gmail.com'
WHERE admin_id = 1;
```

University Admin example:

```
UPDATE university_users
SET email = 'bracu.transport@gmail.com'
WHERE university_id = 1;
```

Student example:

```
UPDATE passengers
SET email = 'samiha.tasnim@g.bracu.ac.bd'
WHERE passenger_id = 1;
```

Faculty example:

```
UPDATE passengers
SET email = 'kamal.uddin@g.bracu.ac.bd'
WHERE passenger_id = 2;
```

However, update the actual seed `INSERT` statements as well so a **fresh phpMyAdmin import already contains the correct emails**.

Do not rely only on UPDATE queries executed after import.

---

9. UPDATE UNIVERSITY DATA

If the current demo universities use placeholder names such as:

```
MCU
GFU
ETU
```

either keep those university names with consistent custom academic domains, or—if the project has already decided to demonstrate BRACU, NSU, and AIUB—update the complete demo dataset consistently.

Do not create mismatched data such as:

```
University Name: Metro City University
Email domain: g.bracu.ac.bd
```

University name, abbreviation, university email, passenger emails, routes, admins, and labels should all correspond consistently.

---

10. EMAIL VALIDATION

Add backend validation appropriate to each account type.

Student / Faculty

The system should allow university academic email formats such as:

```
firstname.lastname@g.bracu.ac.bd
```

Do not hard-code only `bracu`.

Validation should work dynamically according to the passenger's university.

Concept:

```
@g.<university-domain>.ac.bd
```

If a passenger belongs to BRACU, their demo academic email should correspond to BRACU.

If a passenger belongs to NSU, it should correspond to NSU.

Do not allow a BRACU passenger to be accidentally seeded with an NSU domain.

---

11. DO NOT USE CLIENT-SIDE VALIDATION ALONE

JavaScript may validate the email format for user convenience.

PHP must validate it again.

Example:

```
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    // reject invalid email
}
```

Additional university-domain validation may then be performed server-side.

Do not rely only on HTML:

```
<input type="email">
```

---

12. EMAIL MUST REMAIN UNIQUE

Keep database uniqueness constraints.

A single email cannot belong to multiple accounts of the same authentication domain.

For passengers:

```
passengers.email
```

must remain unique according to the existing schema.

University Admin emails must also remain unique.

System Admin emails must remain unique.

---

13. LOGIN PAGE UPDATE

Update the login forms and demo-account hints.

If the login page displays example credentials, use the new addresses.

Example Passenger Login:

```
Email
samiha.tasnim@g.bracu.ac.bd

Password
password
```

University Admin:

```
bracu.transport@gmail.com
password
```

System Admin:

```
uniride.admin@gmail.com
password
```

Remove outdated examples.

---

14. LOGIN LOGIC MUST CONTINUE TO USE DATABASE EMAILS

Do not hard-code these demo emails in the authentication logic.

Correct architecture:

```
Login Form
    ↓
PHP
    ↓
SELECT user using submitted email
    ↓
password_verify()
    ↓
PHP Session
```

The demo addresses are only database data.

Authentication must work for any future valid user.

---

15. README UPDATE

Update the:

```
How to Run & Verify
```

section.

Use a clean version similar to:

```
4. How to Run & Verify

1. Start Apache and MySQL in XAMPP.

2. Copy the uniride folder to:

   C:\xampp\htdocs\uniride

3. Open phpMyAdmin:

   http://localhost/phpmyadmin/

4. Import:

   database/uniride_complete_phpmyadmin.sql

5. Access the application:

   http://localhost/uniride/
```

Do not show a developer's absolute local path such as:

```
d:\PRTOM\cse370\projects\uniride\database\...
```

because that path will not exist on another computer.

Instead write:

```
Import:
database/uniride_complete_phpmyadmin.sql
```

---

16. README DEMO LOGIN SECTION

Replace the old demo section with:

```
Demo Login Accounts
```

System Admin

```
Email: uniride.admin@gmail.com
Password: password
```

BRACU University Admin

```
Email: bracu.transport@gmail.com
Password: password
```

NSU University Admin

```
Email: nsu.transport@gmail.com
Password: password
```

AIUB University Admin

```
Email: aiub.transport@gmail.com
Password: password
```

Student — BRACU

```
Email: samiha.tasnim@g.bracu.ac.bd
Password: password
```

Faculty — BRACU

```
Email: kamal.uddin@g.bracu.ac.bd
Password: password
```

---

17. UI PROFILE DISPLAY

When a logged-in Passenger views their profile, display the professional university email.

Example:

```
Samiha Tasnim
Student
samiha.tasnim@g.bracu.ac.bd
BRAC University
```

Do not display old placeholder emails anywhere in the interface.

---

18. ADMIN DASHBOARD DISPLAY

When University Admin information is shown, display:

```
BRACU Transport Admin
bracu.transport@gmail.com
```

instead of old test/demo addresses.

System Admin should display:

```
UniRide System Admin
uniride.admin@gmail.com
```

---

19. SEARCH THE ENTIRE PROJECT FOR OLD EMAILS

Before completing the change, search the entire codebase for:

```
@uniride.com
@uniride.demo
@mcu.edu
@mcu.demo
@gfu.demo
@etu.demo
@example.com
```

and other outdated demonstration addresses.

Replace them where appropriate.

Check:

- SQL files
- PHP files

- HTML
- JavaScript
- README
- Login page
- Dashboard
- Profile page
- Screenshots/demo text where editable
- Comments containing demo credentials
- Sample API data
- Database seed scripts

Do not leave contradictory credentials in different files.

---

20. DO NOT BREAK EXISTING FOREIGN KEYS

Changing emails must not require changing:

```
passenger_id
university_id
admin_id
university_user_id
```

unless the broader demo university dataset is intentionally being migrated.

Relationships must remain valid.

Emails are account/login attributes, not replacement relational primary keys.

---

21. UNIVERSITY DOMAIN MAPPING

For maintainability, create a consistent mapping between universities and their email-domain abbreviations.

Example concept:

```
BRAC University
→ bracu
→ g.bracu.ac.bd

North South University
→ nsu
→ g.nsu.ac.bd

American International University-Bangladesh
→ aiub
→ g.aiub.ac.bd
```

If beneficial, add a field to the University table such as:

```
academic_domain
```

Example:

```
bracu.ac.bd
```

Then passenger-email validation can derive:

```
g.bracu.ac.bd
```

from the university record.

Only introduce this migration if implemented consistently in the SQL schema, seed data, PHP code, and README.

---

22. OPTIONAL AUTOMATIC EMAIL GENERATION

When University Admin creates a new passenger, optionally auto-suggest an academic email based on their name.

For example:

Input:

```
First Name: Samir
Last Name: Hossain
University: BRACU
```

Suggested:

```
samir.hossain@g.bracu.ac.bd
```

If the email already exists, require the administrator to resolve the duplicate rather than silently creating conflicting accounts.

This feature should be a convenience only.

The final email must still be stored in MySQL.

---

23. FINAL VERIFICATION

Before declaring this change complete, verify all of the following.

Database

Confirm:

```
SELECT email FROM admins;
```

returns the new Gmail-style System Admin address.

Confirm:

```
SELECT email FROM university_users;
```

returns the new professional university-admin Gmail addresses.

Confirm:

```
SELECT
    name,
    email,
```

```
    passenger_type
FROM passengers;
```

returns academic addresses in the required format.

Student Login

Verify:

samiha.tasnim@g.bracu.ac.bd

can log in using:

password

Faculty Login

Verify:

kamal.uddin@g.bracu.ac.bd

can log in using:

password

University Admin Login

Verify:

bracu.transport@gmail.com

can log in.

System Admin Login

Verify:

uniride.admin@gmail.com

can log in.

UI Consistency

Verify the same email addresses appear correctly in:

- Login examples
- Passenger profile
- Admin profile
- Dashboard
- README

- Database
- Demo account documentation

---

24. IMPORTANT IMPLEMENTATION INSTRUCTION

Do not merely change the README text.

Actually update:

```
MySQL seed data
        +
PHP authentication
        +
profile display
        +
login demo information
        +
README
```

so all components use the same accounts.

The final architecture must remain:

```
HTML / CSS / JavaScript
          ↓
         PHP
          ↓
         PDO
          ↓
      uniride_db
```

The credentials shown in the README must correspond to **real records inside** `uniride_db`.

FINAL COMMAND

Implement this change across the existing UniRide project.

Do **not** rebuild or replace existing working booking, route, schedule, complaint, ticket transfer, favorite-route, notification, QR, semester-billing, or database functionality.

Only extend/refine the account and email system while preserving existing features.

The required account-email convention is:

```
Students:
firstname.lastname@g.universityinitial.ac.bd

Faculty:
firstname.lastname@g.universityinitial.ac.bd

System Admin:
uniride.admin@gmail.com

University Admin:
university.transport@gmail.com
```

Use `.ac.bd`, **not** `.acbd`.

Ensure the database, PHP application, UI, demo credentials, and README are all synchronized before considering the implementation complete.

Route-01: Abdullahpur-A to BRAC University

Stoppages	1st Trip (Pickup time)	2nd Trip (Pickup time)
Abdullahpur (Polwel Carnation Shopping Centre)	6:40 AM	8:30 AM
House Building (Labaid Diagnostic)	6:43 AM	8:33 AM
Azampur (In front of Uttara East Police Station)	6:46 AM	8:36 AM
Jasimuddin (Popular Diagnostic Centre)	6:49 AM	8:39 AM
Airport (In front of passenger shed situated 50 yards back)	6:51 AM	8:43 AM
Kawla (near foot over-bridge)	6:52 AM	8:44 AM
Khilket (first foot over-bridge)	6:56 AM	8:47 AM
Biswa Road (near foot over-bridge)	6:59 AM	8:49 AM
BRAC University (arrival time)	7:30 AM	10:40 AM

Route-01: Abdullahpur-B to BRAC University

Stoppages	1st Trip (Pickup time)
Jashimuddin (Opposite to RAK Shopping Complex, Uttara Sector #01, Road #14	6:20 AM
Korotoa Courier Point (Opposite to Korotoa Courier Service, Uttara Sector #03, Road #18	6:22 AM
Shahid Mugdho Corner (In front of Shahid Mugdho Corner (Robindro Sorobor)	6:23 AM
Lake Drive Road – Uttara Model Town, Opposite to Brac Bank, Sector #07	6:25 AM
Zam Zam Tower (In front of Zam Zam Tower, Sector #13, Uttara	6:27 AM
Tagore University Point (In front of Tagore University of Creative Arts, Sector #12	6:29 AM
Khalpar Mor – In front of Police Box	6:30 AM
Diabari Gol Chattar (In front of Fresh Ceramics, Uttara)	6:32 AM
Uttara Centre (MRT Station) - Under Metro Rail Station	6:34 AM
Mirpur Ceramics (near CNG filling station)	6:38 AM
Mirpur DOHS Gate (near Trust bus stop)	6:43 AM
Pallabi Thana (Shagufta mour)	6:45 AM
Kalshi Bus Stand	6:48 AM
ECB Chattar	6:51 AM
BRAC University (arrival time)	7:40 AM

Route-02: Mirpur-A to BRAC University

Stoppages	1st Trip (Pickup time)	2nd Trip (Pickup time)
Mirpur Bangla College	6:20 AM	8:20 AM
Dhaka Laboratory School and College (In front of Staff Quarter)	6:22 AM	8:22 AM
Sony Cinema Hall (near KFC)	6:24 AM	8:25 AM
Mirpur College Gate (Mirpur 2)	6:26 AM	8:27 AM
Swimming Federation Gate (Mirpur stadium)	6:28 AM	8:29 AM
Popular Diagnostic Centre (Mirpur 6)	6:31 AM	8:33 AM
Pallabi Post Office (near Purobi Cinema Hall)	6:34 AM	8:37 AM
Mirpur 11.5 (Banalata Sweets)	6:35 AM	8:38 AM
Mirpur Ceramics (near CNG Filling Station)	6:36 AM	8:40 AM
Mirpur DOHS Gate (near Trust Bus Stop)	6:40 AM	8:45 AM
Pallabi Thana (Shagufta mour)	6:43 AM	8:47 AM
Kalshi Bus Stand	6:45 AM	8:49 AM
ECB Chattar	6:48 AM	8:52 AM
BRAC University (arrival time)	7:30 AM	10:20 AM

Route-02: Mirpur-B to BRAC University

Stoppages	1st Trip (Pickup time)	2nd Trip (Pickup time)
Mirpur 14 Bus Stand (opposite to Marks Medical College Hospital)	6:35 AM	8:35 AM
In front of NAM Garden Officers Quarter, Mirpur 13 (near BRTA 2nd Gate)	6:37 AM	8:37 AM
Opposite to Water Tank (Mirpur 10)	6:39 AM	8:40 AM
Al Helal Specialized Hospital (Mirpur 10)	6:43 AM	8:44 AM
In front of BRAC Healthcare, Kazipara	6:46 AM	8:46 AM
Shewrapara (near Jamuna Plaza and Metro Rail Station)	6:48 AM	8:48 AM
Taltola Bus Stand (opposite to Glory School & College)	6:51 AM	8:50 AM
BRAC University (arrival time)	7:30 AM	10:30 AM

Route-03: Jigatola-A to BRAC University

Stoppages	1st Trip (Pickup time)	2nd Trip (Pickup time)
Jigatola Bus Stand (in front of Japan-Bangladesh Friendship Hospital)	6:35 AM	8:40 AM
Keari Plaza, Dhanmondi 15 Bus Stand	6:37 AM	8:42 AM
Shankar Bus Stand	6:40 AM	8:45 AM
Meena Bazar (Dhanmondi 27)	6:42 AM	8:47 AM
In front of Lalmatia Mother Care Hospital	6:45 AM	8:50 AM
BRAC University (arrival time)	7:30 AM	10:30 AM

Route-03: Jigatola-B to BRAC University

Stoppages	1st Trip (Pickup time)
Jigatola Bus Stand (in front of Japan-Bangladesh Friendship Hospital)	7:40 AM
Keari Plaza, Dhanmondi 15 Bus Stand	7:42 AM
Shankar Bus Stand	7:45 AM
Meena Bazar (Dhanmondi 27)	7:47 AM
In front of Lalmatia Mother Care Hospital	7:50 AM
BRAC University (arrival time)	8:45 AM

Route-04: Azimpur to BRAC University

Stoppages	1st Trip (Pickup time)	2nd Trip (Pickup time)
Azimpur Etimkhana Mour	6:25 AM	8:30 AM
Azimpur Chowrasta	6:27 AM	8:33 AM
New Market (gate no 2, Janata Bank)	6:29 AM	8:36 AM
Happy Arcade Shopping Mall (beside City College)	6:31 AM	8:39 AM
Dhanmondi, Road No 7 (ARA Center Shopping Mall)	6:33 AM	8:42 AM
Kalabagan Kriza Chokro (near foot over-bridge)	6:35 AM	8:44 AM
Sobhanbag (near foot over-bridge)	6:37 AM	8:47 AM
West End of Manik Mia Avenue (opposite Aarong)	6:39 AM	8:51 AM
Khejur Bagan Mour (Manik Mia Avenue)	6:41 AM	8:53 AM
BRAC University (arrival time)	7:30 AM	10:45 AM

Route-05: Baldha Garden to BRAC University

Stoppages	1st Trip (Pickup time)	2nd Trip (Pickup time)
Doyaganj Mour (near mandir)	6:30 AM	8:30 AM
Baldha Garden	6:35 AM	8:35 AM
Ittefaq Mour	6:38 AM	8:38 AM
Motijheel (foot over-bridge)	6:40 AM	8:40 AM
Arambag (near Janata Bank)	6:41 AM	8:42 AM
Fakirapool Mour (Apex showroom)	6:43 AM	8:44 AM
Purana Paltan (Hatil showroom)	6:44 AM	8:45 AM
Kakrail Mour (near Eastern Bank)	6:46 AM	8:48 AM
Shantinagar Mour (One Bank)	6:48 AM	8:50 AM
Moghbazar T&T Office	6:51 AM	8:54 AM
Moghbazar Mour (NCC Bank)	6:55 AM	9:00 AM
BRAC University (arrival time)	7:40 AM	10:30 AM

Route-06: Mohammadpur-A to BRAC University

Stoppages	1st Trip (Pickup time)	2nd Trip (Pickup time)
Shiya Masjid (near Top Ten shop)	6:30 AM	8:40 AM
Opposite to Suchona Community Center	6:32 AM	8:42 AM
Shampa Market	6:34 AM	8:45 AM
Shyamoli (Bata mour)	6:37 AM	8:48 AM
Opposite to Shyamoli Cinema Hal	6:39 AM	8:51 AM
Agargaon Radio Office	6:42 AM	8:53 AM
Opposite to Agargaon flower shops	6:44 AM	8:55 AM
BRAC University (arrival time)	7:35 AM	10:45 AM

Route-06: Mohammadpur-B to BRAC University

Stoppages	1st Trip (Pickup time)	2nd Trip (Pickup time)
In front of Bata Showroom (opposite to Japan Garden City's main gate)	6:35 AM	8:30 AM
In front of Baitus Sujud Jame Mosque (Near Mohammadpur Bus Stand/Nurjahan Road)	6:40 AM	8:36 AM
Town Hall Market (in front of Bikrampur Mistanna Bhandar)	6:42 AM	8:38 AM
West end of Manik Mia Avenue (opposite to Aarong)	6:45 AM	8:43 AM
New Model Multilateral High School (opposite to Kalabagan bus counters)	6:50 AM	8:48 AM
Water Bhaban (Near Panthapath Mour)	6:53 AM	8:53 AM
BRAC University (arrival time)	7:30 AM	10:30 AM

Route-07: Narayanganj to BRAC University

Stoppages	1st Trip (Pickup time)
Narayanganj Central Shahid Minar (Chashara)	6:30 AM
Himaloy Chinese & Community Centre (opposite to Narayanganj Zila Parishad)	6:34 AM
Shibu Market (in front of Dutch Bangla Bank)	6:37 AM
Jalkuri Bus Stand, Fatullah (in front of First Security Islami Bank)	6:40 AM
Bhuighar (near Darussunnah Islamia Kamil Madrasah)	6:43 AM
Signboard (near Islamia Eye Hospital)	6:48 AM
Shamsul Hoque School & College (under foot over-bridge)	6:51 AM
Sanarpar Bus Stand, Siddhirganj	6:54 AM
Madaninagar Fazil Madrasah, Siddhirganj (near foot over-bridge)	6:57 AM
Khankuye Jame Masjid, Narayanganj (near foot over-bridge)	7:02 AM
Opposite of Rani Mahal Cinema Hall, Demra	7:04 AM
IFIC Bank Ltd (opposite to Saruia Bazar)	7:05 AM
In front of Demra Ideal College	7:11 AM
BRAC University (arrival time)	7:35 AM

Route-08: Bashundhara Residential Area to BRAC University

Stoppages	1st Trip (Pickup time)	2nd Trip (Pickup time)
Ebenezer International School, Bashundhara R/A	7:00 AM	9:00 AM
Block-F, Road No: 08, Bashundhara R/A (opposite to IFIC Bank)	7:05 AM	9:05 AM
In front of Aga Khan Academy, Bashundhara R/A	7:10 AM	9:10 AM
BRAC University (arrival time)	7:40 AM	10:35 AM

1st Outbound Timing (Drop off) from Basement 01

BRAC University to Abdullahpur-A: Route No-01	2:05 PM
BRAC University to Mirpur-A: Route No-02	2:05 PM
BRAC University to Mirpur-B: Route No-02	2:05 PM
BRAC University to Jigatola-A: Route No-03	2:05 PM
BRAC University to Azimpur: Route No-04	2:05 PM
BRAC University to Baldha Garden: Route No-05	2:10 PM
BRAC University to Mohammadpur-A: Route No-06	2:10 PM
BRAC University to Mohammadpur-B: Route No-06	2:05 PM
BRAC University to Bashundhara: Route No-08	2:05 PM

Fare (except for Narayanganj and Bashundhara routes)

BDT 110 (Taka One Hundred and Ten Only) for each passenger for each trip. For round trip (coming and going), BDT 220 (Taka Two Hundred and Twenty Only).

Fare for Narayanganj route

BDT 160 (Taka One Hundred and Sixty Only) for each passenger for each trip For a *round trip* (coming and going), it is *BDT 320 (Taka Three Hundred and Twenty Only)*.

Fare for Bashundhara route

BDT 50 (Taka Fifty Only) for each passenger for each trip. For a *round trip* (coming and going), it is *BDT 100 (Taka One Hundred Only)*.

2nd Outbound Timing (Drop off) from Basement 01

BRAC University to Abdullahpur-A: Route No-01	5:10 PM
BRAC University to Abdullahpur-B: Route No-01	5:15 PM
BRAC University to Mirpur-A: Route No-02	5:10 PM
BRAC University to Mirpur-B: Route No-02	5:10 PM
BRAC University to Jigatola-A: Route No-03	5:10 PM
BRAC University to Jigatola-B: Route No-03	5:15 PM
BRAC University to Azimpur: Route No-04	5:10 PM
BRAC University to Baldha Garden: Route No-05	5:15 PM
BRAC University to Mohammadpur-A: Route No-06	5:10 PM
BRAC University to Mohammadpur-B: Route No-06	5:15 PM
BRAC University to Narayanganj: Route No-07	5:15 PM
BRAC University to Bashundhara: Route No-08	5:10 PM

For any official query, please contact:

Md. Lutfor Rahman
(Assistant Director, Operations, Transport Office)
Email : rahman@bracu.ac.bd

Mijan Hosen
(Assistant Manager, Operations, Transport Office)
E-mail : mijan.hosen@bracu.ac.bd

read and analyse both images and create a full schedule plan which I can copy and paste and give the agent to integrate into the database schedule section

I read both schedule images carefully. The main database issue is that your current `schedules` table only stores a route-level departure/arrival time, while this timetable contains **individual stoppages with pickup times, A/B route variants, multiple inbound trips, outbound campus departures, and route-specific fares**.

Also, the images **do not provide outbound stop-by-stop drop-off times, service weekdays/dates, or specific bus registration assignments**. The prompt below explicitly tells the agent **not to invent those values**.

Copy-paste this entire prompt into your coding agent:

INTEGRATE COMPLETE BRAC UNIVERSITY BUS ROUTE & TRANSPORT TIMETABLE INTO UNIRIDE

Update the existing **UniRide — Multi-University Bus Ticket Management System** and integrate the following BRAC University transport timetable into the existing **MySQL database, PHP backend, schedule-management system, route pages, passenger dashboard, and booking workflow**.

Do **not** rebuild the project from scratch.

The existing technology stack remains:

- HTML5
- CSS3
- Vanilla JavaScript
- PHP
- MySQL
- PDO
- phpMyAdmin
- XAMPP

Existing database:

`uniride_db`

This timetable belongs specifically to:

`BRAC University / BRACU`

Do not overwrite or interfere with schedule data belonging to other universities.

---

1. SOURCE-FIDELITY RULE

The schedule data below is extracted from the provided BRAC University transportation timetable.

Use the supplied:

- Route names
- Route numbers
- Route variants
- Stoppages

- Pickup times
- Arrival times
- Outbound departure times
- Fare amounts

exactly as specified.

Do not silently "correct" unusual time gaps.

For example, if the source says:

```
Route 01-A
Second Trip:
Biswa Road = 8:49 AM
BRAC University arrival = 10:40 AM
```

preserve those values.

Do not invent data that was not provided.

In particular, the timetable does **not** provide:

- Weekday/service-day information
- Specific calendar dates
- Bus registration numbers for each trip
- Exact outbound drop-off time at each stop
- Outbound destination arrival times
- Student/Faculty bus classification for each timetable trip

Do not fabricate these values.

---

2. IMPORTANT DATABASE DESIGN CHANGE

The existing database cannot fully represent this timetable using only:

```
routes
schedules
```

because a route contains many stoppages and every trip has an individual pickup time at each stop.

Extend the existing schema without breaking current functionality.

Add proper normalized tables.

Recommended structure:

```
route_stops
timetable_templates
timetable_stop_times
```

Optionally add:

```
timetable_service_days
```

later when actual operating days are known.

3. ROUTE STOPS TABLE

Create:

`route_stops`

Recommended fields:

```
route_stop_id
route_id
stop_order
stop_name
stop_note
is_terminal
created_at
```

Example structure:

```
CREATE TABLE route_stops (
    route_stop_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    route_id INT UNSIGNED NOT NULL,
    stop_order INT UNSIGNED NOT NULL,
    stop_name VARCHAR(180) NOT NULL,
    stop_note VARCHAR(255) NULL,
    is_terminal TINYINT(1) NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    UNIQUE KEY uq_route_stop_order (
        route_id,
        stop_order
    ),

    FOREIGN KEY (route_id)
        REFERENCES routes(route_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Improve the structure if necessary.

4. TIMETABLE TEMPLATE TABLE

The supplied source gives recurring clock times rather than calendar dates.

Therefore do not insert arbitrary dates such as:

`2026-08-20`

for every timetable record just to satisfy the current `schedules` table.

Create a timetable template system.

Recommended:

`timetable_templates`

Fields:

```
timetable_id
route_id
direction
trip_number
trip_label
```

```
origin_departure_time
terminal_arrival_time
campus_departure_time
status
created_at
```

Direction values:

```
INBOUND
OUTBOUND
```

For inbound:

```
origin_departure_time = first pickup time
terminal_arrival_time = BRAC University arrival time
campus_departure_time = NULL
```

For outbound:

```
campus_departure_time = BRAC University departure time
terminal_arrival_time = NULL
```

Do not invent destination arrival time.

---

5. TIMETABLE STOP TIMES

Create:

```
timetable_stop_times
```

Recommended fields:

```
timetable_stop_time_id
timetable_id
route_stop_id
stop_sequence
scheduled_time
event_type
```

Event types:

```
PICKUP
ARRIVAL
DROPOFF
```

For inbound trips:

- Intermediate stops = `PICKUP`
- BRAC University = `ARRIVAL`

For outbound trips, the timetable only provides departure from BRAC University.

Do not create fictional stop-by-stop outbound drop-off times.

---

6. CONNECT TIMETABLE TO BOOKABLE SCHEDULES

Keep the existing:

```
schedules
```

table for actual date-specific/bookable trips.

Add an optional reference such as:

```
timetable_id
```

to **schedules**.

Workflow should be:

```
Recurring timetable template
        ↓
Admin selects operating date
        ↓
Admin assigns actual bus
        ↓
Date-specific schedule generated
        ↓
Passenger books that schedule
```

This prevents the static timetable and real booked schedules from becoming mixed together.

---

7. ROUTE VARIANTS

A/B routes have different stops.

Therefore each A/B branch must be stored as its own database route even if the timetable gives them the same route number.

Create these BRACU route codes:

```
BRAC-R01-A
BRAC-R01-B

BRAC-R02-A
BRAC-R02-B

BRAC-R03-A
BRAC-R03-B

BRAC-R04

BRAC-R05

BRAC-R06-A
BRAC-R06-B

BRAC-R07

BRAC-R08
```

Total inbound route variants:

```
12
```

---

8. ROUTE DEFINITIONS

Create/update the BRACU routes as follows.

Route 01-A

```
Route Code: BRAC-R01-A
Display Number: Route-01
Variant: A
Route Name: Abdullahpur-A to BRAC University
Start Location: Abdullahpur
End Location: BRAC University
One-Way Fare: BDT 110
Round-Trip Fare: BDT 220
```

Route 01-B

```
Route Code: BRAC-R01-B
Display Number: Route-01
Variant: B
Route Name: Abdullahpur-B to BRAC University
Start Location: Abdullahpur
End Location: BRAC University
One-Way Fare: BDT 110
Round-Trip Fare: BDT 220
```

Route 02-A

```
Route Code: BRAC-R02-A
Display Number: Route-02
Variant: A
Route Name: Mirpur-A to BRAC University
Start Location: Mirpur
End Location: BRAC University
One-Way Fare: BDT 110
Round-Trip Fare: BDT 220
```

Route 02-B

```
Route Code: BRAC-R02-B
Display Number: Route-02
Variant: B
Route Name: Mirpur-B to BRAC University
Start Location: Mirpur
End Location: BRAC University
One-Way Fare: BDT 110
Round-Trip Fare: BDT 220
```

Route 03-A

```
Route Code: BRAC-R03-A
Display Number: Route-03
Variant: A
Route Name: Jigatola-A to BRAC University
Start Location: Jigatola
End Location: BRAC University
One-Way Fare: BDT 110
Round-Trip Fare: BDT 220
```

Route 03-B

```
Route Code: BRAC-R03-B
Display Number: Route-03
Variant: B
Route Name: Jigatola-B to BRAC University
Start Location: Jigatola
```

End Location: BRAC University
One-Way Fare: BDT 110
Round-Trip Fare: BDT 220

Route 04

Route Code: BRAC-R04
Display Number: Route-04
Route Name: Azimpur to BRAC University
Start Location: Azimpur
End Location: BRAC University
One-Way Fare: BDT 110
Round-Trip Fare: BDT 220

Route 05

Route Code: BRAC-R05
Display Number: Route-05
Route Name: Baldha Garden to BRAC University
Start Location: Baldha Garden
End Location: BRAC University
One-Way Fare: BDT 110
Round-Trip Fare: BDT 220

Route 06-A

Route Code: BRAC-R06-A
Display Number: Route-06
Variant: A
Route Name: Mohammadpur-A to BRAC University
Start Location: Mohammadpur
End Location: BRAC University
One-Way Fare: BDT 110
Round-Trip Fare: BDT 220

Route 06-B

Route Code: BRAC-R06-B
Display Number: Route-06
Variant: B
Route Name: Mohammadpur-B to BRAC University
Start Location: Mohammadpur
End Location: BRAC University
One-Way Fare: BDT 110
Round-Trip Fare: BDT 220

Route 07

Route Code: BRAC-R07
Display Number: Route-07
Route Name: Narayanganj to BRAC University
Start Location: Narayanganj
End Location: BRAC University
One-Way Fare: BDT 160
Round-Trip Fare: BDT 320

Route 08

```
Route Code: BRAC-R08
Display Number: Route-08
Route Name: Bashundhara Residential Area to BRAC University
Start Location: Bashundhara Residential Area
End Location: BRAC University
One-Way Fare: BDT 50
Round-Trip Fare: BDT 100
```

---

9. FARE RULES

Apply these fares.

Routes 01–06

```
One-way trip: BDT 110
Round trip: BDT 220
```

This applies to:

```
01-A
01-B
02-A
02-B
03-A
03-B
04
05
06-A
06-B
```

Route 07 — Narayanganj

```
One-way trip: BDT 160
Round trip: BDT 320
```

Route 08 — Bashundhara

```
One-way trip: BDT 50
Round trip: BDT 100
```

The existing booking system normally charges per booked trip.

Therefore:

```
bookings.fare = route one-way fare
```

unless a dedicated round-trip purchase feature is explicitly used.

---

10. COMPLETE INBOUND TIMETABLE

Implement the following data exactly.

ROUTE 01-A — ABDULLAHPUR-A → BRAC UNIVERSITY

Trip 1

```
01 | Abdullahpur (Polwel Carnation Shopping Centre) | 6:40 AM
02 | House Building (Labaid Diagnostic) | 6:43 AM
03 | Azampur (In front of Uttara East Police Station) | 6:46 AM
04 | Jasimuddin (Popular Diagnostic Centre) | 6:49 AM
05 | Airport (In front of passenger shed situated 50 yards back) | 6:51 AM
06 | Kawla (near foot over-bridge) | 6:52 AM
07 | Khilket (first foot over-bridge) | 6:56 AM
08 | Biswa Road (near foot over-bridge) | 6:59 AM
09 | BRAC University | ARRIVAL 7:30 AM
```

Template summary:

```
First pickup: 6:40 AM
Campus arrival: 7:30 AM
```

Trip 2

```
01 | Abdullahpur (Polwel Carnation Shopping Centre) | 8:30 AM
02 | House Building (Labaid Diagnostic) | 8:33 AM
03 | Azampur (In front of Uttara East Police Station) | 8:36 AM
04 | Jasimuddin (Popular Diagnostic Centre) | 8:39 AM
05 | Airport (In front of passenger shed situated 50 yards back) | 8:43 AM
06 | Kawla (near foot over-bridge) | 8:44 AM
07 | Khilket (first foot over-bridge) | 8:47 AM
08 | Biswa Road (near foot over-bridge) | 8:49 AM
09 | BRAC University | ARRIVAL 10:40 AM
```

Template summary:

```
First pickup: 8:30 AM
Campus arrival: 10:40 AM
```

Preserve the 10:40 AM source value.

ROUTE 01-B — ABDULLAHPUR-B → BRAC UNIVERSITY

Only one inbound trip is provided.

Trip 1

```
01 | Jashimuddin (Opposite to RAK Shopping Complex, Uttara Sector #01, Road #14) | 6:20 AM

02 | Korotoa Courier Point (Opposite to Korotoa Courier Service, Uttara Sector #03, Road #18) |
6:22 AM

03 | Shahid Mughdho Corner (In front of Shahid Mughdho Corner / Robindro Sorobor) | 6:23 AM

04 | Lake Drive Road – Uttara Model Town, Opposite to Brac Bank, Sector #07 | 6:25 AM

05 | Zam Zam Tower (In front of Zam Zam Tower, Sector #13, Uttara) | 6:27 AM

06 | Tagore University Point (In front of Tagore University of Creative Arts, Sector #12) | 6:29
AM

07 | Khalpar Mor – In front of Police Box | 6:30 AM

08 | Diabari Gol Chattar (In front of Fresh Ceramics, Uttara) | 6:32 AM

09 | Uttara Centre (MRT Station) – Under Metro Rail Station | 6:34 AM
```

```
10 | Mirpur Ceramics (near CNG filling station) | 6:38 AM

11 | Mirpur DOHS Gate (near Trust bus stop) | 6:43 AM

12 | Pallabi Thana (Shagufta mour) | 6:45 AM

13 | Kalshi Bus Stand | 6:48 AM

14 | ECB Chattar | 6:51 AM

15 | BRAC University | ARRIVAL 7:40 AM
```

Summary:

```
First pickup: 6:20 AM
Campus arrival: 7:40 AM
```

Do not create a second inbound trip for Route 01-B unless an administrator adds one later.

ROUTE 02-A — MIRPUR-A → BRAC UNIVERSITY

Trip 1

```
01 | Mirpur Bangla College | 6:20 AM
02 | Dhaka Laboratory School and College (In front of Staff Quarter) | 6:22 AM
03 | Sony Cinema Hall (near KFC) | 6:24 AM
04 | Mirpur College Gate (Mirpur 2) | 6:26 AM
05 | Swimming Federation Gate (Mirpur stadium) | 6:28 AM
06 | Popular Diagnostic Centre (Mirpur 6) | 6:31 AM
07 | Pallabi Post Office (near Purobi Cinema Hall) | 6:34 AM
08 | Mirpur 11.5 (Banalata Sweets) | 6:35 AM
09 | Mirpur Ceramics (near CNG Filling Station) | 6:36 AM
10 | Mirpur DOHS Gate (near Trust Bus Stop) | 6:40 AM
11 | Pallabi Thana (Shagufta mour) | 6:43 AM
12 | Kalshi Bus Stand | 6:45 AM
13 | ECB Chattar | 6:48 AM
14 | BRAC University | ARRIVAL 7:30 AM
```

Trip 2

```
01 | Mirpur Bangla College | 8:20 AM
02 | Dhaka Laboratory School and College (In front of Staff Quarter) | 8:22 AM
03 | Sony Cinema Hall (near KFC) | 8:25 AM
04 | Mirpur College Gate (Mirpur 2) | 8:27 AM
05 | Swimming Federation Gate (Mirpur stadium) | 8:29 AM
06 | Popular Diagnostic Centre (Mirpur 6) | 8:33 AM
07 | Pallabi Post Office (near Purobi Cinema Hall) | 8:37 AM
08 | Mirpur 11.5 (Banalata Sweets) | 8:38 AM
09 | Mirpur Ceramics (near CNG Filling Station) | 8:40 AM
10 | Mirpur DOHS Gate (near Trust Bus Stop) | 8:45 AM
11 | Pallabi Thana (Shagufta mour) | 8:47 AM
12 | Kalshi Bus Stand | 8:49 AM
13 | ECB Chattar | 8:52 AM
14 | BRAC University | ARRIVAL 10:20 AM
```

ROUTE 02-B — MIRPUR-B → BRAC UNIVERSITY

Trip 1

```
01 | Mirpur 14 Bus Stand (opposite to Marks Medical College Hospital) | 6:35 AM
02 | In front of NAM Garden Officers Quarter, Mirpur 13 (near BRTA 2nd Gate) | 6:37 AM
03 | Opposite to Water Tank (Mirpur 10) | 6:39 AM
04 | Al Helal Specialized Hospital (Mirpur 10) | 6:43 AM
05 | In front of BRAC Healthcare, Kazipara | 6:46 AM
06 | Shewrapara (near Jamuna Plaza and Metro Rail Station) | 6:48 AM
07 | Taltola Bus Stand (opposite to Glory School & College) | 6:51 AM
08 | BRAC University | ARRIVAL 7:30 AM
```

Trip 2

```
01 | Mirpur 14 Bus Stand (opposite to Marks Medical College Hospital) | 8:35 AM
02 | In front of NAM Garden Officers Quarter, Mirpur 13 (near BRTA 2nd Gate) | 8:37 AM
03 | Opposite to Water Tank (Mirpur 10) | 8:40 AM
04 | Al Helal Specialized Hospital (Mirpur 10) | 8:44 AM
05 | In front of BRAC Healthcare, Kazipara | 8:46 AM
06 | Shewrapara (near Jamuna Plaza and Metro Rail Station) | 8:48 AM
07 | Taltola Bus Stand (opposite to Glory School & College) | 8:50 AM
08 | BRAC University | ARRIVAL 10:30 AM
```

ROUTE 03-A — JIGATOLA-A → BRAC UNIVERSITY

Trip 1

```
01 | Jigatola Bus Stand (in front of Japan-Bangladesh Friendship Hospital) | 6:35 AM
02 | Keari Plaza, Dhanmondi 15 Bus Stand | 6:37 AM
03 | Shankar Bus Stand | 6:40 AM
04 | Meena Bazar (Dhanmondi 27) | 6:42 AM
05 | In front of Lalmatia Mother Care Hospital | 6:45 AM
06 | BRAC University | ARRIVAL 7:30 AM
```

Trip 2

```
01 | Jigatola Bus Stand (in front of Japan-Bangladesh Friendship Hospital) | 8:40 AM
02 | Keari Plaza, Dhanmondi 15 Bus Stand | 8:42 AM
03 | Shankar Bus Stand | 8:45 AM
04 | Meena Bazar (Dhanmondi 27) | 8:47 AM
05 | In front of Lalmatia Mother Care Hospital | 8:50 AM
06 | BRAC University | ARRIVAL 10:30 AM
```

ROUTE 03-B — JIGATOLA-B → BRAC UNIVERSITY

Only one inbound trip is provided.

```
01 | Jigatola Bus Stand (in front of Japan-Bangladesh Friendship Hospital) | 7:40 AM
02 | Keari Plaza, Dhanmondi 15 Bus Stand | 7:42 AM
03 | Shankar Bus Stand | 7:45 AM
04 | Meena Bazar (Dhanmondi 27) | 7:47 AM
05 | In front of Lalmatia Mother Care Hospital | 7:50 AM
06 | BRAC University | ARRIVAL 8:45 AM
```

Do not create a second inbound trip from the source.

ROUTE 04 — AZIMPUR → BRAC UNIVERSITY

Trip 1

01 | Azimpur Etimkhana Mour | 6:25 AM
02 | Azimpur Chowrasta | 6:27 AM
03 | New Market (gate no 2, Janata Bank) | 6:29 AM
04 | Happy Arcade Shopping Mall (beside City College) | 6:31 AM
05 | Dhanmondi, Road No 7 (ARA Center Shopping Mall) | 6:33 AM
06 | Kalabagan Krira Chokro (near foot over-bridge) | 6:35 AM
07 | Sobhanbag (near foot over-bridge) | 6:37 AM
08 | West End of Manik Mia Avenue (opposite Aarong) | 6:39 AM
09 | Khejur Bagan Mour (Manik Mia Avenue) | 6:41 AM
10 | BRAC University | ARRIVAL 7:30 AM

Trip 2

01 | Azimpur Etimkhana Mour | 8:30 AM
02 | Azimpur Chowrasta | 8:33 AM
03 | New Market (gate no 2, Janata Bank) | 8:36 AM
04 | Happy Arcade Shopping Mall (beside City College) | 8:39 AM
05 | Dhanmondi, Road No 7 (ARA Center Shopping Mall) | 8:42 AM
06 | Kalabagan Krira Chokro (near foot over-bridge) | 8:44 AM
07 | Sobhanbag (near foot over-bridge) | 8:47 AM
08 | West End of Manik Mia Avenue (opposite Aarong) | 8:51 AM
09 | Khejur Bagan Mour (Manik Mia Avenue) | 8:53 AM
10 | BRAC University | ARRIVAL 10:45 AM

ROUTE 05 — BALDHA GARDEN → BRAC UNIVERSITY

Trip 1

01 | Doyaganj Mour (near mandir) | 6:30 AM
02 | Baldha Garden | 6:35 AM
03 | Ittefaq Mour | 6:38 AM
04 | Motijheel (foot over-bridge) | 6:40 AM
05 | Arambag (near Janata Bank) | 6:41 AM
06 | Fakirapool Mour (Apex showroom) | 6:43 AM
07 | Purana Paltan (Hatil showroom) | 6:44 AM
08 | Kakrail Mour (near Eastern Bank) | 6:46 AM
09 | Shantinagar Mour (One Bank) | 6:48 AM
10 | Moghbazar T&T Office | 6:51 AM
11 | Moghbazar Mour (NCC Bank) | 6:55 AM
12 | BRAC University | ARRIVAL 7:40 AM

Trip 2

01 | Doyaganj Mour (near mandir) | 8:30 AM
02 | Baldha Garden | 8:35 AM
03 | Ittefaq Mour | 8:38 AM
04 | Motijheel (foot over-bridge) | 8:40 AM
05 | Arambag (near Janata Bank) | 8:42 AM
06 | Fakirapool Mour (Apex showroom) | 8:44 AM
07 | Purana Paltan (Hatil showroom) | 8:45 AM
08 | Kakrail Mour (near Eastern Bank) | 8:48 AM
09 | Shantinagar Mour (One Bank) | 8:50 AM
10 | Moghbazar T&T Office | 8:54 AM
11 | Moghbazar Mour (NCC Bank) | 9:00 AM
12 | BRAC University | ARRIVAL 10:30 AM

ROUTE 06-A — MOHAMMADPUR-A → BRAC UNIVERSITY

Trip 1

01 | Shiya Masjid (near Top Ten shop) | 6:30 AM
02 | Opposite to Suchona Community Center | 6:32 AM
03 | Shampa Market | 6:34 AM
04 | Shyamoli (Bata mour) | 6:37 AM
05 | Opposite to Shyamoli Cinema Hall | 6:39 AM
06 | Agargaon Radio Office | 6:42 AM
07 | Opposite to Agargaon flower shops | 6:44 AM
08 | BRAC University | ARRIVAL 7:35 AM

Trip 2

01 | Shiya Masjid (near Top Ten shop) | 8:40 AM
02 | Opposite to Suchona Community Center | 8:42 AM
03 | Shampa Market | 8:45 AM
04 | Shyamoli (Bata mour) | 8:48 AM
05 | Opposite to Shyamoli Cinema Hall | 8:51 AM
06 | Agargaon Radio Office | 8:53 AM
07 | Opposite to Agargaon flower shops | 8:55 AM
08 | BRAC University | ARRIVAL 10:45 AM

ROUTE 06-B — MOHAMMADPUR-B → BRAC UNIVERSITY

Trip 1

01 | In front of Bata Showroom (opposite to Japan Garden City's main gate) | 6:35 AM

02 | In front of Baitus Sujud Jame Mosque (Near Mohammadpur Bus Stand/Nurjahan Road) | 6:40 AM

03 | Town Hall Market (in front of Bikrampur Mistanna Bhandar) | 6:42 AM

04 | West end of Manik Mia Avenue (opposite to Aarong) | 6:45 AM

05 | New Model Multilateral High School (opposite to Kalabagan bus counters) | 6:50 AM

06 | Water Bhaban (Near Panthapath Mour) | 6:53 AM

07 | BRAC University | ARRIVAL 7:30 AM

Trip 2

01 | In front of Bata Showroom (opposite to Japan Garden City's main gate) | 8:30 AM

02 | In front of Baitus Sujud Jame Mosque (Near Mohammadpur Bus Stand/Nurjahan Road) | 8:36 AM

03 | Town Hall Market (in front of Bikrampur Mistanna Bhandar) | 8:38 AM

04 | West end of Manik Mia Avenue (opposite to Aarong) | 8:43 AM

05 | New Model Multilateral High School (opposite to Kalabagan bus counters) | 8:48 AM

06 | Water Bhaban (Near Panthapath Mour) | 8:53 AM

07 | BRAC University | ARRIVAL 10:30 AM

ROUTE 07 — NARAYANGANJ → BRAC UNIVERSITY

Only one inbound trip is provided.

01 | Narayanganj Central Shahid Minar (Chashara) | 6:30 AM

02 | Himaloy Chinese & Community Centre (opposite to Narayanganj Zila Parishad) | 6:34 AM

03 | Shibu Market (in front of Dutch Bangla Bank) | 6:37 AM

04 | Jalkuri Bus Stand, Fatullah (in front of First Security Islami Bank) | 6:40 AM

05 | Bhuighar (near Darussunnah Islamia Kamil Madrasah) | 6:43 AM

06 | Signboard (near Islamia Eye Hospital) | 6:48 AM

07 | Shamsul Hoque School & College (under foot over-bridge) | 6:51 AM

08 | Sanarpar Bus Stand, Siddhirganj | 6:54 AM

09 | Madaninagar Fazil Madrasah, Siddhirganj (near foot over-bridge) | 6:57 AM

10 | Khankaye Jame Masjid, Narayanganj (near foot over-bridge) | 7:02 AM

11 | Opposite of Rani Mahal Cinema Hall, Demra | 7:04 AM

12 | IFIC Bank Ltd (opposite to Sarulia Bazar) | 7:05 AM

13 | In front of Demra Ideal College | 7:11 AM

14 | BRAC University | ARRIVAL 7:35 AM

Fare:

BDT 160 one-way
BDT 320 round-trip

ROUTE 08 — BASHUNDHARA RESIDENTIAL AREA → BRAC UNIVERSITY

Trip 1

01 | Ebenezer International School, Bashundhara R/A | 7:00 AM

02 | Block-F, Road No: 08, Bashundhara R/A (opposite to IFIC Bank) | 7:05 AM

03 | In front of Aga Khan Academy, Bashundhara R/A | 7:10 AM

04 | BRAC University | ARRIVAL 7:40 AM

Trip 2

01 | Ebenezer International School, Bashundhara R/A | 9:00 AM

02 | Block-F, Road No: 08, Bashundhara R/A (opposite to IFIC Bank) | 9:05 AM

03 | In front of Aga Khan Academy, Bashundhara R/A | 9:10 AM

04 | BRAC University | ARRIVAL 10:35 AM

Fare:

BDT 50 one-way
BDT 100 round-trip

---

11. FIRST OUTBOUND / DROP-OFF DEPARTURES

The source labels this:

```
1st Outbound Timing (Drop off) from Basement 01
```

Store the following campus departure times.

```
BRAC University → Abdullahpur-A
Route: 01-A
Departure: 2:05 PM

BRAC University → Mirpur-A
Route: 02-A
Departure: 2:05 PM

BRAC University → Mirpur-B
Route: 02-B
Departure: 2:05 PM

BRAC University → Jigatola-A
Route: 03-A
Departure: 2:05 PM

BRAC University → Azimpur
Route: 04
Departure: 2:05 PM

BRAC University → Baldha Garden
Route: 05
Departure: 2:10 PM

BRAC University → Mohammadpur-A
Route: 06-A
Departure: 2:10 PM

BRAC University → Mohammadpur-B
Route: 06-B
Departure: 2:05 PM

BRAC University → Bashundhara
Route: 08
Departure: 2:05 PM
```

IMPORTANT:

The source does **not** list a first outbound departure for:

```
Route 01-B
Route 03-B
Route 07 Narayanganj
```

Do not invent first-outbound trips for those routes.

---

12. SECOND OUTBOUND / DROP-OFF DEPARTURES

The source labels this:

```
2nd Outbound Timing (Drop off) from Basement 01
```

Store:

```
BRAC University → Abdullahpur-A
Route: 01-A
Departure: 5:10 PM
```

```
BRAC University → Abdullahpur-B
Route: 01-B
Departure: 5:15 PM

BRAC University → Mirpur-A
Route: 02-A
Departure: 5:10 PM

BRAC University → Mirpur-B
Route: 02-B
Departure: 5:10 PM

BRAC University → Jigatola-A
Route: 03-A
Departure: 5:10 PM

BRAC University → Jigatola-B
Route: 03-B
Departure: 5:15 PM

BRAC University → Azimpur
Route: 04
Departure: 5:10 PM

BRAC University → Baldha Garden
Route: 05
Departure: 5:15 PM

BRAC University → Mohammadpur-A
Route: 06-A
Departure: 5:10 PM

BRAC University → Mohammadpur-B
Route: 06-B
Departure: 5:15 PM

BRAC University → Narayanganj
Route: 07
Departure: 5:15 PM

BRAC University → Bashundhara
Route: 08
Departure: 5:10 PM
```

---

13. OUTBOUND STOP DATA

The source provides only the BRAC University departure time for outbound trips.

It does **not** provide individual drop-off timestamps.

Therefore:

Do not generate fake values such as:

```
Mirpur Bangla College = 5:45 PM
Mirpur 10 = 5:50 PM
```

unless an administrator later enters real data.

The database may reverse the stop order for display purposes, but the time field for individual outbound stops must remain:

```
NULL
```

until official data is provided.

---

14. DATABASE ROUTE DIRECTION

Do not duplicate all route records simply because buses operate both ways.

Prefer:

```
route = BRAC-R02-A

direction = INBOUND
```

and:

```
route = BRAC-R02-A

direction = OUTBOUND
```

Timetable templates determine which direction is being operated.

For display:

Inbound:

```
Mirpur-A → BRAC University
```

Outbound:

```
BRAC University → Mirpur-A
```

---

15. ROUTE LIST UI

Update the passenger Routes/Schedules section.

Each card should show:

```
Route 02-A
Mirpur-A → BRAC University

Trip 1: 6:20 AM
Arrives: 7:30 AM

Trip 2: 8:20 AM
Arrives: 10:20 AM

Fare: BDT 110
```

Provide buttons:

```
View Stops
View Timetable
Book Trip
Favorite
```

---

16. VIEW STOPS UI

When the passenger clicks:

```
View Stops
```

display a vertical route timeline.

Example:

```
Route 03-A — Trip 1

● 6:35 AM
  Jigatola Bus Stand

  |

● 6:37 AM
  Keari Plaza, Dhanmondi 15 Bus Stand

  |

● 6:40 AM
  Shankar Bus Stand

  |

● 6:42 AM
  Meena Bazar

  |

● 6:45 AM
  Lalmatia Mother Care Hospital

  |

● 7:30 AM
  BRAC University
```

This information must come from:

```
route_stops
+
timetable_stop_times
```

not hard-coded HTML.

---

17. INBOUND / OUTBOUND TOGGLE

The schedule page should offer:

```
[ To University ] [ From University ]
```

or:

```
Inbound
Outbound
```

Inbound displays pickup times and BRACU arrival.

Outbound displays campus departure times.

---

18. TRIP BADGES

Use clear labels:

```
1st Trip
2nd Trip
```

For routes that contain only one source trip, show only:

```
1st Trip
```

Do not display an empty second-trip tab.

---

19. BRAC UNIVERSITY ARRIVAL

The final inbound stop should visually differ from pickup stops.

Display:

```
BRAC University
Arrival Time: 7:30 AM
```

Do not describe campus arrival as another pickup point.

---

20. OUTBOUND DEPARTURE LOCATION

The source identifies outbound departure as:

```
Basement 01
```

Display:

```
Departure Point:
BRAC University — Basement 01
```

for outbound services.

---

21. SCHEDULE MANAGEMENT — UNIVERSITY ADMIN

Update the University Admin schedule interface so BRACU transport management can:

- View timetable templates
- View stops
- Add stop
- Edit stop
- Reorder stops
- Edit pickup time
- Add/remove trip template
- Edit inbound arrival time
- Edit outbound departure time
- Activate/deactivate timetable
- Assign an actual bus
- Generate a date-specific bookable schedule
- Cancel generated schedules

Do not require administrators to rewrite the entire route to change one stop time.

---

22. ACTUAL BOOKING SCHEDULE

The timetable itself is not automatically a seat-bookable journey until:

```
Date + Bus
```

have been assigned.

For example:

```
Timetable:
BRAC-R03-A
Inbound Trip 1
6:35 AM → 7:30 AM

        ↓

University Admin generates:

Date: 2026-09-01
Bus: BRACU-BUS-07

        ↓

Actual schedule created

        ↓

40 seats + 10 standing slots become bookable
```

This keeps timetable design normalized.

---

23. DO NOT INVENT SERVICE DAYS

The provided timetable images do not state:

```
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
```

Therefore do not assume that these trips operate Monday–Friday or Sunday–Thursday.

Store them as timetable templates.

If the application requires service-day settings, set them through an admin configuration screen rather than hard-coding assumptions.

---

24. DO NOT INVENT BUS ASSIGNMENTS

The images show route/timetable information but do not identify the registration number of the bus running each trip.

Therefore do not arbitrarily write:

```
Route 01-A = Bus 1
Route 02-A = Bus 2
```

as permanent official assignments.

Actual buses should be assigned by university management when generating bookable schedules.

---

25. DATABASE SEED SCRIPT

Update:

`database/uniride_complete_phpmyadmin.sql`

so a fresh phpMyAdmin import automatically creates:

- BRACU route variants
- Route stoppages
- Inbound timetable templates
- Stop pickup times
- Campus arrival times
- Outbound departure templates
- Route fares

Do not require the user to manually insert all these records after import.

Use transactions while inserting the timetable seed data.

---

26. DO NOT HARDCODE ROUTE IDs

Do not assume:

```
BRACU = university_id 1
Route 01 = route_id 1
```

inside PHP.

Find records using stable codes such as:

```
BRACU
BRAC-R01-A
BRAC-R02-A
```

and use actual generated IDs.

---

27. ROUTE FARES AND BOOKING

When passenger books:

```
Route 01–06
→ BDT 110

Route 07
→ BDT 160
```

```
Route 08
→ BDT 50
```

The authoritative fare must come from MySQL.

Do not calculate the authoritative value from JavaScript.

The semester-billing system must continue using the route fare stored in MySQL.

---

28. FAVORITE ROUTES

Favorite functionality must work with every route variant.

For example:

```
BRAC-R01-A
```

and:

```
BRAC-R01-B
```

are separate favorites because they follow different stop patterns.

Favorite routes still appear first.

---

29. SCHEDULE SEARCH

Allow passengers to search by:

```
Route number
Route name
Area
Stoppage
Pickup time
Direction
```

Examples:

```
Mirpur
Jigatola
Airport
Bashundhara
Narayanganj
```

Searching:

```
Airport
```

should identify Route 01-A because Airport is one of its stoppages.

This requires joining:

```
routes
route_stops
```

---

30. PASSENGER PICKUP STOP

Enhance booking so passengers may optionally choose their boarding stop.

Example:

```
Route 02-A
Trip 1

Boarding stop:
Mirpur College Gate

Pickup:
6:26 AM
```

If implemented, save the selected:

```
route_stop_id
```

with the booking.

Do not allow passengers to choose:

```
BRAC University
```

as an inbound pickup stop because it is the arrival terminal.

---

31. OUTBOUND DESTINATION STOP

For outbound trips, passenger may choose a destination stop from the route.

However, because the source provides no outbound drop-off time, display:

```
Drop-off time: Not provided
```

rather than estimating a time.

---

32. ROUTE TIMETABLE SUMMARY

The system should be able to show a summary similar to:

```
ROUTE 06-A — MOHAMMADPUR-A

TO BRAC UNIVERSITY

Trip 1
First pickup: 6:30 AM
Campus arrival: 7:35 AM

Trip 2
First pickup: 8:40 AM
Campus arrival: 10:45 AM

FROM BRAC UNIVERSITY

1st Outbound:
2:10 PM

2nd Outbound:
5:10 PM

Fare:
BDT 110 each trip
```

All values must come from MySQL.

---

33. ROUTES WITHOUT FIRST OUTBOUND

For:

```
Route 01-B
Route 03-B
Route 07
```

the source contains no first outbound entry.

Display:

```
1st Outbound: Not listed
```

or omit that trip.

Do not invent times.

Their available second outbound departures are:

```
01-B → 5:15 PM
03-B → 5:15 PM
07 → 5:15 PM
```

---

34. COMPLETE TIMETABLE SUMMARY

Use the following route summary for verification.

```
01-A Abdullahpur-A
Inbound Trip 1: 6:40 AM → 7:30 AM
Inbound Trip 2: 8:30 AM → 10:40 AM
Outbound 1: 2:05 PM
Outbound 2: 5:10 PM
Fare: 110

01-B Abdullahpur-B
Inbound Trip 1: 6:20 AM → 7:40 AM
Inbound Trip 2: NOT PROVIDED
Outbound 1: NOT PROVIDED
Outbound 2: 5:15 PM
Fare: 110

02-A Mirpur-A
Inbound Trip 1: 6:20 AM → 7:30 AM
Inbound Trip 2: 8:20 AM → 10:20 AM
Outbound 1: 2:05 PM
Outbound 2: 5:10 PM
Fare: 110

02-B Mirpur-B
Inbound Trip 1: 6:35 AM → 7:30 AM
Inbound Trip 2: 8:35 AM → 10:30 AM
Outbound 1: 2:05 PM
Outbound 2: 5:10 PM
Fare: 110

03-A Jigatola-A
Inbound Trip 1: 6:35 AM → 7:30 AM
Inbound Trip 2: 8:40 AM → 10:30 AM
Outbound 1: 2:05 PM
Outbound 2: 5:10 PM
Fare: 110

03-B Jigatola-B
Inbound Trip 1: 7:40 AM → 8:45 AM
Inbound Trip 2: NOT PROVIDED
Outbound 1: NOT PROVIDED
```

```
Outbound 2: 5:15 PM
Fare: 110

04 Azimpur
Inbound Trip 1: 6:25 AM → 7:30 AM
Inbound Trip 2: 8:30 AM → 10:45 AM
Outbound 1: 2:05 PM
Outbound 2: 5:10 PM
Fare: 110

05 Baldha Garden
Inbound Trip 1: 6:30 AM → 7:40 AM
Inbound Trip 2: 8:30 AM → 10:30 AM
Outbound 1: 2:10 PM
Outbound 2: 5:15 PM
Fare: 110

06-A Mohammadpur-A
Inbound Trip 1: 6:30 AM → 7:35 AM
Inbound Trip 2: 8:40 AM → 10:45 AM
Outbound 1: 2:10 PM
Outbound 2: 5:10 PM
Fare: 110

06-B Mohammadpur-B
Inbound Trip 1: 6:35 AM → 7:30 AM
Inbound Trip 2: 8:30 AM → 10:30 AM
Outbound 1: 2:05 PM
Outbound 2: 5:15 PM
Fare: 110

07 Narayanganj
Inbound Trip 1: 6:30 AM → 7:35 AM
Inbound Trip 2: NOT PROVIDED
Outbound 1: NOT PROVIDED
Outbound 2: 5:15 PM
Fare: 160

08 Bashundhara Residential Area
Inbound Trip 1: 7:00 AM → 7:40 AM
Inbound Trip 2: 9:00 AM → 10:35 AM
Outbound 1: 2:05 PM
Outbound 2: 5:10 PM
Fare: 50
```

---

35. DATABASE/API INTEGRATION

Create secure PHP endpoints such as:

```
api/routes.php
api/route-stops.php
api/timetable.php
api/schedules.php
api/generate-schedule.php
```

All queries must:

- Use PDO
- Use prepared statements
- Validate university ownership
- Respect PHP sessions
- Return JSON where appropriate

---

36. EXAMPLE TIMETABLE QUERY

The application should be able to retrieve a route timetable using an equivalent normalized query:

```
SELECT
    r.route_code,
    r.route_name,

    tt.direction,
    tt.trip_number,
    tt.origin_departure_time,
    tt.terminal_arrival_time,
    tt.campus_departure_time,

    rs.stop_order,
    rs.stop_name,
    rs.stop_note,

    tst.scheduled_time,
    tst.event_type

FROM timetable_templates tt

JOIN routes r
    ON r.route_id = tt.route_id

LEFT JOIN timetable_stop_times tst
    ON tst.timetable_id = tt.timetable_id

LEFT JOIN route_stops rs
    ON rs.route_stop_id = tst.route_stop_id

WHERE r.route_code = ?

ORDER BY
    tt.direction,
    tt.trip_number,
    rs.stop_order;
```

Improve the query as needed.

---

37. EXISTING SCHEDULE AVAILABILITY

Do not break:

```
v_schedule_availability
```

or the existing booking workflow.

The timetable template system should complement the existing date-specific schedule system.

Generated bookable schedules must continue to support:

- Seat availability
- 40 regular seats
- 10 standing slots
- Occupancy percentage
- Booking
- QR generation
- Cancellation

- Billing
- Ticket transfer
- Notifications

---

38. ROUTE ADMIN CRUD

The university administrator should be able to modify the timetable later.

Do not hard-code the supplied times into PHP source files.

All data must be stored in MySQL.

Admin changes must persist after refresh.

---

39. UI RESPONSIVENESS

The route schedule tables contain many stops.

On desktop:

Use a full timetable/table or route timeline.

On mobile:

Use:

- Collapsible route card
- Scrollable table
- Vertical stop timeline

Do not create a table that becomes unreadable on mobile.

---

40. SOURCE DATA VERIFICATION

After importing the final SQL, verify each route.

Run database checks to confirm:

```
12 BRACU route variants exist.
```

Verify timetable presence:

```
01-A → 2 inbound + 2 outbound
01-B → 1 inbound + 1 outbound
02-A → 2 inbound + 2 outbound
02-B → 2 inbound + 2 outbound
03-A → 2 inbound + 2 outbound
03-B → 1 inbound + 1 outbound
04   → 2 inbound + 2 outbound
05   → 2 inbound + 2 outbound
06-A → 2 inbound + 2 outbound
06-B → 2 inbound + 2 outbound
07   → 1 inbound + 1 outbound
08   → 2 inbound + 2 outbound
```

Here, an outbound count of `1` for 01-B, 03-B, and 07 refers only to the **second outbound service**, because no first outbound service was supplied.

---

41. IMPORTANT FINAL INTEGRATION RULE

Do not convert this schedule into static HTML.

The architecture must be:

```
BRAC timetable data
        ↓
MySQL
        ↓
PHP / PDO
        ↓
API / server-rendered PHP
        ↓
HTML + CSS + JavaScript
```

Every stop, time, fare, route, and trip shown in the UI must come from MySQL.

---

42. DO NOT BREAK EXISTING FEATURES

Preserve all existing UniRide functionality, including:

- Multi-university isolation
- Admin login
- University Admin login
- Student login
- Faculty login
- Route favorites
- Seat booking
- 40-seat seat map
- 10 standing slots
- QR tickets
- Booking cancellation
- Ticket sale/share
- Complaints
- Notifications
- Profile management
- Semester billing
- Booking history
- Occupancy dashboards

Extend the schedule system rather than replacing unrelated functionality.

43. FINAL TESTING REQUIREMENTS

Before completing the task, verify:

Route 01-A

The application displays:

```
6:40 AM first pickup
7:30 AM first arrival
8:30 AM second pickup
10:40 AM second arrival
2:05 PM outbound
5:10 PM outbound
```

Route 03-B

The application displays:

```
7:40 AM inbound
8:45 AM arrival
No first outbound
5:15 PM second outbound
```

Route 07

The application displays:

```
6:30 AM first pickup
7:35 AM arrival
No first outbound
5:15 PM second outbound
BDT 160
```

Route 08

The application displays:

```
7:00 AM → 7:40 AM
9:00 AM → 10:35 AM
2:05 PM outbound
5:10 PM outbound
BDT 50
```

Verify every stop-level pickup time against the data in this prompt.

44. FINAL COMMAND TO THE CODING AGENT

Integrate this complete BRAC University transportation timetable into the existing UniRide database and application.

Do not just redesign the Schedule page visually.

Actually:

1. Extend the normalized MySQL schema.
2. Add route stoppages.
3. Add timetable templates.
4. Add stop-level pickup times.
5. Insert all BRACU source timetable data.
6. Insert correct route fares.
7. Add inbound/outbound schedule views.
8. Connect them to PHP.
9. Connect PHP to the frontend.
10. Keep timetable templates separate from actual date-specific bookable schedules.
11. Allow administrators to assign buses and dates.
12. Preserve all booking and seat-management features.
13. Update the SQL installation file.
14. Update the README if the schema/import process changes.
15. Test every route and every time listed above.

Do **not invent missing schedule information**.

Where the provided source contains no value, store/display:

```
NULL
Not provided
Not listed
```

as appropriate.

The finished system must make the BRACU transport timetable a real **database-backed schedule module**, not a static image or hard-coded frontend table.

how can I connect my xamp database with the ai agent so that it will continuously update the database while working on the project?

Yes. The key thing is: **phpMyAdmin is only the interface**. Your AI coding agent needs direct access to the **MySQL/MariaDB server running inside XAMPP**.

For your UniRide project, I'd set it up like this.

1. Create a dedicated database user for the AI agent

In phpMyAdmin → **SQL**, run:

```sql
CREATE USER IF NOT EXISTS 'uniride_agent'@'localhost'
IDENTIFIED BY 'ChangeThisToAStrongPassword!';
```

```
GRANT SELECT, INSERT, UPDATE, DELETE,
      CREATE, ALTER, DROP, INDEX,
      REFERENCES, CREATE VIEW, SHOW VIEW,
      TRIGGER, EXECUTE
ON uniride_db.*
TO 'uniride_agent'@'localhost';

FLUSH PRIVILEGES;
```

Do **not** give the agent your XAMPP **`root`** account. Keep this limited to **`uniride_db`**.

---

2. Give your project a `.env` file

Create:

```
C:\xampp\htdocs\uniride\.env
```

with:

```
env
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=uniride_db
DB_USER=uniride_agent
DB_PASSWORD=ChangeThisToAStrongPassword!
```

Add **`.env`** to **`.gitignore`**:

```
gitignore
.env
```

Your AI agent can then use those credentials locally without hard-coding them into PHP files.

---

3. Configure PHP to use that database

Your **`config/database.php`** should effectively connect like this:

```
PHP
<?php

$host = '127.0.0.1';
$port = '3306';
$db   = 'uniride_db';
$user = 'uniride_agent';
$pass = 'ChangeThisToAStrongPassword!';
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;port=$port;dbname=$db;charset=$charset";

$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,
];

try {
    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (PDOException $e) {
    exit('Database connection failed.');
```

```
}
```

Better still, have the agent make PHP load these values from `.env`.

---

4. Let the coding agent use the MySQL command line

If your AI agent runs **locally on your Windows PC** and has terminal access, it can communicate with XAMPP's database through:

```
C:\xampp\mysql\bin\mysql.exe
```

For example:

```
PowerShell
C:\xampp\mysql\bin\mysql.exe -h 127.0.0.1 -P 3306 -u uniride_agent -p uniride_db
```

It will ask for the password.

The agent can also execute SQL files:

```
PowerShell
C:\xampp\mysql\bin\mysql.exe -u uniride_agent -p uniride_db < database\migration.sql
```

That is what allows it to **actually change your live development database**, rather than merely writing SQL for you to paste manually.

---

5. Give the agent this instruction

You can paste this directly into your coding agent:

```
You have permission to integrate directly with my local XAMPP MySQL/MariaDB development database
for this project.

Database configuration:

Host: 127.0.0.1
Port: 3306
Database: uniride_db

Credentials are stored in the project's .env file. Never hard-code or print the password.

XAMPP MySQL CLI is available at:

C:\xampp\mysql\bin\mysql.exe

From now on, whenever you modify the database architecture or a feature requires a database
change:

1. Inspect the current database/schema first.
2. Create a versioned SQL migration inside database/migrations/.
3. Execute the migration against the local uniride_db development database.
4. Verify the change using SHOW TABLES, DESCRIBE, SHOW CREATE TABLE, or relevant SELECT queries.
5. Update database/uniride_complete_phpmyadmin.sql so a fresh installation contains the same final
schema.
6. Update the PHP code to use the modified schema.
7. Test the PHP/database integration.
8. Never delete or reset existing database data unless the requested feature specifically requires
it.
9. Never use DROP DATABASE.
```

```
10. Before destructive ALTER/DROP operations, create a database backup.
11. Use prepared statements in PHP.
12. Do not modify databases other than uniride_db.
13. Keep all schema changes reproducible through SQL migration files.
14. Do not treat phpMyAdmin as the database API; connect directly to MySQL/MariaDB.
15. After every database change, verify that the application and database still work together.

Maintain this architecture:

HTML/CSS/JavaScript
        ↓
PHP
        ↓
PDO
        ↓
Local XAMPP MySQL/MariaDB
        ↓
uniride_db

All dashboard values, routes, schedules, stops, bookings, favorites, complaints, notifications,
fares, seat availability, transfers, and billing must use the live uniride_db database.

Do not fake data in JavaScript or HTML when the value belongs in the database.
```

6. Use migrations instead of letting the agent randomly alter the DB

I strongly recommend this structure:

```
uniride/
└── database/
    ├── uniride_complete_phpmyadmin.sql
    └── migrations/
        ├── 001_initial.sql
        ├── 002_add_route_stops.sql
        ├── 003_add_timetable_templates.sql
        ├── 004_add_stop_times.sql
        └── 005_update_bracu_schedule.sql
```

Then every time the agent changes something, you have a record of **what changed and when**.

This is much safer than repeatedly modifying your main SQL file without tracking changes.

7. Make automatic backups

Before allowing continuous changes, make a backup command available to the agent:

```
PowerShell
C:\xampp\mysql\bin\mysqldump.exe -u uniride_agent -p uniride_db > database\backups\uniride_backup.sql
```

For development, you could instruct it to create a backup before any destructive migration.

8. One important limitation

If your AI agent is a **cloud-based agent**, it cannot normally connect to:

```
127.0.0.1
localhost
```

on your computer. **`localhost`** from the cloud means **the agent's machine**, not yours.

So there are two cases:

Local agent / IDE agent with terminal access: perfect. It can connect directly to XAMPP as described above.

Cloud agent: don't expose XAMPP/MySQL directly to the public internet. Instead, either run the agent locally, use a secure development tunnel/VPN, or let the cloud agent generate migrations while a local process executes them.

For your university project, I strongly recommend the **local-agent + XAMPP + migrations** approach. It's the simplest and safest way to let the agent continuously update the real `uniride_db` while it builds the PHP application.

> so i want to add almost 50 more student information entries to the database for each university. search the web so make sample information for the students of BRACU, NSU, and AIUB and make a prompt for the ai tool to integrate them into the existing database

I checked the universities' official sites so the seed data can feel realistic **without copying real students**. BRAC University currently lists undergraduate programs including CSE, CS, EEE, ECE, BBA, Economics, Architecture, Biotechnology, Microbiology, English, Law and others; BRACU also documents student Google Workspace service under the `g.bracu.ac.bd` domain. BRAC University+1

North South University lists undergraduate programs across Business & Economics, Engineering, Life Sciences and related schools—including CSE, EEE, ETE, BBA, Economics, Microbiology, Biochemistry/Biotechnology and Pharmacy—and its official orientation material says student email is provided through a Google Gmail-based system. North South University+2 AIUB currently lists CSE, Data Science, Computer Network & Cyber Security, EEE, Computer Engineering, IPE, BBA and other undergraduate programs; unlike the generic email format we discussed earlier, AIUB's official documented student-email format is `universityID@student.aiub.edu`. AIUB+2

So I recommend making all 150 people **fully synthetic**—fictional names, IDs, and records—while using the universities' real program terminology and realistic email conventions. That gives your database realism without scraping or impersonating actual students.

Copy this whole prompt into your agent:

ADD 150 REALISTIC BUT FULLY SYNTHETIC UNIVERSITY STUDENTS TO UNIRIDE

Update the existing **UniRide Multi-University Bus Ticket Management System** by adding **150 new synthetic student accounts** to the existing MySQL database:

- 50 students for BRAC University / BRACU
- 50 students for North South University / NSU
- 50 students for American International University-Bangladesh / AIUB

Database:

```
uniride_db
```

Existing stack:

```
HTML
CSS
Vanilla JavaScript
PHP
PDO
MySQL
phpMyAdmin
XAMPP
```

Do not rebuild the project.

Do not delete existing students.

Append these students safely to the existing database.

---

1. CRITICAL PRIVACY RULE

All 150 student records must be **fictional synthetic demo data**.

DO NOT:

- scrape student directories
- copy real student names together with real IDs
- copy real student emails
- copy personal phone numbers
- copy addresses
- copy personal information from Facebook, LinkedIn or university pages
- attempt to identify real students

Web research was used only to determine realistic:

- university names
- academic programs
- department terminology
- institutional email conventions

Names, student IDs and accounts must be generated specifically for this demo project.

---

2. NUMBER OF NEW STUDENTS

Generate exactly:

```
BRACU = 50
NSU   = 50
AIUB  = 50
---------------
TOTAL = 150
```

Verify these numbers after insertion.

Do not count existing demo students toward the 150 new records.

---

3. EXISTING DATABASE STRUCTURE

Use the existing superclass/subclass design.

Main student account:

```
passengers
```

Student subclass information:

```
students
```

Every generated Student therefore requires:

```
1 row in passengers
+
1 corresponding row in students
```

Do not create a separate:

```
bracu_students
nsu_students
aiub_students
```

table.

Preserve the existing normalized design.

---

4. PASSENGER TABLE

For every generated student, populate the existing appropriate fields such as:

```
passenger_id
university_id
passenger_code
name
email
password_hash
passenger_type
avatar_path
email_notifications
in_app_notifications
status
created_at
```

Set:

```
passenger_type = STUDENT
status = ACTIVE
```

unless the schema uses equivalent uppercase enum values.

Use the actual existing column names in the project.

Do not alter the schema merely because an example field differs slightly.

---

5. STUDENT SUBCLASS TABLE

Insert matching records into:

```
students
```

Populate:

```
passenger_id
student_identifier
department
program
semester_label
```

Use the new `passenger_id` created in `passengers`.

Maintain:

```
passengers 1 : 1 students
```

for these generated Student records.

---

6. UNIVERSITY LOOKUP

Do not hard-code database numeric IDs such as:

```
BRACU = 1
NSU   = 2
AIUB  = 3
```

unless those values are first confirmed from the current database.

Locate universities using stable fields such as:

```
university_code
university_name
```

Examples:

```
BRACU
NSU
AIUB
```

Retrieve the actual `university_id` before inserting students.

---

7. IF UNIVERSITY RECORDS USE OLD PLACEHOLDER NAMES

Inspect the current database first.

If the project still contains placeholder universities such as:

```
MCU
GFU
ETU
```

but the application has already been migrated visually to:

```
BRAC University
North South University
American International University-Bangladesh
```

synchronize the database first.

Do not create duplicate universities simply because the display name differs.

There must be exactly one intended record for each university.

---

8. STUDENT NAME GENERATION

Generate natural-looking fictional Bangladeshi names.

Use a diverse mixture of names such as:

```
Samir Hossain
Nafisa Rahman
Tahmid Hasan
Tasnim Ahmed
Rafiul Islam
Nabila Chowdhury
Farhan Kabir
Sadia Karim
Rahat Mahmud
Tanjim Noor
Mehjabin Sultana
Adnan Rahman
Nusrat Jahan
Mahin Ahmed
Arif Hossain
Sanjida Islam
Rayhan Chowdhury
Nafis Ahmed
Fariha Rahman
Zarin Tasnim
```

These are examples of naming style only.

Generate enough unique fictional combinations for all 150 accounts.

Requirements:

- Full name must be unique enough for demo purposes.
- Avoid obvious placeholder names such as `Student 1`.
- Do not copy lists of actual students from university websites.
- Mix masculine, feminine and neutral/common Bangladeshi names naturally.
- Avoid creating all records with the same surname.

---

9. UNIQUE PASSENGER CODES

Generate consistent passenger codes.

BRACU

```
BRACU-STU-0001
BRACU-STU-0002
...
BRACU-STU-0050
```

NSU

```
NSU-STU-0001
NSU-STU-0002
...
NSU-STU-0050
```

AIUB

```
AIUB-STU-0001
AIUB-STU-0002
...
AIUB-STU-0050
```

Before insertion, check existing values to prevent collisions.

If these codes already exist, safely choose the next unused sequence.

---

10. EMAIL POLICY

These are local demonstration accounts.

Never send actual external emails to generated demo addresses.

Disable outbound production email for seeded demo accounts.

The database should use realistic institution-style addresses while making clear in documentation that the accounts are synthetic.

---

11. BRACU STUDENT EMAIL FORMAT

For BRACU demo students, use the project's realistic Google Workspace-style pattern:

```
firstname.lastname@g.bracu.ac.bd
```

Examples:

```
samir.hossain@g.bracu.ac.bd
nafisa.rahman@g.bracu.ac.bd
tahmid.hasan@g.bracu.ac.bd
```

BRAC University's official materials document student Google Workspace service under the `g.bracu.ac.bd` domain.

If two synthetic students would produce the same email, append a deterministic suffix:

```
samir.hossain2@g.bracu.ac.bd
```

Do not use:

```
@example.com
@mcu.demo
@bracu.demo
```

---

12. NSU STUDENT EMAIL FORMAT

NSU officially provides student email using Google's Gmail platform.

For this UniRide demo dataset, use a consistent realistic project convention:

```
firstname.lastname@northsouth.edu
```

Examples:

```
rafi.hasan@northsouth.edu
tasnim.ahmed@northsouth.edu
farhan.kabir@northsouth.edu
```

These addresses are generated synthetic demo records and must not be copied from actual NSU users.

For duplicate generated names:

```
rafi.hasan2@northsouth.edu
```

---

13. AIUB STUDENT EMAIL FORMAT

AIUB officially documents the student-address pattern as:

```
universityID@student.aiub.edu
```

Therefore AIUB should not use the generic BRACU-style naming convention.

Generate fictional AIUB-style student identifiers and construct:

```
<synthetic_student_id>@student.aiub.edu
```

Example synthetic pattern:

```
26-90001-1@student.aiub.edu
26-90002-1@student.aiub.edu
26-90003-1@student.aiub.edu
```

The numeric IDs above are generated demonstration identifiers only.

Do not obtain or reproduce real AIUB student IDs.

---

14. PASSWORD FOR DEMO STUDENTS

Use one easy demo password if appropriate:

```
password
```

BUT never store:

```
password
```

directly in MySQL.

Generate a real PHP password hash using:

```
password_hash('password', PASSWORD_DEFAULT);
```

Authentication must continue using:

```
password_verify();
```

If seed data requires one reusable bcrypt hash, generate it using PHP and document:

```
Demo Password: password
```

in README.

---

15. BRACU PROGRAM DISTRIBUTION

Generate 50 BRACU students using the following approximate distribution:

```
CSE / Computer Science & Engineering       12
BBA                                         8
EEE / Electrical & Electronic Engineering   6
ECE / Electronic & Communication Eng.       4
Economics                                    4
English                                      4
Architecture                                 3
Biotechnology                                3
Microbiology                                 2
Computer Science                             2
Law                                          2
-----------------------------------------------
TOTAL                                       50
```

Use realistic full program names where appropriate.

Examples:

```
Bachelor of Science in Computer Science and Engineering
Bachelor of Business Administration
Bachelor of Science in Electrical and Electronic Engineering
Bachelor of Science in Electronic and Communication Engineering
Bachelor of Social Science in Economics
Bachelor of Arts in English
Bachelor of Architecture
Bachelor of Science in Biotechnology
Bachelor of Science in Microbiology
Bachelor of Science in Computer Science
Bachelor of Laws
```

Do not invent nonexistent departments purely to fill rows.

---

16. BRACU DEPARTMENT LABELS

Use concise department values compatible with the application's filters.

Examples:

```
CSE
Business
EEE
ECE
Economics
English
Architecture
Biotechnology
Microbiology
Computer Science
Law
```

Keep naming consistent across all 50 BRACU records.

---

17. NSU PROGRAM DISTRIBUTION

Generate 50 NSU students approximately as follows:

```
Computer Science & Engineering             12
BBA                                        12
Electrical & Electronic Engineering         6
Electronic & Telecommunication Eng.         4
Economics                                    5
Architecture                                 3
Microbiology                                 3
Biochemistry & Biotechnology                 2
Pharmacy                                     3
-------------------------------------------
TOTAL                                       50
```

Use program terminology consistent with NSU's academic offerings.

---

18. NSU DEPARTMENT LABELS

Suggested database department values:

```
Electrical Engineering & Computer Science
Business
Economics
Architecture
Microbiology
Biochemistry & Biotechnology
Pharmacy
```

For CSE, EEE and ETE students, program names should distinguish the individual program while the department can remain:

```
Electrical Engineering & Computer Science
```

if that matches the existing application's schema.

---

19. AIUB PROGRAM DISTRIBUTION

Generate exactly 50 AIUB students approximately as follows:

```
Computer Science & Engineering             12
Computer Network & Cyber Security           5
Data Science                                5
Electrical & Electronic Engineering         7
Computer Engineering                        4
Industrial & Production Engineering         3
Business Administration                     8
Architecture                                 3
Economics                                    1
English                                      1
Journalism & Mass Communication              1
-------------------------------------------
TOTAL                                       50
```

Use the university's actual program terminology.

---

20. AIUB DEPARTMENT/FACULTY LABELS

Use consistent values based on the academic area.

Examples:

```
Computer Science
Electrical & Electronic Engineering
Computer Engineering
Industrial & Production Engineering
Business Administration
Architecture
Economics
English
Journalism & Mass Communication
```

If the application stores Faculty instead of Department, appropriate groupings include:

```
Faculty of Science and Technology
Faculty of Engineering
Faculty of Business Administration
Faculty of Arts and Social Sciences
```

Do not change the schema unnecessarily.

Use whichever existing field structure the application already expects.

---

21. SEMESTER DISTRIBUTION

Students should not all be in the same semester.

Generate a realistic mixture across:

```
1st
2nd
3rd
4th
5th
6th
7th
8th
```

Recommended approximate distribution per university:

```
1st semester   6
2nd semester   6
3rd semester   7
4th semester   7
5th semester   7
6th semester   7
7th semester   5
8th semester   5
----------------
TOTAL          50
```

Shuffle programs across semesters.

Do not make all CSE students the same semester.

---

22. SAMPLE BRACU RECORD STRUCTURE

A generated record should conceptually look like:

```
Name:
Samir Hossain
```

```
University:
BRAC University

Passenger Code:
BRACU-STU-0001

Passenger Type:
STUDENT

Email:
samir.hossain@g.bracu.ac.bd

Department:
CSE

Program:
Bachelor of Science in Computer Science and Engineering

Semester:
6th

Status:
ACTIVE

Password:
password
(stored as secure hash)
```

This is an example structure, not a real student.

---

23. SAMPLE NSU RECORD STRUCTURE

Example:

```
Name:
Nafisa Rahman

University:
North South University

Passenger Code:
NSU-STU-0001

Passenger Type:
STUDENT

Email:
nafisa.rahman@northsouth.edu

Department:
Electrical Engineering & Computer Science

Program:
Bachelor of Science in Computer Science & Engineering

Semester:
4th

Status:
ACTIVE

Password:
password
(stored as secure hash)
```

Synthetic demo record only.

---

24. SAMPLE AIUB RECORD STRUCTURE

Example:

```
Name:
Tahmid Hasan

University:
American International University-Bangladesh

Passenger Code:
AIUB-STU-0001

Student Identifier:
26-90001-1

Passenger Type:
STUDENT

Email:
26-90001-1@student.aiub.edu

Department:
Computer Science

Program:
Bachelor of Science in Computer Science & Engineering

Semester:
3rd

Status:
ACTIVE

Password:
password
(stored as secure hash)
```

Synthetic demo record only.

---

25. STUDENT IDENTIFIERS

Generate synthetic university-specific student identifiers.

Do not scrape real IDs.

Suggested project-safe conventions:

BRACU

```
BRACU-26-0001
BRACU-26-0002
...
```

NSU

```
NSU-26-0001
NSU-26-0002
...
```

AIUB

Use an AIUB-style fictional numeric structure:

```
26-90001-1
26-90002-1
26-90003-1
...
```

Document clearly that these are generated demo IDs.

---

26. SQL INSERT STRATEGY

Do not manually create 150 fragile INSERT statements without verification.

Generate the seed consistently.

Use a transaction:

```
START TRANSACTION;
```

Insert:

```
passengers
↓
obtain passenger_id
↓
students
```

for every generated student.

When all records pass validation:

```
COMMIT;
```

On failure:

```
ROLLBACK;
```

---

27. RESPECT THE EXISTING STUDENT TRIGGER

Inspect the existing database before inserting.

The current UniRide schema may automatically create a row in:

```
students
```

when:

```
passengers.passenger_type = 'STUDENT'
```

is inserted.

If that trigger exists:

DO NOT insert a second duplicate `students` row.

Instead:

```
INSERT passenger
↓
trigger creates students row
↓
UPDATE students
SET student_identifier, department, program, semester_label
```

Use the current database behavior rather than fighting it.

---

28. DATABASE-FIRST CHECK

Before writing any seed records, run:

```
SHOW TABLES;
```

and inspect:

```
DESCRIBE passengers;
DESCRIBE students;
DESCRIBE universities;
SHOW TRIGGERS;
```

Also inspect constraints:

```
SHOW CREATE TABLE passengers;
SHOW CREATE TABLE students;
```

Adapt to the existing schema.

Do not assume an outdated version of the database.

---

29. DUPLICATE PROTECTION

Before insertion check:

```
passenger_code
email
student_identifier
```

for duplicates.

Do not delete an existing user merely because a generated record collides.

Generate a different synthetic value instead.

---

30. DO NOT RESET AUTO_INCREMENT

Do not run:

```
TRUNCATE passengers;
```

Do not run:

```
ALTER TABLE passengers AUTO_INCREMENT = 1;
```

Do not recreate existing production/demo IDs unnecessarily.

Append safely.

31. DO NOT DROP DATABASE

Never run:

```
DROP DATABASE uniride_db;
```

for this task.

Never destroy existing:

- bookings
- schedules
- routes
- buses
- complaints
- billing
- favorites
- notifications
- transfers

This task is an additive seed-data operation.

---

32. CREATE A MIGRATION / SEED FILE

Create a new file such as:

```
database/seeds/002_add_150_demo_students.sql
```

or:

```
database/migrations/006_add_demo_students.sql
```

Do not bury the only copy of the new data in a temporary terminal command.

---

33. UPDATE MASTER SQL

After testing the seed successfully, ensure a fresh installation can reproduce the data.

Update:

```
database/uniride_complete_phpmyadmin.sql
```

or add an explicit documented seed import step.

Preferred installation:

```
1. Import master database SQL
2. Demo students automatically exist
```

If keeping seeds separate is cleaner:

```
1. Import uniride_complete_phpmyadmin.sql
2. Import 002_add_150_demo_students.sql
```

Document this clearly in README.

---

34. BACKUP FIRST

Before modifying the live XAMPP database, create a backup.

Example:

```
database/backups/before_150_students.sql
```

Do not overwrite the latest usable backup.

---

35. LIVE XAMPP DATABASE

Apply the migration to the active local:

```
uniride_db
```

Do not merely generate SQL without applying it if the agent has local database access.

After running the seed, verify actual MySQL results.

---

36. VERIFY STUDENT COUNTS

After insertion run an equivalent query:

```
SELECT
    u.university_code,
    COUNT(*) AS student_count

FROM passengers p

JOIN universities u
    ON u.university_id = p.university_id

WHERE p.passenger_type = 'STUDENT'

GROUP BY
    u.university_id,
    u.university_code;
```

Because there may already be demo students, also verify specifically the new passenger-code ranges.

For example:

```
SELECT COUNT(*)
FROM passengers
WHERE passenger_code LIKE 'BRACU-STU-%';
```

The migration itself must add exactly:

```
50 new BRACU
50 new NSU
```

```
50 new AIUB
```

37. VERIFY SUBCLASS CONSISTENCY

Check that every generated Student passenger has exactly one matching Student subclass.

Run an equivalent check:

```
SELECT
    p.passenger_id,
    p.name

FROM passengers p

LEFT JOIN students s
    ON s.passenger_id = p.passenger_id

WHERE p.passenger_type = 'STUDENT'
  AND s.passenger_id IS NULL;
```

For the newly generated records this must return:

```
0 rows
```

38. VERIFY EMAIL UNIQUENESS

Run:

```
SELECT
    email,
    COUNT(*) AS duplicate_count

FROM passengers

GROUP BY email

HAVING COUNT(*) > 1;
```

Expected result:

```
0 duplicated generated emails
```

39. VERIFY PROGRAM DISTRIBUTION

Produce a final report:

```
University
Program
Number of generated students
```

For example:

```
BRACU | CSE | 12
BRACU | BBA | 8
...
```

Confirm the totals equal exactly 50 per university.

40. UPDATE UNIVERSITY ADMIN PASSENGER PAGE

After inserting the students, ensure:

```
University Dashboard
→ Passenger Management
→ Students
```

can display all generated records.

Add or preserve:

- pagination
- search
- filter by program
- filter by department
- filter by semester
- filter by status

Do not attempt to display 50+ students in an unusably long page without pagination.

Recommended:

```
10 / 25 / 50 records per page
```

---

41. SEARCH

University admins should be able to search generated students by:

```
Name
Passenger Code
Student Identifier
Email
Department
Program
Semester
```

Search must query MySQL.

Do not filter only a hard-coded frontend array.

---

42. STUDENT LOGIN

Every generated student account should be able to authenticate through the existing passenger login system.

Login must query:

```
passengers
```

and use:

```
password_verify()
```

Test at least:

```
2 BRACU students
2 NSU students
2 AIUB students
```

after insertion.

---

43. BUS ELIGIBILITY

All 150 generated users are:

```
passenger_type = STUDENT
```

Therefore they should automatically be eligible for:

```
STUDENT buses
```

and blocked from:

```
FACULTY buses
```

according to existing UniRide rules.

Do not create separate eligibility code specifically for these demo users.

---

44. BOOKING TEST

Choose one generated student from each university and verify:

```
BRACU synthetic student
→ can view BRACU routes only
→ can book eligible BRACU student schedule

NSU synthetic student
→ can view NSU routes only

AIUB synthetic student
→ can view AIUB routes only
```

Verify multi-university isolation remains intact.

---

45. FAVORITES TEST

Choose one generated student and:

```
Add favorite
Refresh page
Verify favorite persists
Remove favorite
Verify deletion persists
```

---

46. COMPLAINT TEST

Choose one generated student from each university.

Submit a complaint.

Verify:

```
BRACU passenger → BRACU complaint
NSU passenger → NSU complaint
AIUB passenger → AIUB complaint
```

No cross-university complaint ownership should occur.

---

47. BILLING TEST

If a generated Student makes a valid booking:

```
booking
↓
billing transaction
↓
semester bill
```

must continue to work exactly like existing passenger accounts.

---

48. README

Add a short section:

```
Synthetic Demo Dataset
```

Explain:

```
The project contains 50 generated demo students for each
participating university:

BRAC University: 50
North South University: 50
AIUB: 50

All names, IDs and student accounts are synthetic and included
only for demonstrating database operations, search, filtering,
booking and multi-university functionality.
```

Do not suggest these records represent actual enrolled students.

---

49. EMAIL DISCLAIMER

Because some synthetic addresses use real university domains for realistic local demonstration:

```
Do not send external email to seeded demo accounts.
```

The application should use the database notification system for demonstrations.

If outbound email functionality exists, disable external delivery for seeded demo accounts.

---

50. FINAL OUTPUT FROM THE AGENT

After completing the task, report:

```
✓ Backup created
✓ Database inspected
✓ 50 BRACU students inserted
✓ 50 NSU students inserted
✓ 50 AIUB students inserted
✓ 150 total new Student passengers
✓ 150 matching Student subclass records
✓ No duplicate emails
✓ No duplicate passenger codes
✓ No duplicate student identifiers
✓ Password hashes verified
✓ Student login tested
✓ University isolation tested
✓ Passenger-management page tested
✓ Search/filter tested
✓ Seed/migration file saved
✓ Master SQL/README updated
```

Also provide:

```
1. Path of the SQL seed/migration
2. Summary count by university
3. Summary count by program
4. Three sample demo login accounts per university
```

FINAL COMMAND

Do not merely generate a list of student names.

Actually integrate the 150 synthetic students into the existing UniRide system.

Follow this workflow:

```
Inspect live database
        ↓
Create backup
        ↓
Generate fictional student dataset
        ↓
Validate uniqueness
        ↓
Create reproducible SQL seed/migration
        ↓
Apply to local XAMPP uniride_db
        ↓
Verify database counts
        ↓
Verify Student subclass
        ↓
Test login
        ↓
Test university isolation
        ↓
Test dashboard/search/filter
        ↓
Update master SQL
        ↓
Update README
```

Preserve all existing:

- routes
- BRACU timetable data
- schedules
- buses

- bookings
- QR tickets
- favorites
- complaints
- notifications
- ticket transfers
- semester billing
- Admin accounts
- University Admin accounts
- Faculty accounts

This task must add **exactly 150 new synthetic Student accounts without damaging existing project data**.

One correction from our earlier email convention is worth keeping: for **AIUB**, their official documentation explicitly says student accounts use **`universityID@student.aiub.edu`**, so I would use that in your project instead of **`firstname.lastname@g.aiub.ac.bd`**. AIUB+1 For NSU, the university confirms a Gmail-based student email service, and its official material shows the **`northsouth.edu`** domain in use; the exact synthetic **`firstname.lastname@northsouth.edu`** convention in the prompt is therefore a **project convention**, not a claim that every real NSU student follows that exact naming rule. North South University+1

give me all the data manually which i need to create the database for the whole website

Yes. For the **whole UniRide website**, the cleanest approach is to divide your database data into **master/demo data that you enter yourself** and **transactional data that the website should create automatically**.

Below is a complete seed-data plan you can use manually. All student/faculty names and IDs below are **synthetic demo data**.

1. Global defaults

Use these values throughout the demo database:

```
Database: uniride_db

Demo password for all accounts:
password

IMPORTANT:
Store password_hash('password', PASSWORD_DEFAULT)
Never store the literal password in password_hash.

Seat capacity per standard bus: 40
Standing capacity: 10

Passenger Status: ACTIVE
University Status: ACTIVE
Bus Status: ACTIVE
```

```
Route Status: ACTIVE

email_notifications: 1
in_app_notifications: 1
```

Let numeric IDs such as **`passenger_id`**, **`booking_id`**, **`schedule_id`**, etc. use **`AUTO_INCREMENT`**. Do not manually depend on fixed numeric IDs in PHP.

---

2. Universities

Code	University Name	Address	Contact	Email	Status
BRACU	BRAC University	Dhaka, Bangladesh	+8801700000001	transport.bracu@gmail.com	ACTIVE
NSU	North South University	Dhaka, Bangladesh	+8801700000002	transport.nsu@gmail.com	ACTIVE
AIUB	American International University-Bangladesh	Dhaka, Bangladesh	+8801700000003	transport.aiub@gmail.com	ACTIVE

---

3. System Admin

```
Admin Code: ADMIN-001
Name: UniRide System Administrator
Email: uniride.admin@gmail.com
Password: password
Status: ACTIVE
```

4. University Admins

University	Name	Email	Role	Password
BRACU	BRACU Transport Admin	bracu.transport@gmail.com	UNIVERSITY_ADMIN	password
NSU	NSU Transport Admin	nsu.transport@gmail.com	UNIVERSITY_ADMIN	password
AIUB	AIUB Transport Admin	aiub.transport@gmail.com	UNIVERSITY_ADMIN	password

---

5. Semester data

Create one active semester for every university:

University	Semester	Start	End	Status
BRACU	Fall 2026	2026-08-01	2026-12-31	ACTIVE
NSU	Fall 2026	2026-08-01	2026-12-31	ACTIVE
AIUB	Fall 2026	2026-08-01	2026-12-31	ACTIVE

You can later add Spring 2027 and Summer 2027 from the admin panel.

6. Program-code legend

For the student dataset below:

Code	Program
CSE	Computer Science and Engineering
CS	Computer Science
BBA	Business Administration
EEE	Electrical and Electronic Engineering
ECE	Electronic and Communication Engineering
ETE	Electronic and Telecommunication Engineering
CNCS	Computer Network and Cyber Security
DS	Data Science
CE	Computer Engineering
IPE	Industrial and Production Engineering
ECON	Economics
ENG	English
ARCH	Architecture
BIOTECH	Biotechnology
MICRO	Microbiology
BIOCHEM	Biochemistry and Biotechnology
PHARM	Pharmacy
LAW	Law
JMC	Journalism and Mass Communication

7. BRACU — 50 synthetic students

All rows use:

```
University: BRACU
Passenger Type: STUDENT
Status: ACTIVE
Password: password
```

```
Passenger Code | Name | Email | Student Identifier | Program | Semester

BRACU-STU-0001 | Samir Hossain | samir.hossain@g.bracu.ac.bd | BRACU-26-0001 | CSE | 1st
BRACU-STU-0002 | Nafisa Ali | nafisa.ali@g.bracu.ac.bd | BRACU-26-0002 | CSE | 2nd
BRACU-STU-0003 | Tahmid Azad | tahmid.azad@g.bracu.ac.bd | BRACU-26-0003 | CSE | 3rd
BRACU-STU-0004 | Tasnim Siam | tasnim.siam@g.bracu.ac.bd | BRACU-26-0004 | CSE | 4th
BRACU-STU-0005 | Rafiul Sayeed | rafiul.sayeed@g.bracu.ac.bd | BRACU-26-0005 | CSE | 5th
BRACU-STU-0006 | Nabila Kamal | nabila.kamal@g.bracu.ac.bd | BRACU-26-0006 | CSE | 6th
BRACU-STU-0007 | Farhan Rumi | farhan.rumi@g.bracu.ac.bd | BRACU-26-0007 | CSE | 7th
BRACU-STU-0008 | Sadia Khan | sadia.khan@g.bracu.ac.bd | BRACU-26-0008 | CSE | 8th
BRACU-STU-0009 | Rahat Shuvo | rahat.shuvo@g.bracu.ac.bd | BRACU-26-0009 | CSE | 1st
BRACU-STU-0010 | Tanjim Morshed | tanjim.morshed@g.bracu.ac.bd | BRACU-26-0010 | CSE | 2nd
BRACU-STU-0011 | Mehjabin Mirza | mehjabin.mirza@g.bracu.ac.bd | BRACU-26-0011 | CSE | 3rd
```

```
BRACU-STU-0012 | Adnan Halim | adnan.halim@g.bracu.ac.bd | BRACU-26-0012 | CSE | 4th

BRACU-STU-0013 | Nusrat Wahid | nusrat.wahid@g.bracu.ac.bd | BRACU-26-0013 | BBA | 5th
BRACU-STU-0014 | Mahin Noor | mahin.noor@g.bracu.ac.bd | BRACU-26-0014 | BBA | 6th
BRACU-STU-0015 | Arif Amin | arif.amin@g.bracu.ac.bd | BRACU-26-0015 | BBA | 7th
BRACU-STU-0016 | Sanjida Noman | sanjida.noman@g.bracu.ac.bd | BRACU-26-0016 | BBA | 8th
BRACU-STU-0017 | Rayhan Rifat | rayhan.rifat@g.bracu.ac.bd | BRACU-26-0017 | BBA | 1st
BRACU-STU-0018 | Nafis Mollah | nafis.mollah@g.bracu.ac.bd | BRACU-26-0018 | BBA | 2nd
BRACU-STU-0019 | Fariha Nasrin | fariha.nasrin@g.bracu.ac.bd | BRACU-26-0019 | BBA | 3rd
BRACU-STU-0020 | Zarin Karim | zarin.karim@g.bracu.ac.bd | BRACU-26-0020 | BBA | 4th

BRACU-STU-0021 | Shafin Alam | shafin.alam@g.bracu.ac.bd | BRACU-26-0021 | EEE | 5th
BRACU-STU-0022 | Maliha Bashar | maliha.bashar@g.bracu.ac.bd | BRACU-26-0022 | EEE | 6th
BRACU-STU-0023 | Arafat Emon | arafat.emon@g.bracu.ac.bd | BRACU-26-0023 | EEE | 7th
BRACU-STU-0024 | Ishrat Kader | ishrat.kader@g.bracu.ac.bd | BRACU-26-0024 | EEE | 8th
BRACU-STU-0025 | Samin Rasel | samin.rasel@g.bracu.ac.bd | BRACU-26-0025 | EEE | 1st
BRACU-STU-0026 | Sumaiya Chowdhury | sumaiya.chowdhury@g.bracu.ac.bd | BRACU-26-0026 | EEE | 2nd

BRACU-STU-0027 | Abrar Bhuiyan | abrar.bhuiyan@g.bracu.ac.bd | BRACU-26-0027 | ECE | 3rd
BRACU-STU-0028 | Anika Zaman | anika.zaman@g.bracu.ac.bd | BRACU-26-0028 | ECE | 4th
BRACU-STU-0029 | Tawsif Mizan | tawsif.mizan@g.bracu.ac.bd | BRACU-26-0029 | ECE | 5th
BRACU-STU-0030 | Raisa Rabbani | raisa.rabbani@g.bracu.ac.bd | BRACU-26-0030 | ECE | 6th

BRACU-STU-0031 | Fahim Jamal | fahim.jamal@g.bracu.ac.bd | BRACU-26-0031 | ECON | 7th
BRACU-STU-0032 | Mim Ahmed | mim.ahmed@g.bracu.ac.bd | BRACU-26-0032 | ECON | 8th
BRACU-STU-0033 | Shahriar Mia | shahriar.mia@g.bracu.ac.bd | BRACU-26-0033 | ECON | 1st
BRACU-STU-0034 | Lamisa Iqbal | lamisa.iqbal@g.bracu.ac.bd | BRACU-26-0034 | ECON | 2nd

BRACU-STU-0035 | Rakin Shah | rakin.shah@g.bracu.ac.bd | BRACU-26-0035 | ENG | 3rd
BRACU-STU-0036 | Ayesha Mubin | ayesha.mubin@g.bracu.ac.bd | BRACU-26-0036 | ENG | 4th
BRACU-STU-0037 | Sabbir Sohan | sabbir.sohan@g.bracu.ac.bd | BRACU-26-0037 | ENG | 5th
BRACU-STU-0038 | Oishi Rahman | oishi.rahman@g.bracu.ac.bd | BRACU-26-0038 | ENG | 6th

BRACU-STU-0039 | Muntasir Uddin | muntasir.uddin@g.bracu.ac.bd | BRACU-26-0039 | ARCH | 7th
BRACU-STU-0040 | Jannat Bari | jannat.bari@g.bracu.ac.bd | BRACU-26-0040 | ARCH | 8th
BRACU-STU-0041 | Imran Faisal | imran.faisal@g.bracu.ac.bd | BRACU-26-0041 | ARCH | 1st

BRACU-STU-0042 | Priya Ahsan | priya.ahsan@g.bracu.ac.bd | BRACU-26-0042 | BIOTECH | 2nd
BRACU-STU-0043 | Sakib Bashir | sakib.bashir@g.bracu.ac.bd | BRACU-26-0043 | BIOTECH | 3rd
BRACU-STU-0044 | Nadia Salam | nadia.salam@g.bracu.ac.bd | BRACU-26-0044 | BIOTECH | 4th

BRACU-STU-0045 | Zubair Jahan | zubair.jahan@g.bracu.ac.bd | BRACU-26-0045 | MICRO | 5th
BRACU-STU-0046 | Tania Nawaz | tania.nawaz@g.bracu.ac.bd | BRACU-26-0046 | MICRO | 6th

BRACU-STU-0047 | Fardin Rumi | fardin.rumi@g.bracu.ac.bd | BRACU-26-0047 | CS | 7th
BRACU-STU-0048 | Mahi Shams | mahi.shams@g.bracu.ac.bd | BRACU-26-0048 | CS | 8th

BRACU-STU-0049 | Hasib Rauf | hasib.rauf@g.bracu.ac.bd | BRACU-26-0049 | LAW | 1st
BRACU-STU-0050 | Sohana Touhid | sohana.touhid@g.bracu.ac.bd | BRACU-26-0050 | LAW | 2nd
```

8. NSU — 50 synthetic students

```
NSU-STU-0001 | Saif Sultana | saif.sultana@northsouth.edu | NSU-26-0001 | CSE | 1st
NSU-STU-0002 | Rukaiya Siddique | rukaiya.siddique@northsouth.edu | NSU-26-0002 | CSE | 2nd
NSU-STU-0003 | Shakib Arefin | shakib.arefin@northsouth.edu | NSU-26-0003 | CSE | 3rd
NSU-STU-0004 | Farzana Rony | farzana.rony@northsouth.edu | NSU-26-0004 | CSE | 4th
NSU-STU-0005 | Anik Parvez | anik.parvez@northsouth.edu | NSU-26-0005 | CSE | 5th
NSU-STU-0006 | Orin Pervin | orin.pervin@northsouth.edu | NSU-26-0006 | CSE | 6th
NSU-STU-0007 | Ridwan Mahmud | ridwan.mahmud@northsouth.edu | NSU-26-0007 | CSE | 7th
NSU-STU-0008 | Maisha Sarker | maisha.sarker@northsouth.edu | NSU-26-0008 | CSE | 8th
NSU-STU-0009 | Jubayer Huq | jubayer.huq@northsouth.edu | NSU-26-0009 | CSE | 1st
NSU-STU-0010 | Samiha Nayeem | samiha.nayeem@northsouth.edu | NSU-26-0010 | CSE | 2nd
NSU-STU-0011 | Riad Aziz | riad.aziz@northsouth.edu | NSU-26-0011 | CSE | 3rd
NSU-STU-0012 | Noshin Binte | noshin.binte@northsouth.edu | NSU-26-0012 | CSE | 4th

NSU-STU-0013 | Abeer Kabir | abeer.kabir@northsouth.edu | NSU-26-0013 | BBA | 5th
NSU-STU-0014 | Tasmia Haque | tasmia.haque@northsouth.edu | NSU-26-0014 | BBA | 6th
NSU-STU-0015 | Ayman Anwar | ayman.anwar@northsouth.edu | NSU-26-0015 | BBA | 7th
NSU-STU-0016 | Sharmin Safa | sharmin.safa@northsouth.edu | NSU-26-0016 | BBA | 8th
NSU-STU-0017 | Rakib Mamun | rakib.mamun@northsouth.edu | NSU-26-0017 | BBA | 1st
NSU-STU-0018 | Moumita Hakim | moumita.hakim@northsouth.edu | NSU-26-0018 | BBA | 2nd
NSU-STU-0019 | Afnan Islam | afnan.islam@northsouth.edu | NSU-26-0019 | BBA | 3rd
```

```
NSU-STU-0020 | Nawar Talukder | nawar.talukder@northsouth.edu | NSU-26-0020 | BBA | 4th
NSU-STU-0021 | Shadman Faruq | shadman.faruq@northsouth.edu | NSU-26-0021 | BBA | 5th
NSU-STU-0022 | Elma Hridoy | elma.hridoy@northsouth.edu | NSU-26-0022 | BBA | 6th
NSU-STU-0023 | Rashed Saklain | rashed.saklain@northsouth.edu | NSU-26-0023 | BBA | 7th
NSU-STU-0024 | Lamia Masud | lamia.masud@northsouth.edu | NSU-26-0024 | BBA | 8th

NSU-STU-0025 | Rafid Hasan | rafid.hasan@northsouth.edu | NSU-26-0025 | EEE | 1st
NSU-STU-0026 | Sabrina Akter | sabrina.akter@northsouth.edu | NSU-26-0026 | EEE | 2nd
NSU-STU-0027 | Tanvir Rashid | tanvir.rashid@northsouth.edu | NSU-26-0027 | EEE | 3rd
NSU-STU-0028 | Noshaba Zahid | noshaba.zahid@northsouth.edu | NSU-26-0028 | EEE | 4th
NSU-STU-0029 | Adit Tareq | adit.tareq@northsouth.edu | NSU-26-0029 | EEE | 5th
NSU-STU-0030 | Mariam Aref | mariam.aref@northsouth.edu | NSU-26-0030 | EEE | 6th

NSU-STU-0031 | Samir Rahman | samir.rahman@northsouth.edu | NSU-26-0031 | ETE | 7th
NSU-STU-0032 | Nafisa Uddin | nafisa.uddin@northsouth.edu | NSU-26-0032 | ETE | 8th
NSU-STU-0033 | Tahmid Bari | tahmid.bari@northsouth.edu | NSU-26-0033 | ETE | 1st
NSU-STU-0034 | Tasnim Faisal | tasnim.faisal@northsouth.edu | NSU-26-0034 | ETE | 2nd

NSU-STU-0035 | Rafiul Ahsan | rafiul.ahsan@northsouth.edu | NSU-26-0035 | ECON | 3rd
NSU-STU-0036 | Nabila Bashir | nabila.bashir@northsouth.edu | NSU-26-0036 | ECON | 4th
NSU-STU-0037 | Farhan Salam | farhan.salam@northsouth.edu | NSU-26-0037 | ECON | 5th
NSU-STU-0038 | Sadia Jahan | sadia.jahan@northsouth.edu | NSU-26-0038 | ECON | 6th
NSU-STU-0039 | Rahat Nawaz | rahat.nawaz@northsouth.edu | NSU-26-0039 | ECON | 7th

NSU-STU-0040 | Tanjim Rumi | tanjim.rumi@northsouth.edu | NSU-26-0040 | ARCH | 8th
NSU-STU-0041 | Mehjabin Shams | mehjabin.shams@northsouth.edu | NSU-26-0041 | ARCH | 1st
NSU-STU-0042 | Adnan Rauf | adnan.rauf@northsouth.edu | NSU-26-0042 | ARCH | 2nd

NSU-STU-0043 | Nusrat Touhid | nusrat.touhid@northsouth.edu | NSU-26-0043 | MICRO | 3rd
NSU-STU-0044 | Mahin Sultana | mahin.sultana@northsouth.edu | NSU-26-0044 | MICRO | 4th
NSU-STU-0045 | Arif Siddique | arif.siddique@northsouth.edu | NSU-26-0045 | MICRO | 5th

NSU-STU-0046 | Sanjida Arefin | sanjida.arefin@northsouth.edu | NSU-26-0046 | BIOCHEM | 6th
NSU-STU-0047 | Rayhan Rony | rayhan.rony@northsouth.edu | NSU-26-0047 | BIOCHEM | 7th

NSU-STU-0048 | Nafis Parvez | nafis.parvez@northsouth.edu | NSU-26-0048 | PHARM | 8th
NSU-STU-0049 | Fariha Pervin | fariha.pervin@northsouth.edu | NSU-26-0049 | PHARM | 1st
NSU-STU-0050 | Zarin Mahmud | zarin.mahmud@northsouth.edu | NSU-26-0050 | PHARM | 2nd
```

9. AIUB — 50 synthetic students

```
AIUB-STU-0001 | Shafin Sarker | 26-90001-1@student.aiub.edu | 26-90001-1 | CSE | 1st
AIUB-STU-0002 | Maliha Huq | 26-90002-1@student.aiub.edu | 26-90002-1 | CSE | 2nd
AIUB-STU-0003 | Arafat Nayeem | 26-90003-1@student.aiub.edu | 26-90003-1 | CSE | 3rd
AIUB-STU-0004 | Ishrat Aziz | 26-90004-1@student.aiub.edu | 26-90004-1 | CSE | 4th
AIUB-STU-0005 | Samin Binte | 26-90005-1@student.aiub.edu | 26-90005-1 | CSE | 5th
AIUB-STU-0006 | Sumaiya Kabir | 26-90006-1@student.aiub.edu | 26-90006-1 | CSE | 6th
AIUB-STU-0007 | Abrar Haque | 26-90007-1@student.aiub.edu | 26-90007-1 | CSE | 7th
AIUB-STU-0008 | Anika Anwar | 26-90008-1@student.aiub.edu | 26-90008-1 | CSE | 8th
AIUB-STU-0009 | Tawsif Safa | 26-90009-1@student.aiub.edu | 26-90009-1 | CSE | 1st
AIUB-STU-0010 | Raisa Mamun | 26-90010-1@student.aiub.edu | 26-90010-1 | CSE | 2nd
AIUB-STU-0011 | Fahim Hakim | 26-90011-1@student.aiub.edu | 26-90011-1 | CSE | 3rd
AIUB-STU-0012 | Mim Islam | 26-90012-1@student.aiub.edu | 26-90012-1 | CSE | 4th

AIUB-STU-0013 | Shahriar Talukder | 26-90013-1@student.aiub.edu | 26-90013-1 | CNCS | 5th
AIUB-STU-0014 | Lamisa Faruq | 26-90014-1@student.aiub.edu | 26-90014-1 | CNCS | 6th
AIUB-STU-0015 | Rakin Hridoy | 26-90015-1@student.aiub.edu | 26-90015-1 | CNCS | 7th
AIUB-STU-0016 | Ayesha Saklain | 26-90016-1@student.aiub.edu | 26-90016-1 | CNCS | 8th
AIUB-STU-0017 | Sabbir Masud | 26-90017-1@student.aiub.edu | 26-90017-1 | CNCS | 1st

AIUB-STU-0018 | Oishi Hasan | 26-90018-1@student.aiub.edu | 26-90018-1 | DS | 2nd
AIUB-STU-0019 | Muntasir Akter | 26-90019-1@student.aiub.edu | 26-90019-1 | DS | 3rd
AIUB-STU-0020 | Jannat Rashid | 26-90020-1@student.aiub.edu | 26-90020-1 | DS | 4th
AIUB-STU-0021 | Imran Zahid | 26-90021-1@student.aiub.edu | 26-90021-1 | DS | 5th
AIUB-STU-0022 | Priya Tareq | 26-90022-1@student.aiub.edu | 26-90022-1 | DS | 6th

AIUB-STU-0023 | Sakib Aref | 26-90023-1@student.aiub.edu | 26-90023-1 | EEE | 7th
AIUB-STU-0024 | Nadia Hossain | 26-90024-1@student.aiub.edu | 26-90024-1 | EEE | 8th
AIUB-STU-0025 | Zubair Ali | 26-90025-1@student.aiub.edu | 26-90025-1 | EEE | 1st
AIUB-STU-0026 | Tania Azad | 26-90026-1@student.aiub.edu | 26-90026-1 | EEE | 2nd
AIUB-STU-0027 | Fardin Siam | 26-90027-1@student.aiub.edu | 26-90027-1 | EEE | 3rd
AIUB-STU-0028 | Mahi Sayeed | 26-90028-1@student.aiub.edu | 26-90028-1 | EEE | 4th
```

```
AIUB-STU-0029 | Hasib Kamal | 26-90029-1@student.aiub.edu | 26-90029-1 | EEE | 5th

AIUB-STU-0030 | Sohana Rumi | 26-90030-1@student.aiub.edu | 26-90030-1 | CE | 6th
AIUB-STU-0031 | Saif Khan | 26-90031-1@student.aiub.edu | 26-90031-1 | CE | 7th
AIUB-STU-0032 | Rukaiya Shuvo | 26-90032-1@student.aiub.edu | 26-90032-1 | CE | 8th
AIUB-STU-0033 | Shakib Morshed | 26-90033-1@student.aiub.edu | 26-90033-1 | CE | 1st

AIUB-STU-0034 | Farzana Mirza | 26-90034-1@student.aiub.edu | 26-90034-1 | IPE | 2nd
AIUB-STU-0035 | Anik Halim | 26-90035-1@student.aiub.edu | 26-90035-1 | IPE | 3rd
AIUB-STU-0036 | Orin Wahid | 26-90036-1@student.aiub.edu | 26-90036-1 | IPE | 4th

AIUB-STU-0037 | Ridwan Noor | 26-90037-1@student.aiub.edu | 26-90037-1 | BBA | 5th
AIUB-STU-0038 | Maisha Amin | 26-90038-1@student.aiub.edu | 26-90038-1 | BBA | 6th
AIUB-STU-0039 | Jubayer Noman | 26-90039-1@student.aiub.edu | 26-90039-1 | BBA | 7th
AIUB-STU-0040 | Samiha Rifat | 26-90040-1@student.aiub.edu | 26-90040-1 | BBA | 8th
AIUB-STU-0041 | Riad Mollah | 26-90041-1@student.aiub.edu | 26-90041-1 | BBA | 1st
AIUB-STU-0042 | Noshin Nasrin | 26-90042-1@student.aiub.edu | 26-90042-1 | BBA | 2nd
AIUB-STU-0043 | Abeer Karim | 26-90043-1@student.aiub.edu | 26-90043-1 | BBA | 3rd
AIUB-STU-0044 | Tasmia Alam | 26-90044-1@student.aiub.edu | 26-90044-1 | BBA | 4th

AIUB-STU-0045 | Ayman Bashar | 26-90045-1@student.aiub.edu | 26-90045-1 | ARCH | 5th
AIUB-STU-0046 | Sharmin Emon | 26-90046-1@student.aiub.edu | 26-90046-1 | ARCH | 6th
AIUB-STU-0047 | Rakib Kader | 26-90047-1@student.aiub.edu | 26-90047-1 | ARCH | 7th

AIUB-STU-0048 | Moumita Rasel | 26-90048-1@student.aiub.edu | 26-90048-1 | ECON | 8th
AIUB-STU-0049 | Afnan Chowdhury | 26-90049-1@student.aiub.edu | 26-90049-1 | ENG | 1st
AIUB-STU-0050 | Nawar Bhuiyan | 26-90050-1@student.aiub.edu | 26-90050-1 | JMC | 2nd
```

10. Faculty demo data

Create at least five Faculty passengers per university.

```
BRACU-FAC-0001 | Kamal Uddin | kamal.uddin@g.bracu.ac.bd | BRACU-F-001 | CSE | Professor
BRACU-FAC-0002 | Nusrat Karim | nusrat.karim@g.bracu.ac.bd | BRACU-F-002 | EEE | Associate
Professor
BRACU-FAC-0003 | Farhana Rahman | farhana.rahman@g.bracu.ac.bd | BRACU-F-003 | Business |
Assistant Professor
BRACU-FAC-0004 | Mahmud Hasan | mahmud.hasan@g.bracu.ac.bd | BRACU-F-004 | Economics | Lecturer
BRACU-FAC-0005 | Samira Noor | samira.noor@g.bracu.ac.bd | BRACU-F-005 | English | Senior Lecturer

NSU-FAC-0001 | Rezaul Karim | rezaul.karim@northsouth.edu | NSU-F-001 | CSE | Professor
NSU-FAC-0002 | Farzana Ahmed | farzana.ahmed@northsouth.edu | NSU-F-002 | Business | Associate
Professor
NSU-FAC-0003 | Tariq Hasan | tariq.hasan@northsouth.edu | NSU-F-003 | EEE | Assistant Professor
NSU-FAC-0004 | Sabrina Rahman | sabrina.rahman@northsouth.edu | NSU-F-004 | Economics | Lecturer
NSU-FAC-0005 | Faisal Hossain | faisal.hossain@northsouth.edu | NSU-F-005 | Pharmacy | Senior
Lecturer

AIUB-FAC-0001 | Farhana Noor | farhana.noor@aiub.edu | AIUB-F-001 | CSE | Professor
AIUB-FAC-0002 | Tanvir Ahmed | tanvir.ahmed@aiub.edu | AIUB-F-002 | EEE | Associate Professor
AIUB-FAC-0003 | Sabrina Islam | sabrina.islam@aiub.edu | AIUB-F-003 | Business | Assistant
Professor
AIUB-FAC-0004 | Mahmud Kabir | mahmud.kabir@aiub.edu | AIUB-F-004 | Computer Engineering |
Lecturer
AIUB-FAC-0005 | Nadia Rahman | nadia.rahman@aiub.edu | AIUB-F-005 | Architecture | Senior Lecturer
```

For these:

```
passenger_type = FACULTY
status = ACTIVE
password = password
```

Put common data in **`passengers`** and faculty-specific data in **`faculty`**.

---

11. BRACU buses

Because BRACU has many concurrent route variants, give it multiple demo buses.

```
Registration | Tax Number | Type | Seats | Standing

DHAKA-METRO-B-11-1001 | BRACU-TAX-001 | STUDENT | 40 | 10
DHAKA-METRO-B-11-1002 | BRACU-TAX-002 | STUDENT | 40 | 10
DHAKA-METRO-B-11-1003 | BRACU-TAX-003 | STUDENT | 40 | 10
DHAKA-METRO-B-11-1004 | BRACU-TAX-004 | STUDENT | 40 | 10
DHAKA-METRO-B-11-1005 | BRACU-TAX-005 | STUDENT | 40 | 10
DHAKA-METRO-B-11-1006 | BRACU-TAX-006 | STUDENT | 40 | 10
DHAKA-METRO-B-11-1007 | BRACU-TAX-007 | STUDENT | 40 | 10
DHAKA-METRO-B-11-1008 | BRACU-TAX-008 | STUDENT | 40 | 10
DHAKA-METRO-B-11-1009 | BRACU-TAX-009 | STUDENT | 40 | 10
DHAKA-METRO-B-11-1010 | BRACU-TAX-010 | STUDENT | 40 | 10
DHAKA-METRO-B-11-1011 | BRACU-TAX-011 | STUDENT | 40 | 10
DHAKA-METRO-B-11-1012 | BRACU-TAX-012 | STUDENT | 40 | 10

DHAKA-METRO-B-11-1101 | BRACU-TAX-F01 | FACULTY | 40 | 10
DHAKA-METRO-B-11-1102 | BRACU-TAX-F02 | FACULTY | 40 | 10
```

12. NSU demo buses

These are **synthetic project data**, not claimed as NSU's real fleet.

```
DHAKA-METRO-B-22-2001 | NSU-TAX-001 | STUDENT | 40 | 10
DHAKA-METRO-B-22-2002 | NSU-TAX-002 | STUDENT | 40 | 10
DHAKA-METRO-B-22-2003 | NSU-TAX-003 | STUDENT | 40 | 10
DHAKA-METRO-B-22-2004 | NSU-TAX-004 | STUDENT | 40 | 10

DHAKA-METRO-B-22-2101 | NSU-TAX-F01 | FACULTY | 40 | 10
DHAKA-METRO-B-22-2102 | NSU-TAX-F02 | FACULTY | 40 | 10
```

13. AIUB demo buses

```
DHAKA-METRO-B-33-3001 | AIUB-TAX-001 | STUDENT | 40 | 10
DHAKA-METRO-B-33-3002 | AIUB-TAX-002 | STUDENT | 40 | 10
DHAKA-METRO-B-33-3003 | AIUB-TAX-003 | STUDENT | 40 | 10
DHAKA-METRO-B-33-3004 | AIUB-TAX-004 | STUDENT | 40 | 10

DHAKA-METRO-B-33-3101 | AIUB-TAX-F01 | FACULTY | 40 | 10
DHAKA-METRO-B-33-3102 | AIUB-TAX-F02 | FACULTY | 40 | 10
```

14. BRACU routes

These are the 12 route variants from the transport timetable you supplied.

Route Code	Route Name	Start	Destination	Fare
BRAC-R01-A	Abdullahpur-A to BRAC University	Abdullahpur	BRAC University	110
BRAC-R01-B	Abdullahpur-B to BRAC University	Abdullahpur	BRAC University	110
BRAC-R02-A	Mirpur-A to BRAC University	Mirpur	BRAC University	110
BRAC-R02-B	Mirpur-B to BRAC University	Mirpur	BRAC University	110
BRAC-R03-A	Jigatola-A to BRAC University	Jigatola	BRAC University	110
BRAC-R03-B	Jigatola-B to BRAC University	Jigatola	BRAC University	110
BRAC-R04	Azimpur to BRAC University	Azimpur	BRAC University	110
BRAC-R05	Baldha Garden to BRAC University	Baldha Garden	BRAC University	110
BRAC-R06-A	Mohammadpur-A to BRAC University	Mohammadpur	BRAC University	110

Route Code	Route Name	Start	Destination	Fare
BRAC-R06-B	Mohammadpur-B to BRAC University	Mohammadpur	BRAC University	110
BRAC-R07	Narayanganj to BRAC University	Narayanganj	BRAC University	160
BRAC-R08	Bashundhara R/A to BRAC University	Bashundhara R/A	BRAC University	50

Round-trip informational fares:

```
Routes 01–06: BDT 220
Route 07: BDT 320
Route 08: BDT 100
```

For your booking database, store **one-way fare** as the normal `routes.fare`.

---

15. BRACU timetable templates

Use this as your main timetable summary:

```
Route | Inbound Trip 1 | Inbound Trip 2 | Outbound 1 | Outbound 2

01-A | 6:40 AM → 7:30 AM | 8:30 AM → 10:40 AM | 2:05 PM | 5:10 PM
01-B | 6:20 AM → 7:40 AM | Not provided | Not provided | 5:15 PM

02-A | 6:20 AM → 7:30 AM | 8:20 AM → 10:20 AM | 2:05 PM | 5:10 PM
02-B | 6:35 AM → 7:30 AM | 8:35 AM → 10:30 AM | 2:05 PM | 5:10 PM

03-A | 6:35 AM → 7:30 AM | 8:40 AM → 10:30 AM | 2:05 PM | 5:10 PM
03-B | 7:40 AM → 8:45 AM | Not provided | Not provided | 5:15 PM

04 | 6:25 AM → 7:30 AM | 8:30 AM → 10:45 AM | 2:05 PM | 5:10 PM

05 | 6:30 AM → 7:40 AM | 8:30 AM → 10:30 AM | 2:10 PM | 5:15 PM

06-A | 6:30 AM → 7:35 AM | 8:40 AM → 10:45 AM | 2:10 PM | 5:10 PM
06-B | 6:35 AM → 7:30 AM | 8:30 AM → 10:30 AM | 2:05 PM | 5:15 PM

07 | 6:30 AM → 7:35 AM | Not provided | Not provided | 5:15 PM

08 | 7:00 AM → 7:40 AM | 9:00 AM → 10:35 AM | 2:05 PM | 5:10 PM
```

Do **not** invent missing trips.

---

16. BRACU stop-level data

These should go into `route_stops` and `timetable_stop_times`.

BRAC-R01-A

```
Abdullahpur — 6:40 / 8:30
House Building — 6:43 / 8:33
Azampur — 6:46 / 8:36
Jasimuddin — 6:49 / 8:39
Airport — 6:51 / 8:43
Kawla — 6:52 / 8:44
Khilket — 6:56 / 8:47
Biswa Road — 6:59 / 8:49
BRAC University — ARRIVAL 7:30 / 10:40
```

BRAC-R01-B

```
Jashimuddin – 6:20
Korotoa Courier Point – 6:22
Shahid Mughdho Corner – 6:23
Lake Drive Road – 6:25
Zam Zam Tower – 6:27
Tagore University Point – 6:29
Khalpar Mor – 6:30
Diabari Gol Chattar – 6:32
Uttara Centre MRT – 6:34
Mirpur Ceramics – 6:38
Mirpur DOHS Gate – 6:43
Pallabi Thana – 6:45
Kalshi Bus Stand – 6:48
ECB Chattar – 6:51
BRAC University – ARRIVAL 7:40
```

BRAC-R02-A

```
Mirpur Bangla College – 6:20 / 8:20
Dhaka Laboratory School & College – 6:22 / 8:22
Sony Cinema Hall – 6:24 / 8:25
Mirpur College Gate – 6:26 / 8:27
Swimming Federation Gate – 6:28 / 8:29
Popular Diagnostic Centre – 6:31 / 8:33
Pallabi Post Office – 6:34 / 8:37
Mirpur 11.5 – 6:35 / 8:38
Mirpur Ceramics – 6:36 / 8:40
Mirpur DOHS Gate – 6:40 / 8:45
Pallabi Thana – 6:43 / 8:47
Kalshi Bus Stand – 6:45 / 8:49
ECB Chattar – 6:48 / 8:52
BRAC University – ARRIVAL 7:30 / 10:20
```

BRAC-R02-B

```
Mirpur 14 Bus Stand – 6:35 / 8:35
NAM Garden Officers Quarter – 6:37 / 8:37
Water Tank Mirpur 10 – 6:39 / 8:40
Al Helal Specialized Hospital – 6:43 / 8:44
BRAC Healthcare Kazipara – 6:46 / 8:46
Shewrapara – 6:48 / 8:48
Taltola Bus Stand – 6:51 / 8:50
BRAC University – ARRIVAL 7:30 / 10:30
```

BRAC-R03-A

```
Jigatola Bus Stand – 6:35 / 8:40
Keari Plaza Dhanmondi 15 – 6:37 / 8:42
Shankar Bus Stand – 6:40 / 8:45
Meena Bazar Dhanmondi 27 – 6:42 / 8:47
Lalmatia Mother Care Hospital – 6:45 / 8:50
BRAC University – ARRIVAL 7:30 / 10:30
```

BRAC-R03-B

```
Jigatola Bus Stand – 7:40
Keari Plaza Dhanmondi 15 – 7:42
Shankar Bus Stand – 7:45
Meena Bazar Dhanmondi 27 – 7:47
Lalmatia Mother Care Hospital – 7:50
BRAC University – ARRIVAL 8:45
```

BRAC-R04

```
Azimpur Etimkhana Mour — 6:25 / 8:30
Azimpur Chowrasta — 6:27 / 8:33
New Market Gate 2 — 6:29 / 8:36
Happy Arcade Shopping Mall — 6:31 / 8:39
Dhanmondi Road 7 — 6:33 / 8:42
Kalabagan Krira Chokro — 6:35 / 8:44
Sobhanbag — 6:37 / 8:47
West End of Manik Mia Avenue — 6:39 / 8:51
Khejur Bagan Mour — 6:41 / 8:53
BRAC University — ARRIVAL 7:30 / 10:45
```

BRAC-R05

```
Doyaganj Mour — 6:30 / 8:30
Baldha Garden — 6:35 / 8:35
Ittefaq Mour — 6:38 / 8:38
Motijheel — 6:40 / 8:40
Arambag — 6:41 / 8:42
Fakirapool Mour — 6:43 / 8:44
Purana Paltan — 6:44 / 8:45
Kakrail Mour — 6:46 / 8:48
Shantinagar Mour — 6:48 / 8:50
Moghbazar T&T Office — 6:51 / 8:54
Moghbazar Mour — 6:55 / 9:00
BRAC University — ARRIVAL 7:40 / 10:30
```

BRAC-R06-A

```
Shiya Masjid — 6:30 / 8:40
Suchona Community Center — 6:32 / 8:42
Shampa Market — 6:34 / 8:45
Shyamoli — 6:37 / 8:48
Shyamoli Cinema Hall — 6:39 / 8:51
Agargaon Radio Office — 6:42 / 8:53
Agargaon Flower Shops — 6:44 / 8:55
BRAC University — ARRIVAL 7:35 / 10:45
```

BRAC-R06-B

```
Bata Showroom — 6:35 / 8:30
Baitus Sujud Jame Mosque — 6:40 / 8:36
Town Hall Market — 6:42 / 8:38
West End Manik Mia Avenue — 6:45 / 8:43
New Model Multilateral High School — 6:50 / 8:48
Water Bhaban — 6:53 / 8:53
BRAC University — ARRIVAL 7:30 / 10:30
```

BRAC-R07

```
Narayanganj Central Shahid Minar — 6:30
Himaloy Chinese & Community Centre — 6:34
Shibu Market — 6:37
Jalkuri Bus Stand — 6:40
Bhuighar — 6:43
Signboard — 6:48
Shamsul Hoque School & College — 6:51
Sanarpar Bus Stand — 6:54
Madaninagar Fazil Madrasah — 6:57
Khankaye Jame Masjid — 7:02
Rani Mahal Cinema Hall — 7:04
IFIC Bank Sarulia Bazar — 7:05
Demra Ideal College — 7:11
```

```
BRAC University — ARRIVAL 7:35
```

BRAC-R08

```
Ebenezer International School — 7:00 / 9:00
Block-F Road 08 Bashundhara R/A — 7:05 / 9:05
Aga Khan Academy Bashundhara — 7:10 / 9:10
BRAC University — ARRIVAL 7:40 / 10:35
```

17. NSU sample routes

These are **synthetic demo routes for your project**, not an official NSU timetable.

```
NSU-R01 | Uttara → NSU | Uttara | NSU | 100
NSU-R02 | Mirpur → NSU | Mirpur | NSU | 100
NSU-R03 | Dhanmondi → NSU | Dhanmondi | NSU | 80
NSU-R04 | Mohammadpur → NSU | Mohammadpur | NSU | 90
```

Sample timetable:

```
NSU-R01 | 6:30 → 7:30 | 8:30 → 9:30 | 2:30 PM | 5:15 PM
NSU-R02 | 6:35 → 7:30 | 8:35 → 9:30 | 2:30 PM | 5:15 PM
NSU-R03 | 6:45 → 7:30 | 8:45 → 9:30 | 2:30 PM | 5:15 PM
NSU-R04 | 6:40 → 7:30 | 8:40 → 9:30 | 2:30 PM | 5:15 PM
```

18. AIUB sample routes

Also synthetic demo data:

```
AIUB-R01 | Uttara Sector 10 → AIUB | Uttara | AIUB | 80
AIUB-R02 | Mirpur 10 → AIUB | Mirpur | AIUB | 100
AIUB-R03 | Dhanmondi 27 → AIUB | Dhanmondi | AIUB | 100
AIUB-R04 | Badda → AIUB | Badda | AIUB | 80
```

Sample timetable:

```
AIUB-R01 | 6:45 → 7:30 | 8:45 → 9:30 | 2:30 PM | 5:20 PM
AIUB-R02 | 6:30 → 7:30 | 8:30 → 9:30 | 2:30 PM | 5:20 PM
AIUB-R03 | 6:35 → 7:30 | 8:35 → 9:30 | 2:30 PM | 5:20 PM
AIUB-R04 | 6:50 → 7:30 | 8:50 → 9:30 | 2:30 PM | 5:20 PM
```

19. Bus-to-route assignment seed

For the initial demo, assign:

```
BRACU Student Bus 1001 → BRAC-R01-A
BRACU Student Bus 1002 → BRAC-R01-B
BRACU Student Bus 1003 → BRAC-R02-A
BRACU Student Bus 1004 → BRAC-R02-B
BRACU Student Bus 1005 → BRAC-R03-A
BRACU Student Bus 1006 → BRAC-R03-B
BRACU Student Bus 1007 → BRAC-R04
BRACU Student Bus 1008 → BRAC-R05
BRACU Student Bus 1009 → BRAC-R06-A
BRACU Student Bus 1010 → BRAC-R06-B
BRACU Student Bus 1011 → BRAC-R07
BRACU Student Bus 1012 → BRAC-R08
```

```
NSU 2001 → NSU-R01
NSU 2002 → NSU-R02
NSU 2003 → NSU-R03
NSU 2004 → NSU-R04

AIUB 3001 → AIUB-R01
AIUB 3002 → AIUB-R02
AIUB 3003 → AIUB-R03
AIUB 3004 → AIUB-R04
```

Again, PHP should work with database IDs, not these registration suffixes.

---

20. Actual schedules

Your recurring timetable and actual bookable schedule should be separate.

For testing, generate actual schedules for a date such as:

```
2026-08-20
```

Example:

```
BRAC-R01-A | 2026-08-20 | Bus 1001 | 06:40 | 07:30
BRAC-R02-A | 2026-08-20 | Bus 1003 | 06:20 | 07:30
BRAC-R03-A | 2026-08-20 | Bus 1005 | 06:35 | 07:30
BRAC-R04   | 2026-08-20 | Bus 1007 | 06:25 | 07:30
BRAC-R05   | 2026-08-20 | Bus 1008 | 06:30 | 07:40
BRAC-R06-A | 2026-08-20 | Bus 1009 | 06:30 | 07:35
BRAC-R07   | 2026-08-20 | Bus 1011 | 06:30 | 07:35
BRAC-R08   | 2026-08-20 | Bus 1012 | 07:00 | 07:40
```

You can then have the Admin generate schedules for new dates from timetable templates.

---

21. Sample favorite-route data

You only need some test rows:

```
BRACU-STU-0001 → BRAC-R01-A
BRACU-STU-0002 → BRAC-R02-A
BRACU-STU-0003 → BRAC-R03-A
BRACU-STU-0004 → BRAC-R08

NSU-STU-0001 → NSU-R01
NSU-STU-0002 → NSU-R02

AIUB-STU-0001 → AIUB-R01
AIUB-STU-0002 → AIUB-R02
```

The rest should be created when users click Favorite.

---

22. Sample complaints

```
BRACU-STU-0001
Subject: Bus arrived late
Description: The bus arrived later than the displayed pickup time.
Status: SUBMITTED
```

```
BRACU-STU-0015
Subject: Seat cleanliness
Description: Please improve cleaning before the morning trip.
Status: UNDER_REVIEW

NSU-STU-0001
Subject: Schedule information
Description: The passenger requested clearer schedule information.
Status: SUBMITTED

AIUB-STU-0001
Subject: Bus capacity
Description: The bus became crowded during the morning trip.
Status: SUBMITTED
```

These are demonstration complaints.

---

23. Sample notifications

```
BRACU-STU-0001
Title: Welcome to UniRide
Message: Your BRACU UniRide account is ready.
Type: SYSTEM
is_read: 0

NSU-STU-0001
Title: Welcome to UniRide
Message: Your NSU UniRide account is ready.
Type: SYSTEM
is_read: 0

AIUB-STU-0001
Title: Welcome to UniRide
Message: Your AIUB UniRide account is ready.
Type: SYSTEM
is_read: 0
```

Most notifications should be generated automatically.

---

24. Sample bookings

Do **not** manually fill dozens of bookings. Your booking system should create them.

For initial testing, create approximately six bookings through **`sp_create_booking()`**:

```
BRACU-STU-0001 → BRAC-R01-A → Seat A1
BRACU-STU-0002 → BRAC-R02-A → Seat A2
BRACU-STU-0003 → BRAC-R03-A → Seat B1

NSU-STU-0001 → NSU-R01 → Seat A1

AIUB-STU-0001 → AIUB-R01 → Seat A1
AIUB-STU-0002 → AIUB-R02 → Seat A2
```

The database should create:

```
booking_reference
fare
qr_token
semester billing
notification
booking history
```

automatically.

25. Seat data

You **do not need a separate 40-row seats table** with your current design unless you specifically want one.

Your seat numbers are:

```
1  = A1
2  = A2
3  = A3
4  = A4

5  = B1
6  = B2
7  = B3
8  = B4

9  = C1
...
40 = J4
```

Standing:

```
1 = Standing 01
2 = Standing 02
...
10 = Standing 10
```

26. Tables that should NOT be manually populated heavily

These tables should mostly be filled by your application:

```
bookings
booking_status_history
semester_bills
billing_transactions
ticket_transfers
notifications
```

The correct flow is:

```
User books
→ bookings row

Database trigger/procedure
→ booking_status_history
→ billing_transactions
→ semester_bills
→ notification
```

Similarly:

```
User requests transfer
→ ticket_transfers
```

and:

```
User cancels booking
→ bookings.status = CANCELLED
→ billing adjustment
→ notification
```

Do not pre-fill fake hundreds of transactions just to make the database look large.

---

27. Final table checklist for the whole website

Your completed database should contain these core tables:

```
admins
universities
university_users

passengers
students
faculty

buses
routes
route_stops
bus_route_assignments

timetable_templates
timetable_stop_times
schedules

bookings
booking_status_history

favorite_routes
complaints
ticket_transfers
notifications

semesters
semester_bills
billing_transactions
```

That gives you approximately:

```
3 universities
1 System Admin
3 University Admins

150 Students
15 Faculty
165 Passenger accounts total

26 demo buses

20 routes:
12 BRACU
4 NSU
4 AIUB

100+ BRACU route-stop records
Recurring timetable templates
Actual date-specific schedules

Semester billing
Booking
QR tickets
Favorites
Complaints
Notifications
```

```
Transfers
```

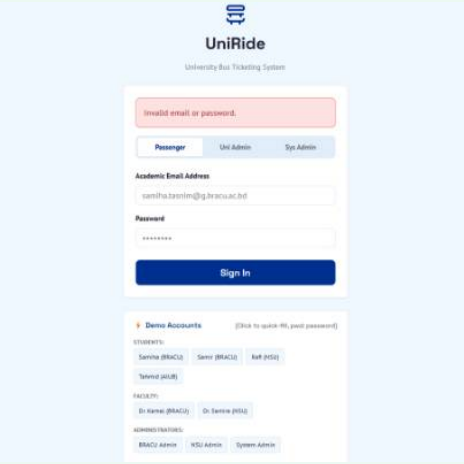
The important distinction is: **universities, users, passengers, buses, routes, stops, timetable templates and semesters are your seed/master data. Bookings, bills, transfers and most notifications are application-generated data.**

If you build it this way, your database will look substantial for the DBMS project without contaminating it with hundreds of arbitrary fake transactions.

> Attach all the data to a single downloadable file

Done — I combined all the manual UniRide database data into a single downloadable file.

Today 11:32 PM



> so remove the demo account section and the login feature for all 3 types people should support the updated database that you have just created. meaning every one including that database can successfully enter the website and also reset their password. make a professional prompt for my ai ageent for this
> Show moreShow less

Yes — the login screen should stop behaving like a demo and become a **real database-backed authentication system**. The demo-account panel should disappear completely, and any valid record in `passengers`, `university_users`, or `admins` should be able to authenticate using its own email and password.

Use this prompt with your AI agent:

REPLACE DEMO LOGIN WITH COMPLETE DATABASE-BACKED AUTHENTICATION

Update the existing **UniRide — Multi-University Bus Ticket Management System** login system.

Do **not** rebuild the project.

The current login page contains:

- Passenger tab
- University Admin tab
- System Admin tab
- Demo Accounts section
- Quick-fill demo-account buttons

Remove the entire **Demo Accounts** section and convert the login system into a real authentication system that works for **every valid account stored in the existing UniRide database**.

Existing stack:

```
HTML5
CSS3
Vanilla JavaScript
PHP
PDO
MySQL
phpMyAdmin
XAMPP
```

Existing database:

```
uniride_db
```

---

1. MAIN GOAL

Every valid account already stored in the database must be able to log into the website.

Authentication must support:

```
1. Students
2. Faculty
3. University Administrators
4. System Administrator
```

Students and Faculty are both stored as Passenger accounts.

The database contains approximately:

```
150 synthetic students
15 faculty members
3 university administrators
1 system administrator
```

plus any other valid records already present.

Do not hard-code a limited list of demo accounts.

If a new user is later added correctly to the database, that user should automatically be able to authenticate without editing PHP source code.

---

2. REMOVE THE DEMO ACCOUNT PANEL

Completely remove the section currently shown underneath the login form containing:

```
Demo Accounts
```

```
STUDENTS:
Samiha (BRACU)
Samir (BRACU)
Rafi (NSU)
Tahmid (AIUB)

FACULTY:
Dr Kamal (BRACU)
Dr Samira (NSU)

ADMINISTRATORS:
BRACU Admin
NSU Admin
System Admin
```

Remove:

- Demo-account cards
- Quick-fill buttons
- "Click to quick-fill"
- Hard-coded demo email JavaScript
- Hard-coded demo passwords
- Any hidden demo-login shortcuts

The login page should look like a genuine production-style authentication page.

---

3. DATABASE TABLES USED FOR AUTHENTICATION

Use the existing database tables.

Passenger Authentication

Authenticate Students and Faculty using:

```
passengers
```

Relevant fields include:

```
passenger_id
university_id
passenger_code
name
email
password_hash
passenger_type
status
```

Possible passenger types:

```
STUDENT
FACULTY
```

Students and Faculty should **not** need separate login tables.

University Administrator Authentication

Authenticate University Administrators using:

```
university_users
```

Relevant fields:

```
university_user_id
university_id
name
email
password_hash
role
status
```

System Administrator Authentication

Authenticate System Admin using:

```
admins
```

Relevant fields:

```
admin_id
admin_code
name
email
password_hash
status
```

---

4. DO NOT HARDCODE USERS

Delete authentication logic such as:

```
if (
    $email === 'samiha.tasnim@g.bracu.ac.bd'
    && $password === 'password'
) {
    // login
}
```

Never maintain arrays such as:

```
$demoUsers = [
   ...
];
```

Never authenticate from:

- JavaScript arrays
- JSON demo data
- HTML values
- LocalStorage
- Session-only demo accounts

Authentication must query MySQL.

---

5. LOGIN PAGE DESIGN

Keep the professional UniRide branding:

```
UniRide
University Bus Ticketing System
```

Keep the clean centered login card.

Remove the Demo Accounts card entirely.

The page should contain:

```
UniRide logo

UniRide
University Bus Ticketing System

[ Passenger ] [ Uni Admin ] [ Sys Admin ]

Academic Email Address

Password

[ Sign In ]

Forgot Password?
```

Optional:

```
Remember Me
```

Do not add unnecessary public registration because university/passenger accounts are managed through the database and university system.

---

6. PASSENGER LOGIN

When:

```
Passenger
```

is selected, authenticate against:

```
passengers
```

Use a query equivalent to:

```
SELECT
    passenger_id,
    university_id,
    passenger_code,
    name,
    email,
    password_hash,
    passenger_type,
    status

FROM passengers

WHERE email = ?
```

```
LIMIT 1;
```

Use PDO prepared statements.

---

7. PASSENGER PASSWORD CHECK

Use:

```
password_verify(
    $submittedPassword,
    $user['password_hash']
);
```

Never use:

```
$submittedPassword === $user['password_hash'];
```

Never store passwords as plain text.

---

8. PASSENGER STATUS CHECK

Only permit login if:

```
status = ACTIVE
```

If account status is:

```
INACTIVE
SUSPENDED
```

deny access.

Display a clean message such as:

```
Your account is currently unavailable.
Please contact your university transport administrator.
```

Do not reveal unnecessary security details.

---

9. STUDENT VS FACULTY LOGIN

Both Student and Faculty use the Passenger login tab.

After authentication, use:

```
passenger_type
```

to determine whether the user is:

```
STUDENT
```

or:

```
FACULTY
```

Store this information in the PHP session.

Example:

```
$_SESSION['user_type'] = 'PASSENGER';
$_SESSION['passenger_type'] = $user['passenger_type'];
```

Both may use the passenger dashboard, while existing Student/Faculty eligibility rules remain active.

---

10. UNIVERSITY ADMIN LOGIN

When:

```
Uni Admin
```

is selected, authenticate against:

```
university_users
```

Example query:

```
SELECT
    university_user_id,
    university_id,
    name,
    email,
    password_hash,
    role,
    status

FROM university_users

WHERE email = ?

LIMIT 1;
```

Verify password with:

```
password_verify();
```

Require:

```
status = ACTIVE
```

---

11. SYSTEM ADMIN LOGIN

When:

```
Sys Admin
```

is selected, authenticate against:

```
admins
```

Example:

```
SELECT
    admin_id,
    admin_code,
    name,
    email,
    password_hash,
    status
```

```
FROM admins

WHERE email = ?

LIMIT 1;
```

Use:

```
password_verify();
```

Require:

```
status = ACTIVE
```

---

12. SESSION STRUCTURE

After successful Passenger authentication, store:

```
$_SESSION['authenticated'] = true;
$_SESSION['user_type'] = 'PASSENGER';

$_SESSION['passenger_id'];
$_SESSION['user_id'];

$_SESSION['university_id'];
$_SESSION['passenger_type'];

$_SESSION['name'];
$_SESSION['email'];
```

For University Admin:

```
$_SESSION['authenticated'] = true;
$_SESSION['user_type'] = 'UNIVERSITY_ADMIN';

$_SESSION['user_id'];
$_SESSION['university_user_id'];
$_SESSION['university_id'];

$_SESSION['name'];
$_SESSION['email'];
$_SESSION['role'];
```

For System Admin:

```
$_SESSION['authenticated'] = true;
$_SESSION['user_type'] = 'SYSTEM_ADMIN';

$_SESSION['user_id'];
$_SESSION['admin_id'];

$_SESSION['name'];
$_SESSION['email'];
```

Do not create confusing or contradictory session values.

---

13. SESSION SECURITY

After successful login run:

```
session_regenerate_id(true);
```

Use PHP sessions securely.

Protect authenticated pages.

Passenger URLs must reject Admin users where appropriate.

University Admin URLs must reject:

```
Passenger
System Admin
```

unless specifically allowed.

System Admin URLs must reject everyone except:

```
SYSTEM_ADMIN
```

---

14. REDIRECT AFTER LOGIN

After successful Passenger login:

```
/passenger/dashboard.php
```

After successful University Admin login:

```
/university/dashboard.php
```

After successful System Admin login:

```
/admin/dashboard.php
```

Do not redirect all account types to the same dashboard.

---

15. UPDATED DATABASE ACCOUNTS MUST ALL WORK

The login system must work with the complete current database.

For example, accounts in the database may include formats such as:

BRACU Students

```
samir.hossain@g.bracu.ac.bd
nafisa.ali@g.bracu.ac.bd
tahmid.azad@g.bracu.ac.bd
...
```

NSU Students

```
saif.sultana@northsouth.edu
rukaiya.siddique@northsouth.edu
...
```

AIUB Students

```
26-90001-1@student.aiub.edu
26-90002-1@student.aiub.edu
...
```

Faculty

Examples:

```
kamal.uddin@g.bracu.ac.bd
reazaul/appropriate NSU faculty account
farhana.noor@aiub.edu
```

University Administrators

Examples:

```
bracu.transport@gmail.com
nsu.transport@gmail.com
aiub.transport@gmail.com
```

System Administrator

```
uniride.admin@gmail.com
```

These are examples only.

PHP must authenticate whatever valid records currently exist in the database.

---

16. CRITICAL PASSWORD HASH MIGRATION

Inspect all existing:

```
admins
university_users
passengers
```

records.

Every account must have a valid PHP-compatible password hash.

The development/demo password may currently be:

```
password
```

for seeded accounts.

However, `password_hash` must contain a hash generated with:

```
password_hash(
    'password',
    PASSWORD_DEFAULT
);
```

Do not store:

```
password
```

directly in `password_hash`.

---

17. VERIFY EXISTING HASHES

Write a temporary secure migration/helper if necessary to ensure all seeded accounts have usable password hashes.

Do not generate fake bcrypt strings manually.

Use PHP to generate valid hashes.

For example, create a temporary CLI migration script that performs:

```
$hash = password_hash(
    'password',
    PASSWORD_DEFAULT
);
```

and updates accounts that need migration.

After migration, remove or protect the migration script.

---

18. DO NOT RESET USER PASSWORDS UNNECESSARILY

Before changing an existing password hash, determine whether it is already a valid usable hash.

Do not overwrite real user passwords indiscriminately.

For the synthetic development accounts generated specifically for UniRide, it is acceptable to initialize them with the development password:

```
password
```

stored securely as a hash.

---

19. ERROR MESSAGES

Do not reveal whether an email exists.

For invalid credentials display:

```
Invalid email or password.
```

Use the same response for:

```
unknown email
wrong password
```

This prevents account enumeration.

For inactive accounts, a generic account-status message may be displayed after authentication checks.

---

20. LOGIN RATE LIMITING

Add basic protection against repeated login attempts.

Recommended:

```
Maximum attempts: 5
Temporary lock/cooldown: 10–15 minutes
```

This can be session-based for the university project or database-backed for a stronger implementation.

Do not permanently lock demo users accidentally.

---

21. NORMALIZE EMAIL INPUT

Before lookup:

```
$email = strtolower(
    trim($_POST['email'])
);
```

Validate with:

```
filter_var(
    $email,
    FILTER_VALIDATE_EMAIL
);
```

Do not alter the password string before `password_verify()`.

---

22. PASSWORD VISIBILITY BUTTON

Add a professional:

Show / Hide Password

icon/button.

Implement using JavaScript only for UI behavior.

Do not expose password values elsewhere.

---

23. FORGOT PASSWORD LINK

Add:

Forgot Password?

under the password field.

This feature must work for:

Passenger
University Admin
System Admin

---

24. PASSWORD RESET DATABASE TABLE

Create a secure table such as:

password_reset_tokens

Recommended structure:

```
CREATE TABLE password_reset_tokens (
    reset_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,

    account_type ENUM(
        'PASSENGER',
        'UNIVERSITY_ADMIN',
        'SYSTEM_ADMIN'
    ) NOT NULL,

    account_id BIGINT UNSIGNED NOT NULL,

    token_hash CHAR(64) NOT NULL,

    expires_at DATETIME NOT NULL,

    used_at DATETIME NULL,

    created_at TIMESTAMP
        NOT NULL
        DEFAULT CURRENT_TIMESTAMP,

    ip_address VARCHAR(45) NULL,

    UNIQUE KEY uq_reset_token_hash (token_hash),

    KEY idx_reset_account (
        account_type,
        account_id
    ),

    KEY idx_reset_expiration (
        expires_at
    )
);
```

Adapt names if necessary.

Because the reset table can reference three different account tables, do not create an invalid polymorphic foreign key.

Validate account ownership in PHP.

---

25. FORGOT PASSWORD WORKFLOW

Create:

```
forgot-password.php
```

The user chooses or retains account type:

```
Passenger
University Admin
System Admin
```

and enters their email.

Workflow:

```
User enters email
        ↓
PHP searches correct account table
        ↓
If valid active account exists
        ↓
Generate cryptographically secure reset token
        ↓
Store only hashed token in MySQL
        ↓
```

```
Create reset URL
       ↓
Send or provide reset process
```

26. DO NOT REVEAL WHETHER EMAIL EXISTS

After password-reset request always display:

```
If an account exists for this email,
password reset instructions have been generated.
```

Do not display:

```
Email not found
```

to anonymous users.

27. RESET TOKEN GENERATION

Use PHP:

```
$token = bin2hex(
    random_bytes(32)
);
```

Do not use:

```
rand()
mt_rand()
uniqid()
```

for password-reset security.

Store only the hash:

```
$tokenHash = hash(
    'sha256',
    $token
);
```

Store:

```
token_hash
```

in MySQL.

Send/use the original token only in the reset URL.

28. RESET TOKEN EXPIRATION

Reset links should expire after approximately:

```
30 minutes
```

or:

```
60 minutes
```

Recommended:

```
30 minutes
```

Reject:

- expired token
- unknown token
- already-used token

---

29. PASSWORD RESET URL

Example:

```
http://localhost/uniride/reset-password.php?token=<token>&type=PASSENGER
```

Do not place:

```
password
account password hash
database password
```

inside the URL.

---

30. LOCAL XAMPP DEVELOPMENT MODE

This project runs locally in XAMPP and may not have a configured SMTP server.

Therefore implement password reset in two modes.

Production / Email Mode

If SMTP/email configuration exists:

Send the password-reset link to the account's email.

Local Development Mode

If:

```
APP_ENV=local
```

and SMTP is unavailable:

Generate the reset token normally and store its hash exactly as production would.

Then display the generated reset URL on a clearly labeled local-development page or write it to:

```
storage/logs/password_reset.log
```

Do not bypass the token-security workflow.

Never automatically reset the password without token verification.

Do not expose local reset links when:

```
APP_ENV=production
```

---

31. RESET PASSWORD PAGE

Create:

```
reset-password.php
```

Fields:

```
New Password
Confirm New Password

[ Reset Password ]
```

Validate:

- token exists
- token hash matches
- token not expired
- token not already used
- account exists
- account is eligible

---

32. PASSWORD REQUIREMENTS

Use sensible minimum password requirements.

Recommended:

```
Minimum 8 characters
```

Optionally require:

```
uppercase
lowercase
number
```

Do not make the university demonstration unnecessarily difficult.

Display a password-strength hint.

---

33. NEW PASSWORD HASHING

After validation:

```
$newHash = password_hash(
    $newPassword,
    PASSWORD_DEFAULT
);
```

Update the proper table depending on account type.

Passenger

```
UPDATE passengers
SET password_hash = ?
WHERE passenger_id = ?;
```

University Admin

```
UPDATE university_users
SET password_hash = ?
WHERE university_user_id = ?;
```

System Admin

```
UPDATE admins
SET password_hash = ?
WHERE admin_id = ?;
```

Use prepared statements.

---

34. MARK RESET TOKEN AS USED

After successful password reset:

```
UPDATE password_reset_tokens
SET used_at = NOW()
WHERE reset_id = ?;
```

A token may only reset a password once.

---

35. INVALIDATE OTHER RESET TOKENS

After successful password reset, invalidate/delete all other unused reset tokens belonging to that account.

This prevents older reset links from remaining usable.

---

36. PASSWORD CHANGE FROM PROFILE

The existing authenticated user profile should also support:

```
Change Password
```

This is different from Forgot Password.

Authenticated workflow:

```
Current Password
New Password
```

```
Confirm New Password
```

Verify current password with:

```
password_verify()
```

before updating.

This should work for:

```
Passenger
University Admin
System Admin
```

if profile pages exist for all roles.

---

37. PASSWORD RESET SHOULD NOT CHANGE ROLE

Resetting a password must never modify:

```
passenger_type
university_id
role
account ID
email
```

Only update:

```
password_hash
```

and reset-token records.

---

38. DATABASE TRANSACTION

Perform the actual password update and token invalidation inside a transaction.

Example flow:

```
START TRANSACTION

validate token
update password_hash
mark current token used
invalidate other reset tokens

COMMIT
```

Rollback on failure.

---

39. LOGOUT

Create/fix:

```
logout.php
```

It should:

```
session_start();
$_SESSION = [];
session_destroy();
```

Then redirect to:

```
/login.php
```

Do not retain authenticated role information after logout.

---

40. AUTHORIZATION HELPERS

Create centralized helpers such as:

```
includes/auth.php
```

Recommended functions:

```
requireLogin();
```

```
requirePassenger();
```

```
requireUniversityAdmin();
```

```
requireSystemAdmin();
```

```
currentUser();
```

```
currentUniversityId();
```

Do not repeat insecure authorization code across every PHP page.

---

41. MULTI-UNIVERSITY SECURITY

After University Admin login:

```
$_SESSION['university_id']
```

must control all university-specific queries.

University Admin must not be able to change URL parameters to access another university.

Example:

Correct:

```
SELECT *
FROM buses
WHERE bus_id = ?
  AND university_id = ?;
```

where the second value comes from:

```
$_SESSION['university_id']
```

---

42. PASSENGER UNIVERSITY SECURITY

Passenger data access must use:

```
$_SESSION['passenger_id']
$_SESSION['university_id']
```

Do not trust:

```
?passenger_id=123
```

to determine ownership.

---

43. LOGIN PAGE TAB BEHAVIOR

The existing three tabs may remain:

```
Passenger
Uni Admin
Sys Admin
```

When the tab changes, JavaScript may set:

```
account_type
```

but the actual authentication must happen in PHP.

Do not authenticate inside JavaScript.

---

44. OPTIONAL BETTER UX

If desired, the tabs can be replaced with a polished role selector.

However, do not combine tables using insecure guesswork.

The selected role should determine which authentication table PHP queries.

---

45. LOGIN FORM CSRF

Add a CSRF token to the login form if the existing security framework supports it.

Definitely use CSRF protection for:

```
password reset
password change
logout POST
profile changes
```

where appropriate.

---

46. SQL INJECTION PROTECTION

All authentication queries must use:

```
PDO prepared statements
```

Never concatenate submitted email into SQL.

Incorrect:

```
$sql =
"SELECT * FROM passengers
 WHERE email='$email'";
```

Correct:

```
$stmt = $pdo->prepare(
    "SELECT *
     FROM passengers
     WHERE email = ?
     LIMIT 1"
);

$stmt->execute([$email]);
```

---

47. XSS PROTECTION

When displaying:

```
name
email
university
```

escape output using:

```
htmlspecialchars(
    $value,
    ENT_QUOTES,
    'UTF-8'
);
```

---

48. UPDATE DATABASE INSTALLATION FILE

Update:

```
database/uniride_complete_phpmyadmin.sql
```

to include:

```
password_reset_tokens
```

and any indexes required for authentication/reset.

A fresh phpMyAdmin import must contain the complete final authentication schema.

---

49. UPDATE EXISTING STUDENT SEED DATA

The complete student dataset must remain compatible with authentication.

Verify all generated BRACU accounts such as:

```
BRACU-STU-0001
...
BRACU-STU-0050
```

have valid:

```
email
password_hash
```

```
status = ACTIVE
passenger_type = STUDENT
```

---

50. UPDATE NSU STUDENT SEED DATA

Verify:

```
NSU-STU-0001
...
NSU-STU-0050
```

can authenticate.

---

51. UPDATE AIUB STUDENT SEED DATA

Verify:

```
AIUB-STU-0001
...
AIUB-STU-0050
```

can authenticate.

AIUB emails such as:

```
26-90001-1@student.aiub.edu
```

must work normally with the same login system.

---

52. VERIFY FACULTY LOGIN

Verify all Faculty rows stored in:

```
passengers
```

with:

```
passenger_type = FACULTY
```

can authenticate from the Passenger tab.

Test at least:

```
5 BRACU Faculty
5 NSU Faculty
5 AIUB Faculty
```

or all Faculty records where practical.

---

53. VERIFY UNIVERSITY ADMIN LOGIN

Test every University Admin:

```
BRACU Admin
NSU Admin
AIUB Admin
```

All must authenticate using records from:

```
university_users
```

---

54. VERIFY SYSTEM ADMIN LOGIN

Test the System Admin account from:

```
admins
```

No hard-coded exception should be required.

---

55. AUTOMATED AUTHENTICATION TEST SCRIPT

Create a local development test script or automated test suite that checks seeded accounts.

Do not expose passwords publicly.

For each account category, verify:

```
database record exists
email is valid
password_hash is valid
password_verify('password', hash) succeeds
status is ACTIVE
correct role is loaded
correct dashboard redirect occurs
```

---

56. DATABASE VERIFICATION

Run queries similar to:

```
SELECT
    passenger_type,
    COUNT(*) AS total
FROM passengers
WHERE status = 'ACTIVE'
GROUP BY passenger_type;
```

Verify the expected Student/Faculty populations.

Also:

```
SELECT COUNT(*)
FROM university_users
WHERE status = 'ACTIVE';
```

and:

```
SELECT COUNT(*)
FROM admins
WHERE status = 'ACTIVE';
```

---

57. IDENTIFY BAD PASSWORD HASHES

Create a PHP verification utility during migration.

For every seeded account, run an equivalent:

```
password_get_info(
    $row['password_hash']
);
```

If the record contains a plain-text placeholder rather than a recognized password hash, migrate it securely.

Do not leave accounts that look correct in MySQL but can never authenticate.

---

58. REMOVE OLD LOGIN CREDENTIALS FROM SOURCE CODE

Search the complete project for obsolete hard-coded demo credentials.

Search terms should include:

```
Demo Accounts
quick-fill
samiha.tasnim
Samir (BRACU)
Rafi (NSU)
Tahmid (AIUB)
password
admin@uniride.demo
mcu.admin
```

Do not blindly remove every occurrence of the word `password`; inspect each result.

Remove only obsolete demo-account UI/logic.

---

59. README UPDATE

Remove the large quick-login/demo section if it suggests only a few special users can authenticate.

Replace it with:

```
Authentication

UniRide uses database-backed authentication.

Students and Faculty authenticate through the Passenger login.

University transport administrators authenticate through
the University Admin login.

The central platform administrator authenticates through
the System Admin login.

Any valid ACTIVE account stored in the corresponding
database table can authenticate.
```

For development documentation, you may provide one optional test account per role, but do not show them on the public login page.

---

60. PASSWORD RESET README

Document:

```
Forgot Password
```

behavior.

Explain local development mode if SMTP is unavailable.

Example:

```
In local XAMPP development, reset links are written
to the configured development reset log.

In production, configure SMTP/email delivery.
```

---

61. UI AFTER REMOVING DEMO ACCOUNTS

The login screen should be visually balanced after deleting the bottom card.

Do not leave a large blank area.

Center the authentication card vertically and horizontally.

Suggested structure:

```
          Bus Icon

          UniRide
 University Bus Ticketing System

  ┌──────────────────────────────┐
  │ Passenger Uni Admin Sys Admin│
  │                              │
  │ Academic Email Address       │
  │ [____________________________]│
  │                              │
  │ Password              Show   │
  │ [____________________________]│
  │                              │
  │ Forgot Password?             │
  │                              │
  │        [ Sign In ]           │
  └──────────────────────────────┘
```

Keep the existing professional blue UniRide visual identity.

---

62. FORGOT PASSWORD PAGE UI

Use the same branding.

Example:

```
UniRide

Reset your password

Choose account type:
Passenger / University Admin / System Admin

Email Address
[_____________________]

[ Send Reset Instructions ]

Back to Sign In
```

Do not display demo-account information.

63. RESET PASSWORD PAGE UI

Example:

```
UniRide

Create New Password

New Password
[_________________]

Confirm New Password
[_________________]

[ Reset Password ]

Back to Sign In
```

After success:

```
Your password has been reset successfully.
You can now sign in.
```

Provide:

```
[ Return to Sign In ]
```

64. IMPORTANT DATABASE BEHAVIOR

Authentication must follow:

```
Login form
    ↓
PHP
    ↓
PDO prepared query
    ↓
Correct database table
    ↓
Find account
    ↓
Verify ACTIVE status
    ↓
password_verify()
    ↓
Create secure session
    ↓
Correct role dashboard
```

Password reset:

```
Forgot Password
    ↓
Lookup account
    ↓
Generate secure token
    ↓
Store hashed token
    ↓
Reset link
    ↓
Validate token
    ↓
```

```
password_hash(new password)
    ↓
Update correct account table
    ↓
Invalidate token
    ↓
Return to login
```

---

65. DO NOT BREAK EXISTING FEATURES

Preserve all current UniRide functionality:

- BRACU routes
- BRACU stop-level timetable
- NSU routes
- AIUB routes
- Schedules
- Seat booking
- 40-seat interface
- 10 standing slots
- QR tickets
- Booking history
- Cancellation
- Ticket transfers
- Student ticket sharing/selling
- Favorite routes
- Complaints
- Notifications
- Semester billing
- Profile management
- Multi-university isolation
- University management
- Admin dashboard

This task is primarily an **authentication and password-management upgrade**.

---

66. REQUIRED TEST CASES

Before declaring the feature complete, test:

Student Authentication

Test several students from:

```
BRACU
NSU
AIUB
```

Confirm each reaches their correct Passenger dashboard.

Faculty Authentication

Test Faculty from all three universities.

Confirm they authenticate from:

```
Passenger
```

and are recognized as:

```
FACULTY
```

University Admin Authentication

Test:

```
BRACU University Admin
NSU University Admin
AIUB University Admin
```

System Admin

Test System Admin.

Wrong Password

Valid email + wrong password:

```
Invalid email or password.
```

Unknown Email

Unknown email:

```
Invalid email or password.
```

Do not reveal:

```
User not found
```

Inactive Account

Confirm inactive users are denied access.

Cross-Role Login

Passenger email entered under Uni Admin:

must fail.

University Admin entered under Passenger:

must fail.

System Admin entered under Passenger:

must fail.

---

67. PASSWORD RESET TEST CASES

Test password reset for:

```
1 BRACU Student
1 NSU Student
1 AIUB Student
1 Faculty
1 University Admin
1 System Admin
```

Verify:

```
token created
token stored hashed
token expires
new password is hashed
old password stops working
new password works
token cannot be reused
```

---

68. CRITICAL LIVE DATABASE TEST

Do not just test against temporary dummy records.

Use the actual current local XAMPP database:

```
uniride_db
```

Verify users from the populated database can log in.

The final result must prove that authentication is truly database-driven.

---

69. UPDATE MASTER DATABASE AND MIGRATIONS

If the live database is changed, create a migration such as:

```
database/migrations/
007_complete_authentication_and_password_reset.sql
```

Also update:

```
database/uniride_complete_phpmyadmin.sql
```

so a fresh install reproduces the final database structure.

Do not make changes only to the live phpMyAdmin database.

---

70. FINAL ACCEPTANCE REQUIREMENTS

The task is complete only when all of these are true:

- ✓ **Demo Accounts UI completely removed**
- ✓ **Quick-fill buttons removed**
- ✓ **No hard-coded demo authentication**
- ✓ **Passenger login reads passengers table**
- ✓ **All Students can authenticate**
- ✓ **All Faculty can authenticate**
- ✓ **University Admin login reads university_users**
- ✓ **All University Admins can authenticate**
- ✓ **System Admin login reads admins**
- ✓ **System Admin can authenticate**
- ✓ **password_verify() is used**
- ✓ **Valid PHP password hashes exist**
- ✓ **Secure PHP sessions work**
- ✓ **Correct dashboard redirect works**
- ✓ **Role authorization works**
- ✓ **University isolation works**
- ✓ **Forgot Password works**
- ✓ **Secure reset tokens work**
- ✓ **Reset tokens expire**
- ✓ **Reset tokens cannot be reused**
- ✓ **Passenger password reset works**
- ✓ **Faculty password reset works**
- ✓ **University Admin password reset works**
- ✓ **System Admin password reset works**
- ✓ **Change Password works for authenticated users**
- ✓ **No plain-text passwords stored**
- ✓ **SQL injection protection works**

✓ **Login error messages do not reveal account existence**

✓ **Database master SQL updated**

✓ **Migration saved**

✓ **README updated**

✓ **Existing UniRide features remain functional**

FINAL COMMAND TO THE AI CODING AGENT

Do not merely redesign the login screen.

Actually connect authentication to the complete existing **`uniride_db`** database.

Every valid ACTIVE account in:

```
passengers
university_users
admins
```

must be able to log into UniRide with its database credentials.

Students and Faculty must authenticate through the Passenger login.

University Administrators must authenticate through the Uni Admin login.

System Administrator must authenticate through the Sys Admin login.

Remove the Demo Accounts section entirely.

Implement a secure Forgot Password / Reset Password workflow for all three account categories.

Do not hard-code special users.

Do not fake login with JavaScript.

Do not store plain-text passwords.

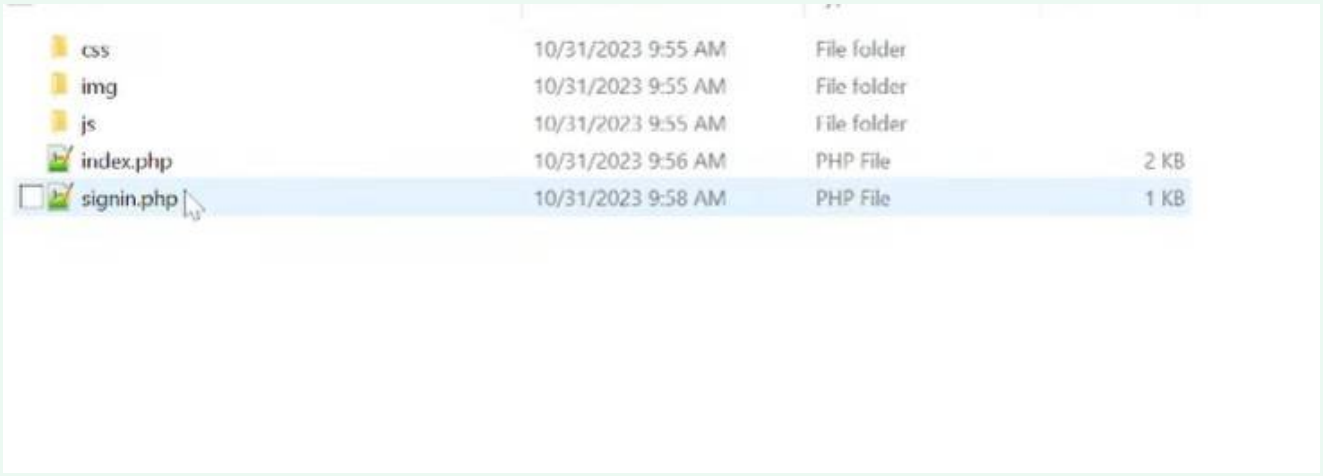
Do not bypass database authentication.

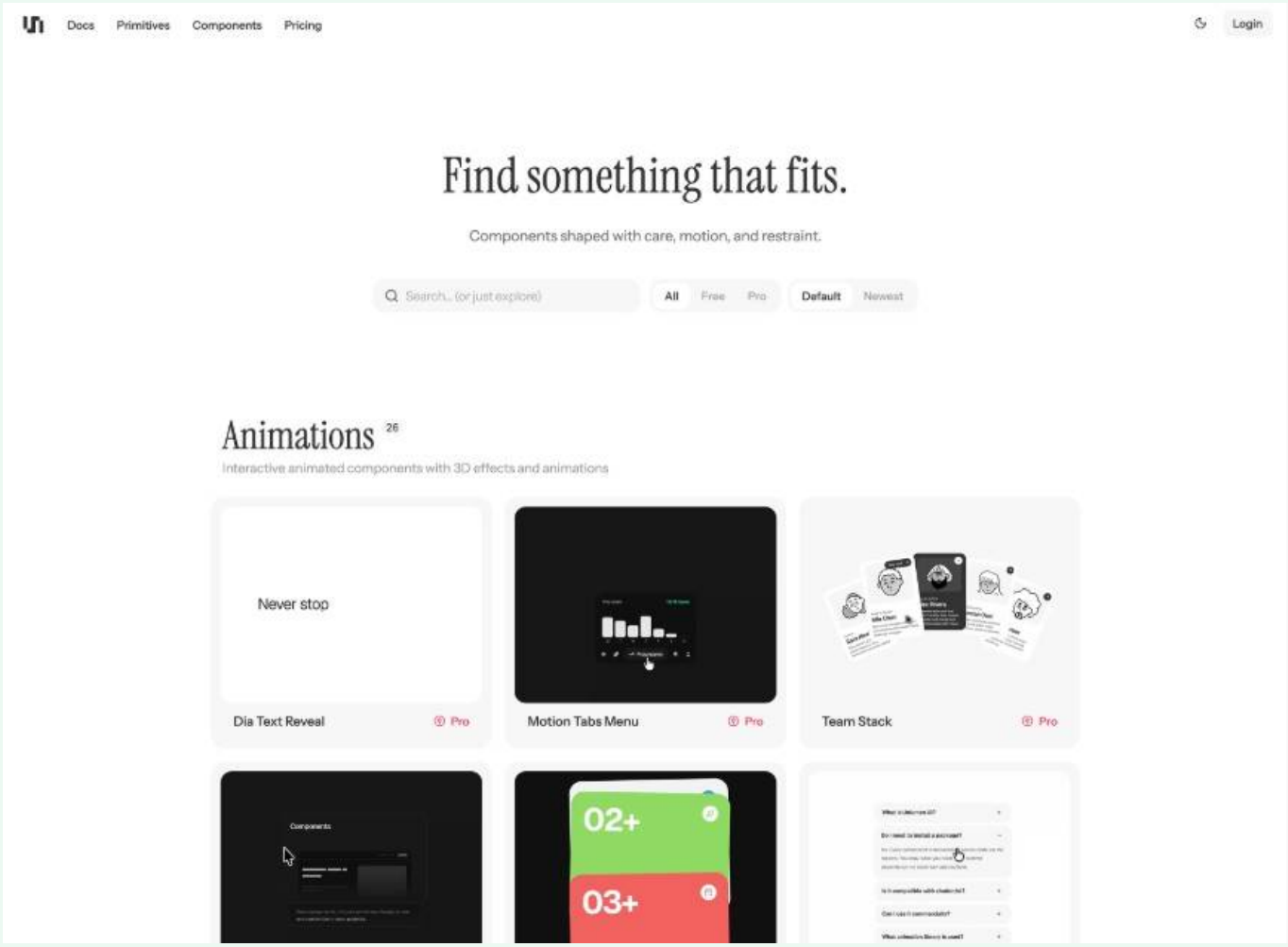
Use:

```
HTML/CSS/JavaScript
        ↓
PHP
        ↓
PDO prepared statements
        ↓
uniride_db
        ↓
password_verify()
        ↓
secure PHP session
        ↓
role-specific dashboard
```

The final authentication system must automatically support both the **current populated database and future users added to the database**, without requiring PHP source-code changes.

Today 1:49 AM





according to all your experience from this project till now,
give me a html and css file and other necessary file shower in the "image 1" for me. use "ui" image for your css inspiration

I created a complete drop-in starter based on the structure in your first image and the minimalist visual language from the UI reference.

It includes the same core folders/files you showed — **`css/`**, **`img/`**, **`js/`**, **`index.php`**, **`signin.php`** — plus the few pieces needed to make it actually useful with your UniRide project: **`config/database.php`**, **`logout.php`**, **`forgot-password.php`**, and a README.

The structure is:

```
uniride_ui_starter/
├── css/
│   └── style.css
├── img/
│   └── logo.svg
├── js/
│   └── app.js
├── config/
│   └── database.php
├── index.php
├── signin.php
├── forgot-password.php
├── logout.php
└── README.md
```

The design follows the UI screenshot's approach: lots of whitespace, large editorial-style headings, compact navigation, pale gray surfaces, subtle rounded cards, thin borders, understated controls, and minimal shadows. I kept UniRide's blue mostly as a restrained brand accent rather than making the entire interface blue.

Importantly, **`signin.php`** is **not a fake demo login**. It already authenticates against your existing **`uniride_db`**:

```
Passenger
→ passengers

Uni Admin
→ university_users

Sys Admin
→ admins
```

It uses PDO prepared statements, **`password_verify()`**, PHP sessions, CSRF protection, active-account checks, and redirects to:

```
passenger/dashboard.php
university/dashboard.php
admin/dashboard.php
```

The landing page also tries to load actual active routes directly from **`uniride_db`**, so once XAMPP/MySQL is running, your route cards can come from the database rather than hard-coded HTML.

One thing to note: **`forgot-password.php`** currently contains the finished UI shell but intentionally does **not** fake the reset process. It should be connected to the **`password_reset_tokens`** workflow we designed earlier.

> **UniRide_fixed_database(1).sql**
> File
> the website login is not working. integrate features so that it login will work on the given data. and give me the code file to dowload and tell me which file should i replace

I found the login problem and built a replacement package specifically for your uploaded database.

Your database is actually named **`bus_reservation_system`**, while the earlier code connected to **`uniride_db`**. There is also an important schema difference: **`passengers`** uses **`password_hash`**, **`university_users`** uses **`password_hash`**, but **System Admin uses a column**

UniRide_fixed_database(1) UniRide_fixed_database(1)

called `password`. The old login code therefore could not authenticate System Admin correctly.

Download the complete fix

The package contains:

```
uniride_login_fix/
│
├── signin.php
├── logout.php
├── forgot-password.php
├── reset-password.php
├── login-diagnostic.php
│
├── config/
│   └── database.php
│
├── database/
│   └── authentication_patch.sql
│
└── README_REPLACE_FILES.md
```

Replace these files

Inside:

```
C:\xampp\htdocs\uniride\
```

replace your current files as follows:

```
OLD / CURRENT                         REPLACE WITH

signin.php
→ signin.php

config/database.php
→ config/database.php

logout.php
→ logout.php

forgot-password.php
→ forgot-password.php
```

Then **add** this new file:

```
reset-password.php
```

You do **not** need to replace:

```
index.php
css/style.css
js/app.js
booking files
route files
schedule files
dashboard files
```

unless those have separate problems.

One SQL file must be imported

After replacing the PHP files, open:

```
http://localhost/phpmyadmin
```

Select:

```
bus_reservation_system
```

and import/run:

That only adds the password-reset-token table. It **does not delete or replace** your students, faculty, buses, routes, schedules, bookings, complaints, or other data.

Your database connection is now correct

The new **`config/database.php`** uses:

```php
PHP
$dbHost = '127.0.0.1';
$dbPort = '3306';
$dbName = 'bus_reservation_system';
$dbUser = 'root';
$dbPass = '';
```

This matches your uploaded SQL database.

Login now supports the whole populated database

The new login does **not** contain hard-coded users.

It works like:

```
Passenger tab
    ↓
passengers table
    ↓
Students + Faculty

Uni Admin tab
    ↓
university_users table

Sys Admin tab
    ↓
admins table
```

Every **`ACTIVE`** account in those tables can authenticate.

I also verified that both password hashes used by your uploaded seed data accept the development password:

```
password
```

So, for example:

```
Passenger
```

```
Email: samiha.tasnim@g.bracu.ac.bd
Password: password
```

or:

```
Passenger
Email: samir.hossain@g.bracu.ac.bd
Password: password
```

Your SQL contains these passenger records with hashed passwords. UniRide_fixed_database(1)

University Admin:

```
Email: bracu.transport@gmail.com
Password: password
```

The BRACU, NSU, and AIUB admin accounts are all present in your database. UniRide_fixed_database(1)

System Admin:

```
Email: uniride.admin@gmail.com
Password: password
```

That account is also present in the uploaded SQL. UniRide_fixed_database(1)

Test the database before logging in

I included:

Temporarily put it here:

```
C:\xampp\htdocs\uniride\login-diagnostic.php
```

Then visit:

```
http://localhost/uniride/login-diagnostic.php
```

You should see something like:

```
UniRide authentication diagnostic
================================

Database: bus_reservation_system

Passengers: ...
University admins: 3
System admins: 1

Password-hash verification:

samiha.tasnim@g.bracu.ac.bd: OK
samir.hossain@g.bracu.ac.bd: OK
...
bracu.transport@gmail.com: OK
nsu.transport@gmail.com: OK
aiub.transport@gmail.com: OK
uniride.admin@gmail.com: OK
```

If they show **OK**, the database/password side is working.

Delete `login-diagnostic.php` afterward.

Password reset now works too

The package also adds a functional localhost reset workflow:

```
Sign In
   ↓
Forgot password?
   ↓
Choose Passenger / Uni Admin / Sys Admin
   ↓
Enter database email
   ↓
Secure token generated
   ↓
Local XAMPP reset link appears
   ↓
Choose new password
   ↓
Password hash updated in correct table
   ↓
Return to login
```

It correctly updates:

```
Passenger
→ passengers.password_hash

University Admin
→ university_users.password_hash

System Admin
→ admins.password
```

That last difference is important because your current SQL uses **`password`**, not **`password_hash`**, for System Admin.

One more useful fix

The login now checks which dashboard file actually exists.

For passengers it checks:

```
passenger/dashboard.php
passenger_dashboard.php
dashboard/passenger.php
```

For university admins:

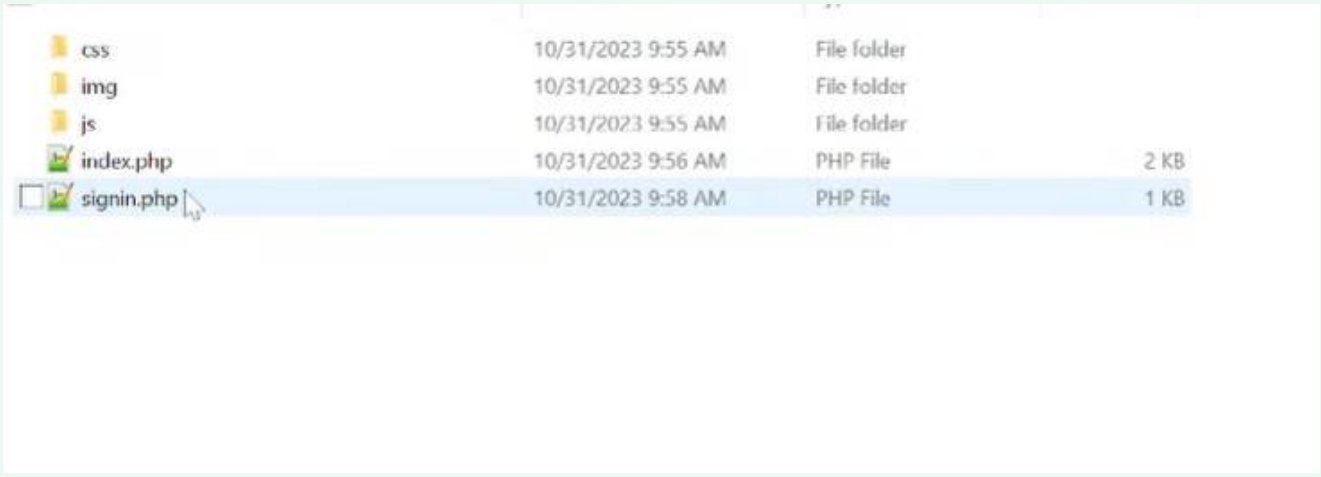
```
university/dashboard.php
university_dashboard.php
dashboard/university.php
```

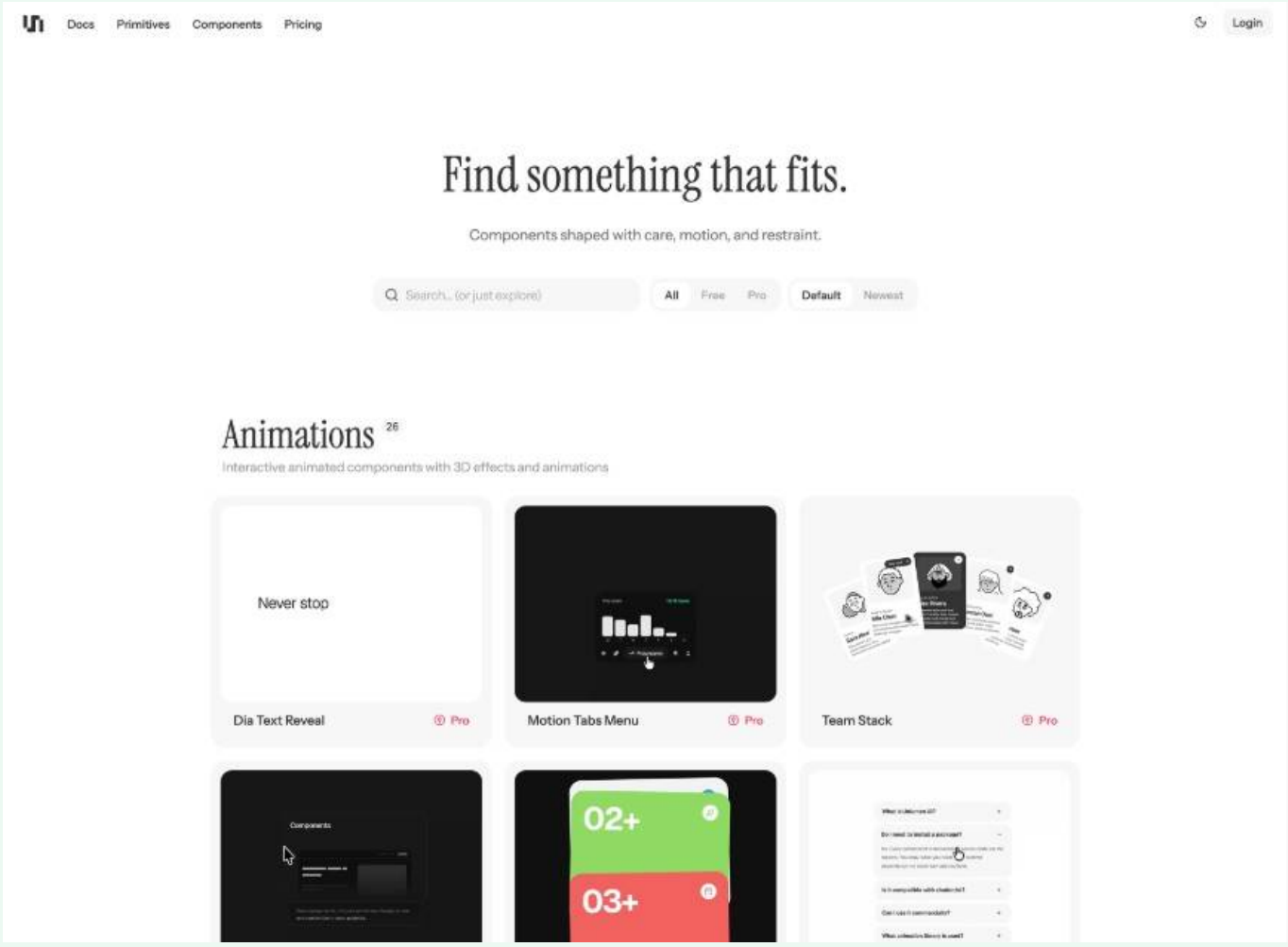
For System Admin:

```
admin/dashboard.php
admin_dashboard.php
dashboard/admin.php
```

If none exists, it sends the user safely to **`index.php`** instead of making a successful login look broken because of a 404.

Start by replacing **`config/database.php`** **and** **`signin.php`** **first**. Those two files fix the primary login failure. Then add the reset files and import **`authentication_patch.sql`**.





UniRide_fixed_database(2).sql
File

according to all your experience from this project till now,
give me a html and css file and other necessary files showed in "image 1" for me. use "ui" image for your css inspiration and the given dataset. and remember that the name of the dataset in the phpadmin website is "uniride2"

Done. I rebuilt the starter around the dataset you uploaded and configured it to connect to **`uniride2`**.

The uploaded SQL uses **`passengers.password_hash`** for Students/Faculty, **`university_users.password_hash`** for University Admins, and a different **`admins.password`** column for the System Admin, so the authentication code handles all three correctly.

UniRide_fixed_database(2) +1

The structure now looks like:

```
uniride/
├── css/
│   └── style.css
├── img/
│   └── logo.svg
├── js/
│   └── app.js
├── config/
│   └── database.php
├── includes/
│   └── auth.php
├── database/
│   └── password_reset_tokens.sql
│
├── index.php
├── signin.php
├── dashboard.php
├── forgot-password.php
├── reset-password.php
├── logout.php
├── login-diagnostic.php
└── README.md
```

The CSS follows the visual language from your UI reference: large editorial headings, lots of whitespace, small navigation, subtle gray controls, search/filter chips, restrained borders, and a clean card grid.

The homepage is also **database-driven**. It reads universities, routes, passenger counts, fares, and schedules directly from your **`uniride2`** tables rather than hard-coding them.

What to replace

Copy the ZIP contents into:

```
C:\xampp\htdocs\uniride\
```

If these files already exist, replace them:

```
index.php
signin.php
logout.php
forgot-password.php
reset-password.php
dashboard.php

css/style.css
js/app.js
config/database.php
```

Also add:

```
img/logo.svg
includes/auth.php
```

Important: database name

The new connection file explicitly uses:

```php
$dbName = 'uniride2';
```

So it matches the database name you told me is visible in phpMyAdmin.

One thing to be aware of: the SQL dump you uploaded still contains commands that create a database called **`bus_reservation_system`**. UniRide_fixed_database(2) That does **not** change this package; the website intentionally connects to **`uniride2`**. Just make sure your actual UniRide tables are inside **`uniride2`**.

Enable password reset

In phpMyAdmin, select:

```
uniride2
```

and import:

That adds only the reset-token table.

Login support

The login page supports:

```
Passenger
→ passengers
→ Students + Faculty

Uni Admin
→ university_users

Sys Admin
→ admins
```

The seeded passenger table contains accounts such as Samiha Tasnim and Samir Hossain with hashed passwords. UniRide_fixed_database(2) Your University Admin records for BRACU, NSU, and AIUB are also present in the database. UniRide_fixed_database(2)
The supplied hashes correspond to the development password:

```
password
```

So you can initially test, for example:

```
Passenger

Email:
samiha.tasnim@g.bracu.ac.bd

Password:
password
```

or:

```
University Admin
```

```
Email:
bracu.transport@gmail.com

Password:
password
```

or:

```
System Admin

Email:
uniride.admin@gmail.com

Password:
password
```

These are **not hard-coded into PHP**; the examples come from the supplied seed data.

Test before doing anything else

I included:

Open:

```
http://localhost/uniride/login-diagnostic.php
```

The first line should say:

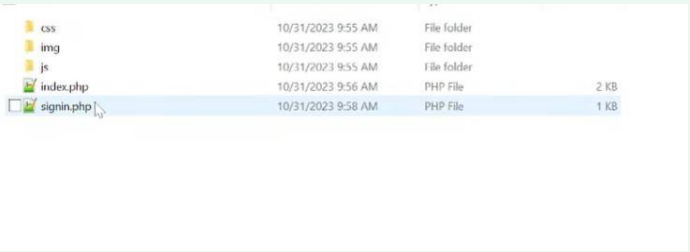
```
Connected database: uniride2
```

Then the login accounts tested should show:

```
OK
```

After checking it, **delete `login-diagnostic.php`**.

I also syntax-checked every PHP file in the package with PHP 8.x, and all of them passed.



now give me all the neccessary css html and other files for this like shown in the picture

Done. I packaged **all the necessary website files** in the same general structure shown in your folder screenshot, while keeping the project connected to **`uniride2`**.

The package contains:

```
uniride2_web_files/
├── css/
│   └── style.css
│
```

```
├── img/
│   └── logo.svg
│
├── js/
│   └── app.js
│
├── config/
│   └── database.php
│
├── includes/
│   └── auth.php
│
├── database/
│   └── password_reset_tokens.sql
│
├── index.php
├── signin.php
├── dashboard.php
├── forgot-password.php
├── reset-password.php
├── logout.php
└── README.txt
```

So the **main visible structure is still like your picture**:

```
css
img
js
index.php
signin.php
```

but I included the extra folders/files needed to make the website actually work with PHP/MySQL.

The database connection is already set to:

```php
$dbName = 'uniride2';
```

and authentication supports your existing database structure:

```
Passenger
→ passengers
→ STUDENT + FACULTY
→ password_hash

Uni Admin
→ university_users
→ password_hash

Sys Admin
→ admins
→ password
```

The CSS follows the UI reference you gave: large serif headings, lots of white space, very light gray surfaces, compact filters, minimal navigation, subtle borders, and clean cards.

Where to put it

Extract the ZIP and copy the contents into:

```
C:\xampp\htdocs\uniride\
```

Then your root should look roughly like:

```
C:\xampp\htdocs\uniride\

css\
img\
js\
config\
includes\
database\

index.php
signin.php
dashboard.php
forgot-password.php
reset-password.php
logout.php
```

Before testing password reset, select **`uniride2`** in phpMyAdmin and import:

Then start **Apache + MySQL** in XAMPP and open:

```
http://localhost/uniride/
```

I also ran PHP syntax validation on every PHP file in the package, and all of them passed.

> make this black slot same as other. give me the updated css file for this

Done. I removed the special black-card styling so that slot now looks the same as every other white route card.

Replace:

```
C:\xampp\htdocs\uniride\css\style.css
```

with the downloaded file.

You do **not** need to change **`index.php`**; the new CSS overrides the existing **`dark-card`** class and makes it visually identical to the normal cards.

> **style.css**
> File
> make this black slot same as other. keep the rest as it is

Done. I changed **only the black route-card styling** so that slot now matches the other white cards. The rest of your uploaded CSS is unchanged. The original stylesheet defines the special **`.dark-card`** rules separately from normal route cards, which is what caused the black appearance.

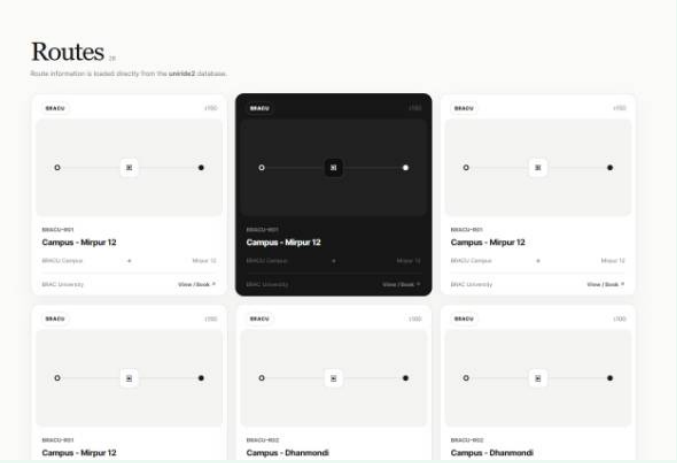
style

Replace only:

```
C:\xampp\htdocs\uniride\css\style.css
```

with the downloaded file. No PHP, HTML, or JavaScript changes are needed.

Sources



style(1).css
File

make this black slot same as other. keep the rest as it is

Done. I changed only the special **`.dark-card`** styling that was making that single route slot black. Your uploaded CSS explicitly gave **`.dark-card`** a black background and white text, plus darker inner elements.

style

Replace only:

```
C:\xampp\htdocs\uniride\css\style.css
```

with this file.

Everything else is unchanged.

according to your understading, give me a readme file for github repository about this projecct

Done — I created a complete GitHub-ready **`README.md`** for your UniRide project.

It includes the project overview, features for Passenger/University Admin/System Admin, **`uniride2`** database setup, folder structure, authentication flow, password reset, database relationships, booking/billing/complaint/transfer features, DBMS concepts demonstrated, XAMPP installation instructions, security notes, and future improvements.

You can place it directly in the root of your GitHub repository as:

```
uniride/
├── README.md
├── index.php
├── signin.php
├── css/
├── js/
├── img/
└── ...
```

Thinking

ChatGPT can make mistakes. Check important info.